

Preventing Unwanted Ads and Harmful Visual Media for Children: Evaluating Solutions and Technologies

Alexandros Chrysogelos

Thesis submitted for the degree of
Master of Science in Artificial
Intelligence, option Engineering and
Computer Science

Supervisor

Bart Preneel

Vincent Rijmen

Assessors

Dave Singelee

Laurens Sion

Assistant-supervisors

José Ignacio Patiño Galdos

Rafael Gálvez Vizcaíno

© Copyright KU Leuven

Without written permission of the supervisor and the author it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilize parts of this publication should be addressed to the Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Leuven, +32-16-327700 or by email info@cs.kuleuven.be.

A written permission of the supervisor is also required to use the methods, products, schematics and programmes described in this work for industrial or commercial use, and for submitting this publication in scientific contests

Preface

I would like to express my sincere appreciation to my daily advisors, José Ignacio Patiño and Rafa Gálvez, for their consistent guidance and valuable feedback throughout the development of this thesis. Their support has been instrumental. I am also deeply grateful to my family for their unwavering encouragement and support during my Master's studies.

Alexandros Chrysogelos

Contents

Contents	ii
Abstract.....	v
List of figures and tables	vi
List of Abbreviations and Symbols.....	vii
1 Introduction.....	1
1.1 Problem Statement	1
1.2 Research Objectives and Scope	3
1.2.1 Implement a Client-Server Architecture for Visual Filtering.....	3
1.2.2 Evaluate and Enhance Detection Accuracy	4
1.2.3 Conduct a Comparative Benchmark of Filtering Technologies.....	5
1.2.4 Analyze System Responsiveness and Latency	5
1.3 Contribution of the Thesis	6
1.4 Structure of the Thesis	7
2 Literature Review	9
2.1 Introduction to Automated Content Moderation.....	9
2.2 Architectural Paradigms in NSFW Visual Filtering	10
2.2.1 The Challenge of Client-Side Filtering.....	10
2.2.2 Server-Side and Hybrid Object Detection Systems	11
2.2.3 Emerging Approaches: Multimodal Large Language Models (MLLMs).....	14
2.3 Identifying the Research Gap.....	15
2.4 Rationale for Technology Selection: The YOLO Framework	16
2.4.1 Analysis of Candidate Models: “EraX-NSFW-V1.0” and “NudeNet”	17
2.4.2 Justifying Model Selection: A Quantitative Comparison.....	17
2.5 Chapter Summary	20

3	Methodology and Implementation Details.....	21
3.1	Architectural Design and Rationale	21
3.1.1	Justification for the Chosen Architecture: Navigating Technical Constraints .	21
3.1.1.1	Browser Environment Constraints	22
3.1.1.2	Platform Focus Rationale.....	23
3.1.1.3	Infeasibility of Network-Level Visual Filtering.....	23
3.1.1.4	Infeasibility of a Cloud-Hosted API	24
3.1.1.5	Architectural Decision Summary and Performance Context.....	25
3.1.2	System Workflow.....	25
3.2	Implementation Details and Tools Used	27
3.2.1	Hardware.....	27
3.2.2	Software	27
3.3	Experimental Setup and Procedures	28
3.3.1	Operational Definition of NSFW Content	28
3.3.2	Implemented Filtering Logic and Adaptive Mechanisms.....	29
3.3.3	Comparative Benchmark Methodology	31
3.3.3.1	Benchmark Dataset and Environment	32
3.3.3.2	Tools Configuration	33
3.3.4	Exclusion of Advertisements from Analysis.....	35
3.3.5	Accuracy Evaluation Procedure.....	35
3.3.6	Latency Measurement Procedure.....	37
3.4	Performance Metrics	39
3.4.1	NSFW Filtering Evaluation	39
3.4.2	Responsiveness Metric.....	40
3.5	Chapter Summary	40
4	Results and Analysis	41
4.1	Baseline Performance of the “EraX-NSFW-V1.0”: A Universal Threshold Approach.....	41
4.2	Enhancing Detection with Context-Aware Adaptive Filtering	45
4.3	Comparative Benchmark Against Alternative Filtering Solutions.....	50
4.4	Analysis of System Responsiveness and Latency	56
4.5	Chapter Summary and Key Findings.....	60

5	System Analysis: Constraints, Limitations, and Ethical Considerations	61
5.1	Foundational Constraints of the Research	61
5.1.1	Architectural Constraints	61
5.1.2	Resource and Experimental Scope Constraints	62
5.1.3	Methodological Limitations for Data Collection	63
5.2	Operational and Security Limitations of the Implemented System	63
5.2.1	Specialization and Scope of Content Detection	63
5.2.2	Architectural Performance and Latency Trade-Offs	64
5.2.3	Architectural Inability to Analyze Ad Content	65
5.2.4	Inherent Limitations and Vulnerabilities of the AI Model	65
5.2.5	Security Vulnerabilities of the Client-Server Architecture	65
5.2.6	Contextual Challenges of the Adaptive Logic	67
5.2.7	Platform and Environment Scope	68
5.2.8	Vulnerability to Heuristic Evasion	68
5.2.9	Synthesizing the Trade-Offs: The Inherent Tension Between Compliance, Security, and Performance	68
5.3	Privacy and Ethical Considerations	69
6	Conclusion and Future Work.....	71
6.1	Summary of Findings and Contributions	71
6.2	A Strategic Roadmap for Future Research and Development	72
6.2.1	Enhancing Performance and User Experience	72
6.2.2	Expanding Protection Scope	74
6.2.3	Addressing Security and Ethical Challenges	75
6.2.4	Long-Term Technological and Architectural Evolution	76
6.3	Concluding Remarks.....	77
A	The First Appendix: AI and Machine Learning Concepts.....	79
	The Second Appendix : System, Web, and Browser Technologies	81
	Bibliography	83

Abstract

The escalating exposure of children to harmful online visual media reveals a critical gap in current safeguarding technologies. Existing filtering solutions often struggle with the dynamic nature of the modern web or face obsolescence due to evolving browser security standards like Manifest V3. To address this challenge, a hybrid client-server filtering system was implemented and evaluated. The architecture uses a Manifest V3-compliant browser extension that communicates with a local backend API hosting the “EraX-NSFW-V1.0” object detection model. The system also integrates a lightweight, context-aware adaptive technique to enhance the pre-trained model’s accuracy by dynamically adjusting its sensitivity based on textual and visual cues, avoiding the need for costly retraining.

Empirical evaluation demonstrated that the developed adaptive system provided a more effective and balanced solution compared to an approach using a single, fixed sensitivity setting for all content. A controlled comparative benchmark against alternative filtering tools further contextualized the system’s performance by stressing the trade-offs between different architectures. A detailed latency analysis quantified the system’s viability, showing it can intervene within the thresholds of human perception while also identifying the architectural trade-offs necessary for Manifest V3 compliance.

Ultimately, the research validates a technically pragmatic and architecturally sustainable approach for a responsive content filtering system. It identifies the key usability and security trade-offs of its architectural choice to host the processing server locally; a decision made to preserve user privacy and avoid prohibitive costs and quantifies the performance overhead of the client-server paradigm that was made necessary by Manifest V3. The primary contributions are the validation of the Manifest V3-compliant architecture, the successful empirical evaluation of a lightweight adaptive technique, and a comprehensive benchmark that provides valuable insight into the architectural and performance trade-offs between different filtering technologies. The work paves the way for more robust and intelligent online child safety solutions.

List of figures and tables

List of Figures

<i>Figure 1: Workflow</i>	26
<i>Figure 2: The experimental setup showing the key software components: the browser client, the tunneling service, and the local backend API</i>	28
<i>Figure 3: An example of the system’s high recall, correctly identifying and blurring explicit content</i>	48
<i>Figure 4: An example of the system’s high precision, correctly allowing safe content to be displayed</i>	48
<i>Figure 5: The “NSFW Filter” at low sensitivity (5%) on a safe website, resulting in several False Positives</i>	54
<i>Figure 6: Distribution of “Time to First Intervention” on explicit content domains</i>	56
<i>Figure 7: The implemented “Safe Search” enforcement feature</i>	74
<i>Figure 8: The user-configurable domain blocklist for manual control</i>	75

List of Tables

<i>Table 1: Performance comparison of YOLOv11n and YOLOv8n variants [95, 96]</i>	18
<i>Table 2: All classes of “NudeNet” [35]</i>	33
<i>Table 3: Baseline performance of the universal threshold approach on “Mixed-Content” data</i>	43
<i>Table 4: Performance of the Context-Aware adaptive filtering system</i>	47
<i>Table 5: Comparative performance benchmark of filtering solutions</i>	52
<i>Table 6: Summary of core architectural and performance trade-offs</i>	72

List of Abbreviations and Symbols

Abbreviations	Definition
---------------	------------

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CDN	Content Delivery Network
COCO	Common Objects in Context, large-scale object detection dataset introduced
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSP	Content Security Policy
DOM	Document Object Model
FN	False Negative
FNR	False Negative Rate
FP	False Positive
FPR	False Positive Rate
GPU	Graphics Processing Unit
I/O	Input/Output
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
ML	Machine Learning
mAP	mean Average Precision
MLLM	Multimodal Large Language Model
MV2	Manifest V2
MV3	Manifest V3
NSFW	Not Safe For Work
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
Regex	Regular Expression

SFW	Safe For Work
SSL	Secure Sockets Layer
TN	True Negative
TP	True Positive
TPR	True Positive Rate (also known as Recall)
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
ViT	Vision Transformer
WASM	WebAssembly
XAI	Explainable AI
YOLO	You Only Look Once

Symbols

%	Percentage
B	Billion
GHz	Gigahertz
M	Million
ms	Millisecond
px	Pixel
s	Second

1 Introduction

This chapter establishes the foundational framework for the thesis. It starts with establishing a broader context regarding children’s engagement with the internet and the associated complexities that arise. Subsequently, the chapter will articulate the specific challenges this study aims to address, detail the precise objectives guiding the research, and outline the novel contributions of the work. Finally, a roadmap will detail the structure of the chapters to guide the reader through the thesis.

1.1 Problem Statement

The internet has become an indispensable part of children’s lives, serving as a source of learning, entertainment, and communication. However, the digital landscape was not originally designed with child safety as a primary concern [1]. As digital platforms continue to grow and change, it has become more urgent than ever to protect young users from harmful visuals, including explicit content and nudity [2]. This matter is not a trivial concern, as the psychological impact is well-documented; systematic reviews have found that excessive pornography consumption can distort adolescents’ perceptions of relationships and body image, while other studies link it to higher levels of anxiety, depression, and low self-esteem [3, 4]. These risks frequently appear on popular sites among minors, including YouTube and Instagram, where content moderation systems sometimes fail to filter out inappropriate material effectively [5].

Parental control tools like “Google Family Link” attempt to address these challenges but often exhibit significant limitations in dynamic online environments [6, 7]. Furthermore, academic security analyses have documented that many such tools function as “surveillance disguised as protection”, introducing serious security and privacy risks by collecting sensitive data without adequate safeguards [8]. Many rely on fixed filtering techniques or simple blocking of certain websites, which are often ineffective against the primary challenge: harmful visual media embedded directly within otherwise safe pages, including both explicit user-generated content and malicious advertisements [9, 10]. The ineffectiveness of such techniques is not merely theoretical; recent studies confirm that platform-native moderation systems are porous. For example, a 2025 study on major video-sharing platforms found that 15% of algorithmically recommended content to child-aged accounts was harmful, with exposure occurring during passive browsing and without any user-initiated searches for problematic material [5]. Authoritative data confirms the scale of early exposure. For example, a Canadian study cited

by Alberta Health Services found that the average age of first exposure to pornography was around 12 years, with a third of adolescents being exposed by age 10 [11]. Consequently, children remain exposed to inappropriate visuals despite ongoing efforts [12]. This continued exposure is amplified by the constantly changing nature of the digital landscape, where comprehensive regulation and consistent enforcement remain ongoing challenges [13, 14].

This situation highlights a critical gap: the need for intelligent solutions that can dynamically analyze visual content in real-time as it appears on users’ screens [15]. AI-driven image recognition solutions show promise in addressing this challenge, as they can detect harmful content with greater accuracy than traditional techniques [16]. However, while the potential of such solutions is clear, this thesis isolates the critical technical bottleneck that currently prevents their widespread implementation: the obstructive limitations imposed by modern browsers. Specifically, running powerful machine learning models directly within a browser extension is not possible under current standards, particularly Google’s Manifest V3, a framework governing how browser extensions interact with the browser and its resources, imposing significant constraints [17, 18, 19]. Therefore, while the societal need for more effective filtering is well-documented, this thesis posits that the primary technical bottleneck preventing the deployment of next-generation, AI-driven visual analysis is the architectural paradigm shift enforced by browser standards like Manifest V3. The central research problem is thus not merely to develop a better classifier, but to engineer a performant and architecturally viable system capable of operating effectively within this newly constrained digital ecosystem.

To overcome this architectural challenge, this thesis proposes and evaluates a pragmatic client-server system. The implemented solution consists of a browser extension that captures visual media, and a local backend API designed to host the “EraX-NSFW-V1.0” machine-learning model (model used in this thesis) [20]. By offloading the intensive visual analysis to the backend, this approach successfully navigates the constraints of Manifest V3, aiming to provide robust, dynamic protection where traditional methods have proven insufficient. Although the proposed architecture could have future applications in areas like visually-driven ad-blocking, the core technical investigation presented herein focuses specifically on the real-time visual analysis of embedded media, a decision driven by the unique and complex architectural challenge this task presents under Manifest V3 [17]. Such a focus was necessary, as it addresses a different challenge than traditional ad-blockers (which primarily use filter lists and are often architecturally prevented from analyzing visual content rendered within isolated, cross-origin advertisement containers [21]), a limitation discussed in detail in Section 5.2.3.

1.2 Research Objectives and Scope

This research pursued a structured set of goals to address the complex challenge of protecting children from harmful online visual media. These objectives guided the work through its foundational implementation, subsequent enhancement with a context-aware filtering engine, and final validation against other technologies. The following subsections articulate each of these specific aims in detail.

1.2.1 Implement a Client-Server Architecture for Visual Filtering

The core objective of this research was to implement a functional client-server system for visual content filtering. The system's design was tailored to operate under the technical constraints imposed by Google's Manifest V3. This makes the Manifest V3 environment a critical focus, particularly as Chromium browsers like Google Chrome, Opera, and Brave (which collectively account for the vast majority of the desktop market share [22]) have phased out the previous Manifest V2 standard, as its full deprecation occurred in June 2025 [23]. While Mozilla Firefox currently maintains support for both Manifest V2 and V3 standards [24], this research focused on the more restrictive Chromium environment to address the significant technical challenges it presents for advanced extensions.

The implemented architecture consists of two primary components: a browser extension responsible for capturing visual elements (like images and GIF-video frames) and a backend Python Flask API (a backend server built in the Python language using Flask, a lightweight framework designed to simplify the creation of web applications that can receive and process data requests [25]) designed to host the machine learning model. While this architecture effectively navigates technical constraints, its design introduces important trade-offs that are analyzed in detail in Section 5.2. These trade-offs include not only the usability challenges of running a local server but also crucial security considerations.

To enable this communication while adhering to browser security policies, a tunneling service was employed to generate an ephemeral, public HTTPS endpoint (a secure web address for the API) (e.g., https://*****trycloudflare.com) that forwards requests to the Python API on the user's computer. This method, while necessary, exposes the local application to the internet, creating a potential attack surface that must be managed (for instance, a malicious actor on the internet could attempt to send crafted requests to the public address to probe for vulnerabilities or crash the local server) [26]. This approach was necessary to address several key constraints simultaneously. First, it satisfies the "host_permissions" requirement of Manifest V3 (which requires an extension to explicitly declare which domains it can communicate with) [27]. Second, it prevents "Mixed Content Blocking" (a browser security feature that stops a secure webpage from loading content from an insecure server) [28]. Finally, it addresses restrictions from both Cross-Origin Resource Sharing (CORS, a policy that prevents a web page from making requests to a different domain than the one it came from) and the Content Security Policy (CSP, a standard that controls which resources a page is allowed to load to prevent code

injection attacks) [29, 30]. For its detection capability, the system leverages the pre-trained YOLOv11 “EraX-NSFW-V1.0” model [20]. The nano variant of this model was selected to prioritize faster inference speeds, a critical factor in ensuring the system’s total processing time remains under the human perception threshold [31].

1.2.2 Evaluate and Enhance Detection Accuracy

A further objective was to evaluate the system’s effectiveness and then explore a practical method for its enhancement. The research first aimed to establish a performance baseline for the system’s accuracy by testing the complete end-to-end system on a live, uncontrolled browsing dataset. This involved measuring its effectiveness under realistic conditions using a single, universal confidence threshold, a method where the system’s sensitivity is controlled by a single value (for example, a threshold of 0.5 requires the model to be at least 50% confident before it flags an image as NSFW).

The second step of this objective was to implement and test an enhancement to this baseline performance. It is important to note that retraining the AI model itself was determined to be outside the scope of this research. This was due to prohibitive barriers such as the immense financial cost (e.g., the high cost of cloud services [32]) and computational resources required (for instance, the developers of the “EraX-NSFW-V1.0” model reportedly used eight high-performance GPUs for training [20, 33]) and the difficulty of acquiring a massive, diverse, and ethically sourced dataset of labeled images for this sensitive task [15].

Consequently, the primary goal was to enhance detection accuracy by augmenting the system’s effectiveness at a system-level, rather than by retraining the model itself. The research leveraged the implemented client-server architecture to achieve this. In an attempt to capture the context of a webpage that might be rich in harmful material (e.g., an adult content website), the research focused on developing and testing a context-aware adaptive filtering technique. The objective of this technique was to make the system more intelligent by using clues from the page to dynamically adjust the model’s sensitivity. This was accomplished through three distinct mechanisms, without retraining the model itself. The first two mechanisms use regular expressions (a sequence of characters that specifies a match pattern in text [34]) to analyze textual clues, specifically, one scans the domain URL for high-risk keywords while the other checks an image’s alt text (a brief textual description of an image). The third mechanism is a model-driven feedback loop where a high-confidence NSFW detection on any single image is treated as a strong signal that the entire website is high-risk, increasing the system’s sensitivity for all subsequent images processed during that session.

1.2.3 Conduct a Comparative Benchmark of Filtering Technologies

A key objective of the research was benchmarking the implemented system’s performance against alternative filtering technologies and architectures. To achieve this objective, the evaluation involved a comparative analysis against two distinct types of alternatives: a similar pre-trained model using the same client-server architecture (“NudeNet”) and a client-only solution (“NSFW Filter”) [35, 36]. Such an approach allowed for a direct comparison of different machine learning models’ capabilities while also contrasting the performance and architectural implications of the two system designs. To ensure the accuracy and fairness of these comparisons, the tests were conducted in a controlled testing environment involving two custom-built websites: one hosting SFW images and the other hosting NSFW images. The underlying machine learning models and tools concerning the research are reviewed in the literature in Chapter 2 and compared in Chapter 4.

1.2.4 Analyze System Responsiveness and Latency

Finally, the research aimed to analyze the system’s practical viability by evaluating its responsiveness under real-world conditions. Because the system’s perceived speed is a critical component of the user experience, a key objective was to measure its latency to determine if it could operate within the thresholds of human perception, making the filtering feel effectively real-time [31]. The key metric for this analysis is the “Time to First Intervention”, a measure of the system’s initial reaction speed. The decision to focus exclusively on the first visual element processed on a page is based on two complementary reasons: one philosophical and one methodological.

Philosophically, from a child safety perspective, preventing the first unexpected exposure to harmful content is the paramount objective. Research on the psychological impact of such events confirms that an initial, unwanted encounter with explicit material can cause significant negative emotional reactions, including shock, disgust, and confusion [37]. This metric, therefore, directly quantifies the system’s ability to prevent that initial harmful exposure from being fully processed by the user.

Methodologically, measuring any image after the first would produce an unreliable result for reaction speed. Due to the need for synchronous DOM manipulation on JavaScript’s single main thread, each subsequent image is placed in a queue and must wait for the previous one to complete [38, 39]. Its total processing time would therefore be contaminated by a variable queuing delay, not just the system’s true processing speed. Focusing on the first element provides the cleanest and most repeatable measurement of the system’s core responsiveness.

A complete analysis, however, required more than just measuring this total duration. A further objective was therefore to deconstruct the “Time to First Intervention” into its primary components, including the overhead from client-side processes and the backend processing time. The client-side processes include DOM traversal (the extension searching the webpage’s structure to find images) [40]. After, canvas rendering (drawing the image data in-memory)

[41], and Base64 encoding (converting the image to a text string) [42]. The backend processing time is primarily composed of the model’s inference time, which is the duration the AI model takes to analyze the image.

1.3 Contribution of the Thesis

The escalating exposure of children to harmful visual content online highlights critical gaps within current safeguarding technologies [2, 5]. While various filtering tools exist, they often struggle with the dynamic nature of the modern web or face obsolescence due to evolving browser security standards like Manifest V3 [17, 23]. This research addresses the need for a resilient and intelligent system by proposing, developing, and evaluating a novel hybrid filtering system. The primary contributions of this work include the following:

- 1 **Quantitative Analysis and Validation of a Manifest V3-Compliant Paradigm:** A primary contribution is the validation of the local client-server model as an architecturally sustainable solution to the constraints of Manifest V3. The research demonstrates this paradigm’s viability by achieving intervention speeds within the thresholds of human perception. Critically, the latency analysis provides a quantitative breakdown of this performance, identifying that the most significant bottleneck in the implemented system was not the AI model’s inference time, but the substantial architectural overhead (~200 ms) required by the client-side data pipeline.
- 2 **Empirical Validation of a Resource-Efficient Enhancement Technique:** This thesis validates a lightweight, context-aware adaptive logic as a practical, system-level method for significantly improving a pre-trained model’s accuracy. This addresses the need for effective performance gains without incurring the prohibitive financial and computational costs of model retraining.
- 3 **A Strategic Analysis of Architectural Trade-Offs:** The thesis provides a comprehensive benchmark that delivers strategic insights into the inherent performance trade-offs between client-only, server-based, and adaptive filtering architectures, contextualizing the effectiveness of the proposed system against established tools.

1.4 Structure of the Thesis

This thesis is organized into the following chapters. Chapter 2 provides a comprehensive literature review of the technological landscape for NSFW content filtering, establishing the research gap this work addresses. Chapter 3 details the methodology, including the system's architecture, the experimental setup, and the performance metrics used for evaluation. Chapter 4 presents the core empirical findings, detailing the results of the baseline tests, the adaptive system evaluation, the comparative benchmark, and the latency analysis. Chapter 5 provides a critical analysis of the project, discussing the architectural constraints, the limitations of the final system, and the relevant ethical considerations. Finally, Chapter 6 concludes the thesis by summarizing the key findings and outlining potential avenues for future work.

2 Literature Review

This chapter provides the foundational context for the thesis by conducting a comprehensive review of the technological landscape surrounding the automated detection of NSFW visual content. It begins by establishing the societal and technical challenges of content moderation, then explores the recent architectural paradigms that have been developed, from client-side solutions to server-side systems and emerging multimodal models. Following this review, the chapter emphasizes a specific area where existing research falls short. Finally, it demonstrates the technical background for the core technologies selected to address this gap, justifying their viability for the system proposed in this thesis.

2.1 Introduction to Automated Content Moderation

The expansion of the digital landscape and the corresponding surge in user-generated content present a significant challenge for online platforms attempting to maintain safe and respectful environments [43]. A primary concern within this landscape is the filtering of Not Safe For Work (NSFW) material, which includes explicit, violent, and other forms of inappropriate imagery [43, 44]. While such content affects all users, research documents its particularly profound and negative impact on the psychological well-being of children and adolescents [45, 46]. This issue is further stressed as studies show the age at which minors first encounter sexually explicit materials has been steadily decreasing [44, 47, 48].

The scale of this challenge renders traditional manual moderation methods ineffective [43]. Human-led efforts are not able to handle the volume of modern content uploads and simultaneously expose individuals to potentially harmful and uncomfortable material. Consequently, an urgent need arises for effective automated solutions that employ artificial intelligence [49]. These systems can detect and filter sensitive images, therefore creating a healthier and safer online environment for underage users. Platforms have several motivations for content moderation, including maintaining user trust and ensuring brand safety. An additional factor is to comply with legal standards, especially regulations designed to protect children from exposure to harmful explicit content [43, 50].

However, despite the shift to AI, current automated techniques often prove inadequate. For instance, researchers note that many existing methods face substantial limitations in detection accuracy, particularly when harmful content only occupies a small part of an image or is partially obscured by the background [49]. Such challenges create a critical deficiency in the ability to reliably protect children and encourage further research into more sophisticated and robust filtering systems [44, 49].

2.2 Architectural Paradigms in NSFW Visual Filtering

The challenge of filtering harmful online material has provoked the development of diverse automated systems [51]. Research in this area has branched into several distinct architectural paradigms, ranging from client-side solutions that run entirely within a user’s browser to more powerful server-side and hybrid systems [52, 53]. Recently, the field has seen revolutionary innovation centered on large-scale, multimodal AI models [50]. Each of these approaches offers a unique trade-off between privacy, performance, and deployment complexity [32, 52]. Therefore, this review will categorize and examine these primary techniques to build a comprehensive picture of the current technological landscape.

2.2.1 The Challenge of Client-Side Filtering

A prominent example of a client-side methodology is the “NSFW Filter” browser extension, which is powered by the “NSFWJS” library (released in 2020) and is able to blur an image if detected as unsafe [54]. To perform within the browser environment, the library can leverage two well-regarded image recognition models, specifically InceptionV3 and MobileNetV2. Upon their release, both models were considered highly efficient. InceptionV3 was known for combining high accuracy with speed, while MobileNetV2 was specifically designed for fast performance on devices with limited computational resources [55, 56]. According to the developers of “NSFWJS”, the accuracy of the two models fluctuates between 90% and 93% for the task of categorizing images into five distinct classes: “Drawings” (safe artwork, including anime), “Neutral” (safe general images), “Hentai” (explicit drawings/anime), “Porn” (explicit photographic images, sexual acts), and “Sexy” (suggestive but non-explicit images) [57].

However, the critical limitation of this approach is its reliance on the Manifest (a framework governing how browser extensions interact with the browser and its resources, which imposes significant constraints) V2 browser standard [17], which has been phased out in Chromium browsers, with its support having ended in June 2025 [17]. The mandated transition to the more restrictive Manifest V3 makes this client-only approach technically challenging to maintain in Chromium browsers [18]. Furthermore, the extension appears to be no longer actively maintained, with its last update being two years ago [36]. Such reliance on a soon-to-be-obsolete technology makes this type of client-only filter an unsustainable solution for long-term use on modern browsers.

The fundamental architectural challenge extends even to more advanced client-side techniques. One might argue that a solution could use Web Workers to handle heavy computations in the background, thus preventing the user interface from freezing [58]. While Web Workers do allow for such background processing, the security model of Manifest V3 prevents them from directly seeing or interacting with the images on a webpage [40, 58]. This means the essential first step of capturing image data must still happen on the main browser thread, leaving a critical performance bottleneck that further justifies the need for the client-server architecture explored in this thesis.

Even more advanced client-side technologies are ultimately rendered non-viable by a fundamental architectural barrier in the form of the browser’s Same-Origin Policy (a core security rule that prevents a script from one domain from accessing data from another) [59]. For instance, an architecture using WebAssembly (WASM), which is a binary instruction format that allows code to run in the browser at near-native speed, could theoretically offer significant advantages such as eliminating network latency and maximizing user privacy [60]. However, the Same-Origin Policy makes such a universal client-side filter impractical. The policy forbids a tool running in the browser from accessing the pixel data of most images it would need to analyze. Since the modern web is built on serving content from many different domains using Content Delivery Networks (CDNs) (globally distributed networks of servers that deliver web content based on the user’s geographic location) [61], this restriction renders a purely client-side filter ineffective, as it is architecturally forbidden from analyzing the cross-origin images that require inspection.

It is worth noting that Mozilla Firefox currently maintains support for the older and more flexible Manifest V2 standard, which could theoretically allow for different extension architectures [24]. Despite this, this research intentionally focuses on the more restrictive Chromium/Manifest V3 environment. This decision was made due to the dominant market share of Chromium-based browsers and the specific research goal of confronting the significant technical challenges posed by the industry-wide shift to Manifest V3 [22, 62].

2.2.2 Server-Side and Hybrid Object Detection Systems

To overcome the fundamental architectural and security barriers that render client-side filtering non-viable, research has logically progressed towards server-side and hybrid architectures. Instead of performing analysis on a user’s device, a powerful strategy employs specialized object detection models within server-side or hybrid architectures [51]. The field of computer vision has historically been dominated by Convolutional Neural Networks (CNNs, which also form the basis for the YOLO framework) [63]. However, two architectural paradigms have become particularly influential in shaping the current landscape: CNNs and the more recent Vision Transformers (ViTs) (a class of models that apply the Transformer architecture, originally from natural language processing to visual tasks) [63].

In contrast to the hierarchical filtering of CNNs, ViTs process an image by dividing it into patches and analyzing them as a sequence, using self-attention mechanisms (a technique that allows the model to weigh the importance of different sections of the image to better capture their overall context) to effectively model long-range, global relationships between image sections [63]. While traditional ViTs were known for being computationally intensive, it is important to acknowledge that recent research has produced lightweight variants (e.g., MobileViT, EdgeViT) demonstrating competitive real-time performance, challenging the dominance of CNNs in applications where speed is critical [64, 65]. However, the primary goal of this thesis was not to conduct a theoretical comparison of emerging architectures, but to identify a proven, accessible, and pre-trained model specifically optimized for NSFW content detection on consumer-grade CPUs. A survey of the landscape for such specialized tools revealed that the most mature and readily available models were predominantly based on the

YOLO framework, which belongs to the CNN family [20, 35]. The selection of a YOLO-based model was therefore a pragmatic decision, driven by the current state of available tools rather than a claim of absolute architectural superiority.

Given that the YOLO framework used in this thesis belongs to the CNN family, it is important to understand the distinctions within this paradigm. Within the well-established CNN approach, a foundational distinction in object detection lies between single-stage and two-stage detectors. Unlike two-stage detectors, which involve separate steps for proposing regions and then classifying objects within them, single-stage detectors perform both tasks simultaneously in a single forward pass, which streamlines the detection process [66].

Single-stage object detectors are known for their speed, as they process an image in a single forward pass to identify objects. This method enables the precise detection of sensitive regions within an image, rather than classifying the image as a whole. Numerous studies have validated this direction by evaluating detectors like YOLOv3 (YOLO: “You Only Look Once”, as its name suggests, the core idea is to look at an image only once to comprehend what is in it and where) an earlier version of the YOLO framework, for pornography detection [51].

Other notable example is the Single Shot MultiBox Detector (SSD), which efficiently detects objects of various sizes within an image [67]. Advanced models like EfficientDet (a single-stage object detector known for efficiently balancing accuracy and speed across various scales) further demonstrate how to balance speed and good accuracy, even on everyday computing devices [51, 68]. The primary advantage of these single-stage detectors lies in their fast identification capabilities, making them suitable for dynamic online environments where fast content filtering is important [67, 69].

Conversely, two-stage object detectors follow a two-step process. They start by identifying potential regions of interest within an image, then they classify and refine those regions, typically resulting in highly precise detections [48, 70]. Such models can identify specific sensitive areas within an image, such as human body parts, in detail [53, 66]. A notable example is Faster R-CNN (a highly efficient iteration of the original R-CNN framework, which pioneered the use of region-based convolutional neural networks for object detection) [71]. Variations like Mask R-CNN (an extension of Faster R-CNN capable of precisely identifying and outlining specific objects within an image) extend this capability by precisely outlining objects through instance segmentation [53, 72]. Furthermore, models like Cascade Mask R-CNN can refine their detections multiple times through consecutive stages, leading to exceptionally high accuracy segmentation [53, 73]. Such a high level of detail and accuracy comes with a significant trade-off, as these models typically require more processing time than their single-stage alternatives [53, 71].

Beyond the core detector architectures, research has also explored more complex hybrid and multimodal systems. For instance, Phan et al. illustrated the hybrid approach by developing a multi-stage classifier that combines several techniques for a more nuanced judgment [53]. Their system first uses an object detector to find potentially sensitive body parts and then merges these findings with other high-level data, such as the presence of a person and the amount of visible skin. A final classification model, specifically a Support Vector Machine

(SVM: a classical machine learning algorithm effective at classification tasks [74]), processes this fused information to accurately predict if the content is pornographic [53]. To demonstrate the real-world applicability of this approach it was also implemented as a Google Chrome extension for web filtering [53].

Another research on hybrid systems from 2020 explored the integration of different data formats, such as combining visual and audio signals, to improve detection accuracy [75]. Specifically, Song & Kim (2020) proposed an “Enhanced Multimodal Stacking Scheme” for real-time online content detection [75]. Their method focused on extracting what they termed “implicative features”, which are representations designed to capture how patterns change over time. To achieve this for visual content, they first used a Convolutional Neural Network to extract features from video frames. These features were then processed as a sequence by a bi-directional Recurrent Neural Network (RNN: a type of neural network designed to recognize patterns in sequential data like text or video frames [76]) analyzing the sequence of frames both forward and backward, allowing it to capture the context of each frame based on what happened before and after it [75, 77]. The system’s architecture employed a “stacking ensemble” which consists of three distinct classifiers (one for visual data, one for audio, and one for their fusion) arranged in a specific order [75]. This sequential process was designed to reduce detection omissions by allowing a later classifier to capture what an earlier one might have missed, focusing on improving speed and accuracy for real-time content moderation at that time [75].

Finally, a different research strategy focuses less on combining external techniques and more on fundamentally enhancing the core object detection model itself. This model-centric approach aims to boost accuracy by integrating advanced components directly into a standard detector’s architecture. For instance, Zhang et al. demonstrated this method by proposing an improved YOLOv8 model for identifying sensitive images [49]. They refined the standard YOLOv8 architecture with new modules designed to better analyze features at multiple scales and adapt to objects of various sizes [49]. This represents a distinct approach for improvement, as it refines the detector’s intrinsic feature extraction capabilities instead of combining it with external techniques [49].

2.2.3 Emerging Approaches: Multimodal Large Language Models (MLLMs)

A notable recent innovation in content moderation involves leveraging the advanced capabilities of Multimodal Large Language Models (MLLMs) [50, 78]. Unlike traditional Convolutional Neural Networks (CNNs: deep learning models specialized for analyzing visual patterns), MLLMs are large-scale AI systems designed to process and reason information across multiple data types at once (for instance, images and text) [50, 66]. The application of these models to content moderation aims to solve a key challenge: general-purpose MLLMs, without specific training, often produce judgments that are inconsistent with the precise, rule-based standards required for platform safety (in other words, the specific written policies of a platform, such as rules against “sexy” or “horrifying” content, or a rule that “Disallows images with lower legs exposed”) [50]. As demonstrated by the “ICM-Assistant” project, a key benefit is the ability to fine-tune MLLMs through a process called instruction tuning (a method of fine-tuning a model on examples of specific tasks [50]), allowing them not to provide a simple classification, but a more nuanced and explainable moderation decision based on a given set of rules [50].

This research trend is validated by extensive studies benchmarking the vision capabilities of foundation models that are continuously being released, including those from OpenAI, Google, and Meta [44]. These studies suggest that MLLMs can outperform many traditional deep learning techniques in moderation tasks by achieving higher accuracy and lower error rates [44]. However, this often comes with a significant computational cost, encouraging researchers to explore alternative architectures [32].

Such an example is the Receptance Weighted Key Value (RWKV) model, which takes advantage of the strengths of RNNs with those of Transformers (advanced architectures that use sophisticated attention mechanisms, allowing them to weigh the importance of different parts of the input data to better capture context) [32, 79]. The development of efficient hybrid models aims to address the high resource demands of larger systems [32]. Ultimately, such a focus on efficiency reveals an important trade-off. The advanced reasoning capabilities of MLLMs often result in high computational demands, making them infeasible for the specific scope of this thesis: a real-time tool designed to run on common, consumer-grade hardware. This project targets typical desktop and laptop computers that rely on a standard CPU and do not have the specialized, high-end Graphics Processing Units (GPUs) required to run large models. For context, deploying even moderately sized MLLMs often requires enterprise-grade hardware, such as servers equipped with multiple NVIDIA A100 or V100 GPUs, and massive amounts of VRAM (64GB or more), resources that are far beyond the reach of a typical home user, who often relies on a computer with integrated graphics or a low-tier GPU with minimal VRAM (e.g., 2-4GB) [50, 80]. This fundamental mismatch establishes lightweight, specialized systems as the only practical solution for this project’s goal of creating an accessible, locally run safety tool.

2.3 Identifying the Research Gap

A review of the existing techniques for NSFW visual filtering reveals significant architectural limitations and specific opportunities for novel research. The literature shows that purely client-side solutions, while once effective for their privacy benefits, are becoming architecturally non-viable for Chromium browsers. This is due to the mandatory transition to the more restrictive Manifest V3 standard for Chromium browsers, which renders the aforementioned approach unsustainable [17]. Compounding this, the “NSFW Filter” browser extension, a client-side solution discussed earlier, exemplifies the challenges faced by such approaches due to its reliance on the outdated Manifest V2 standard and its lack of active maintenance [36], making it unresponsive to new threats or content trends. Furthermore, automated systems, especially lighter ones, frequently struggle with contextual understanding, cultural subjectivity, and detecting subtle or manipulated harmful content like deepfakes [66].

While server-side and hybrid object detection systems have been proposed, the focus of such studies is primarily on the performance of their classification and detection methods rather than on their practical deployment [53, 81]. Consequently, a notable gap exists in research that presents and empirically validates a pragmatic, end-to-end architectural blueprint for a Manifest V3-compliant browser extension designed for this filtering purpose.

Furthermore, recent research explores the use of powerful model architectures, from Vision Transformers (ViTs) to Multimodal Large Language Models (MLLMs), for their advanced reasoning and ability to provide explainable moderation decisions [44, 50, 63]. However, these advanced systems introduce a significant trade-off, as their high computational requirements present a challenge for lightweight, real-time applications [32]. It should be noted that this computational intensity, along with the strict architectural limitations imposed by modern browser environments such as Manifest V3, renders direct client-side execution of such massive models infeasible for a browser extension [17, 18, 50, 80]. Consequently, the literature reveals a critical gap not in the existence of powerful classifiers, but in the engineering of a balanced and deployable system: a solution capable of delivering enhanced performance without the significant overhead of large-scale models for real-time deployment. Therefore, this thesis attempts to address this gap by proposing and evaluating a novel system that combines a Manifest V3-compliant architecture with a context-aware adaptive technique, aiming to provide a practical, efficient, and robust solution for real-time NSFW content filtering.

2.4 Rationale for Technology Selection: The YOLO Framework

The previous section established an emerging need for a novel system to address the identified research gap, specifically an efficient and Manifest V3-compliant solution for real-time browser-based NSFW content filtering. To achieve this, the core technology selection is paramount and requires balancing detection capability and computational efficiency. This section is dedicated to the selection process. It provides the necessary technical background and a data-driven rationale for the initial choice of technology, which will be empirically validated in later chapters. Since the final training data for specialized models is often private, this rationale is based on a comparative analysis of their underlying public frameworks [15, 20, 35, 50].

For the purposes of this thesis, the “You Only Look Once” (YOLO) framework was selected as it is a well-established single-stage object detection technology [66]. It has become a leading approach in the field because it achieves an effective balance between detection speed and accuracy, making the framework particularly well-suited for real-time applications where fast processing is essential [66].

For a safety tool designed to prevent unwanted visual exposure, the definition of “real-time” performance is not arbitrary but is instead grounded in the practical limits of human perception. While a response time under 100 ms is perceived as instantaneous, this research aims to develop a system that can intervene faster than the average human reaction time of approximately 250 ms [31]. The objective of achieving this speed is to allow the system to act before a user can fully process potentially harmful content.

It should be emphasized that while the field of object detection evolves rapidly with newer architectures, this project leverages a proven, pre-trained model. Training a new model from scratch would require significant resources beyond the scope of this research, such as multiple high-end GPUs and massive, curated datasets [20]. The financial investment for such computational power is also considerable, as the operational costs for cloud infrastructure and specialized hardware can be substantial [32]. Furthermore, acquiring a suitable training dataset for this task presents immense ethical and practical challenges, as demonstrated by studies that found illegal material in major public datasets, leading to their removal [15]. This reality of the research landscape necessitates a significant ethical trade-off. While opting for a pre-trained model avoids the direct risks of curating a new dataset, it means accepting the inherent limitation that the private training data cannot be audited for the same potential biases or problematic content found in public datasets [15, 82].

2.4.1 Analysis of Candidate Models: “EraX-NSFW-V1.0” and “NudeNet”

One candidate model considered was “NudeNet” which is an effective model specialized for NSFW detection [35]. Published studies on its earlier versions suggest strong performance up to 93% for its specialized task [83, 84, 85]. The latest version of the model is built upon the lightweight and efficient smaller variants (nano, and small) of the YOLOv8 architecture. This foundation on such lightweight variants optimizes “NudeNet” for the high-speed processing required for real-time analysis. In terms of detection scope, the model is capable of identifying eighteen different anatomical classes such as: “FEMALE_GENITALIA_COVERED”, “ANUS_EXPOSED”, “FEMALE_BREAST_COVERED” and other anatomical body regions [35].

The second candidate model, and the one primarily used in this thesis, is “EraX-NSFW-V1.0”, a highly efficient model designed for NSFW detection [20]. Developed by “EraX AI”, its purpose is to provide control over harmful publications and limit children’s access [20], aligning with the project’s objectives. The model is built upon the powerful YOLOv11 architecture and is available in several variants (nano, small, and medium) to balance performance and resource needs. Each variant offers a specific trade-off, where smaller models like the “nano” version prioritize processing speeds, while the larger variants provide higher detection accuracy.

Its effectiveness stems from being trained on a large, private dataset of over 31,000 images, enabling it to robustly identify five explicit anatomical and situational classes: “anus”, “make_love”, “nipple”, “penis”, and “vagina”. While the model’s architecture provides the functionality to precisely blur only the detected harmful regions, this thesis implements a stricter protective measure. To ensure maximum protection and align with the project’s goals, a positive detection is used to trigger a blur of the entire image, immediately concealing the detected visual element from the user.

2.4.2 Justifying Model Selection: A Quantitative Comparison

To understand the foundations of the models used, it is helpful to review the key advancements in their underlying YOLO frameworks. The YOLOv8 architecture (on which “NudeNet” is based) introduced a major simplification by moving away from a rigid system of pre-defined boxes (also known as anchor boxes) [86]. Instead of trying to fit objects into fixed shapes, YOLOv8 directly predicts an object’s center and dimensions, giving it greater flexibility [86]. Furthermore, its architecture uses separate components to specialize in different tasks, one for identifying an object, and another for identifying its location improving overall precision [86, 87]. The subsequent YOLOv11 framework (on which “EraX-NSFW-V1.0” is based) further enhanced performance by focusing on efficiency and its improved ability to interpret complex scenes [88]. It achieves this through more advanced feature analysis and a refined attention mechanism, allowing the model to concentrate on the most critical regions of an image and

making it more effective at detecting partially obscured objects or in an unclear background [87, 88].

To quantitatively justify the selection of a model built on the more recent framework, this section compares the performance on public benchmarks of the nano-scale variants of YOLOv11 and YOLOv8, as published by their developers, Ultralytics [89, 90]. The focus on the nano-scale variants is intentional, as the project prioritizes real-time performance on common consumer hardware (meaning larger, more computationally expensive models would be unsuitable for the project’s accessibility goals). This decision was further solidified during preliminary testing, where larger model variants proved unsuitable for the test environment, causing system instability (e.g., temporary freezes) and demonstrating that only the most lightweight models could be reliably benchmarked. In this manner, contrasting the most lightweight versions of each framework is the most relevant way to assess their efficiency. It is important to note that the following performance data, which includes accuracy on the general-purpose COCO dataset, is used here for a relative comparison of the frameworks’ potential [89, 91]. The final accuracy of specialized models like “EraX-NSFW-V1.0” and “NudeNet” is dependent on their training processes and methodologies (including factors like the size and quality of their private training datasets and the hardware used for optimization, which are beyond the scope of this public benchmark comparison) [20, 35].

To ensure the benchmark is both fair and relevant to the project’s goal of broad accessibility, this analysis focuses on performance on common consumer-grade hardware. Therefore, the system is designed for typical desktop and laptop computers that rely on a standard Central Processing Unit (CPU) and does not make a specialized Graphics Processing Unit (GPU) a prerequisite. This design choice makes public benchmarks measuring CPU performance a highly relevant criterion for comparing the architectures of the models used in this thesis. One such benchmark is the Speed CPU ONNX metric [92].

While the specific CPU used is a primary factor in determining absolute processing speed, the public benchmarks from Ultralytics rely on the ONNX format to ensure a fair comparison. ONNX (Open Neural Network Exchange) is an open standard that allows models from different frameworks, like PyTorch or TensorFlow, to be run in a single, high-performance inference engine called the ONNX Runtime [93]. The use of a common inference engine for benchmarking provides a crucial methodological control [94]. This approach ensures different models are tested under consistent conditions, in the same optimized software environment, meaning performance differences are more likely to reflect the models’ inherent architectural efficiency rather than variations between their native software frameworks (e.g., different library versions or specific optimizations in PyTorch versus TensorFlow). The following table presents these public benchmark speeds, offering a direct comparison of the two foundational frameworks’ performance potential.

Model	Size (pixels)	mAPval 50-95 (Accuracy)	Params (M) (Complexity)	Speed CPU ONNX (ms)
YOLOv8n	640	37.3	3.2	80.4
YOLOv11n	640	39.5	2.6	56.1

Table 1: Performance comparison of YOLOv11n and YOLOv8n variants [95, 96]

A direct comparison between the nano-scale variants of YOLOv11 and YOLOv8 reveals the clear performance advancements of the newer framework. One of the metrics for judging detection capability is mAP (mean Average Precision). The “50-95” in the score means the model’s accuracy is averaged across many tests, from easy ones (requiring only a 50% correct match) to very strict ones (requiring a 95% correct match) [97]. It measures the model’s accuracy; a higher score indicates a better ability to correctly identify objects [97]. The YOLOv11n model achieves a mAP score of 39.5, a considerable improvement of nearly 6% over the 37.3 score of YOLOv8n (a comparison of potential on a general dataset, as the final performance on a specialized task like NSFW detection depends on the developers’ private training) [95, 96]. For the purposes of this thesis, the better accuracy (even a small) of the underlying YOLOv11n framework suggests that the EraX-NSFW-V1.0 model is potentially more reliable for correctly identifying harmful visual content.

Beyond accuracy, model efficiency can be assessed by the number of Parameters (Params), which reflects the model’s size and architectural complexity (a key factor for efficiency on common consumer hardware) [98]. A lower parameter count typically signifies a more lightweight and optimized design. The data shows that YOLOv11n is a simpler architecture, with only 2.6 million parameters compared to the 3.2 million in YOLOv8n [95, 96]. This reduction of approximately 19% demonstrates that the YOLOv11n framework achieves its higher accuracy without additional architectural complexity, suggesting a more advanced and efficient design.

For a real-time safety application, processing Speed is arguably the most critical metric for the user experience (the target hardware for this project, prioritizing accessibility over specialized GPUs) [81]. The Speed CPU ONNX (ms) value indicates how quickly the model can analyze an image on a standard CPU, where a lower number is desired [98]. In this aspect, YOLOv11n holds a substantial advantage, processing an image in just 56.1 ms, making it over 30% faster than YOLOv8n’s time of 80.4 ms [95, 96]. Such a performance gain translates directly to a more responsive filtering tool that can operate without introducing noticeable lag.

Collectively, the public benchmarks from Ultralytics’ official website demonstrate that the YOLOv11n framework’s combination of higher accuracy, reduced complexity, and superior speed marks it as a more advanced and capable technology than YOLOv8n [95, 96]. This potential for high efficiency is a critical factor for the developed system, as the restrictions of Manifest V3 necessitate a client-server architecture where a fast backend model is essential to minimize user-perceived latency [17, 18, 80]. It is crucial, however, to contextualize these public benchmarks. While valuable for assessing a framework’s foundational efficiency, they do not serve as definitive proof of performance on a specialized task [97]. The final effectiveness of models like “EraX-NSFW-V1.0” and “NudeNet” is critically dependent on factors not captured by such benchmarks, particularly the quality of their private training data [20, 35]. For this reason, the public benchmark data, bolstered by preliminary internal testing which suggested a similar performance trend, was used to form a testable hypothesis rather than a final conclusion. Based on this combined evidence, it was hypothesized that a specialized model built upon the YOLOv11n framework, namely “EraX-NSFW-V1.0”, would offer a more effective balance of detection accuracy and real-time performance for this project’s

requirements compared to a model based on the YOLOv8n framework. The controlled comparative benchmark presented in Section 4.3 was designed to empirically test this hypothesis, and its findings provide strong support for this initial model selection.

2.5 Chapter Summary

This chapter provides an in-depth review of the current technologies involved in automated NSFW visual content filtering. The research of existing literature revealed that purely client-side solutions are becoming unsustainable due to the wide adoption of Manifest V3 [23, 52]. Meanwhile, more advanced research has diverged into different paths, including creating complex hybrid systems that fuse visual features with other data modalities like audio [75], improving the internal architecture of the object detectors themselves [49], or leveraging powerful but computationally expensive, Multimodal Large Language Models [32, 50]. This analysis identified a critical research gap for a pragmatic browser-compliant system that enhances a pre-trained, lightweight model with a context-aware adaptive technique, offering a practical middle-ground solution. Having established this gap, the chapter then presented the necessary technical background. It surveyed the broader landscape of object detection architectures and provided a data-driven hypothesis for selecting a YOLOv11-based model as the most promising candidate for the real-time application developed in this thesis. The following chapter will detail the specific methodology used to design and implement this novel solution.

3 Methodology and Implementation Details

This chapter details the methodology used to design, implement, and evaluate the proposed visual content filtering system. It begins by explaining the system’s client-server architecture and the technical rationale behind its design (Section 3.1), followed by a list of the specific hardware and software tools employed (Section 3.2). The chapter then outlines the experimental procedures for evaluating the system’s accuracy and responsiveness (Section 3.3) and formally defines the performance metrics used for the analysis (Section 3.4). This chapter provides the necessary context for the examination and the results that follow.

3.1 Architectural Design and Rationale

The filtering system is built upon a client-server architecture, a design choice primarily driven by the need to operate within the technical constraints of contemporary web browsers, specifically, Manifest V3. The proposed system consists of a browser extension that operates in the browser (the client) and a locally hosted Python Flask API (the server). The following subsections will first justify this architectural decision by examining the specific limitations of the browser environment (Section 3.1.1) and then outline the complete system workflow (Section 3.1.2), which details the process from capturing visual elements in the browser to the final classification and blurring action.

3.1.1 Justification for the Chosen Architecture: Navigating Technical Constraints

The decision to use a locally-hosted client-server model was a two-stage process dictated by distinct constraints. First, fundamental browser security policies and the Manifest V3 standard made a purely client-side approach architecturally non-viable, thus necessitating a client-server paradigm (as detailed in Sections 3.1.1.1-3.1.1.3). Second, core project goals of user privacy and cost-effectiveness made a locally-hosted server the only practical implementation of that paradigm, an alternative to a cloud-based approach (Section 3.1.1.4). The following subsections detail the specific technical and project-level constraints that informed this final architectural choice.

3.1.1.1 Browser Environment Constraints

The design of the system was primarily shaped by security measures and operational frameworks inherent to contemporary web browsers. The most impactful of these is Google Chrome’s Manifest V3, which imposes powerful restrictions that render purely client-side solutions impractical for this task [17]. A key limitation is that Manifest V3 prevents the direct execution of complex machine learning models like “EraX-NSFW-V1.0” within the browser extension itself, as their size and processing requirements are beyond what is permitted in the constrained client-side environment [18, 19, 20, 99]. Furthermore, common client-side optimization techniques like Web Workers are not viable, as a specific Manifest V3 security constraint prevents them from directly accessing a webpage’s Document Object Model (DOM: a representation of the page’s structure that allows the extension to access and capture image elements), a necessary action for capturing image data for analysis [40, 58].

Even beyond the limitations of Manifest V3, a more fundamental browser security rule, the Same-Origin Policy, makes a universal client-side filter architecturally non-viable [59]. Because the modern web is built on serving content from different domains using Content Delivery Networks (CDNs) (globally distributed networks of servers that deliver web content based on the user’s geographic location) [61], this policy forbids an extension from accessing the pixel data of most images it needs to analyze.

Given these overlapping architectural barriers, a client-server architecture was adopted as the only effective and pragmatic strategy. In this model, the browser extension is responsible for client-side tasks like interacting with the webpage’s DOM to capture visual media and prepare it for analysis (e.g., through Base64 encoding, a method of converting binary image data into a text format for safe transmission), while the backend API handles the processing [39, 42].

This architecture, however, introduces the challenge of enabling secure communication between the browser extension and the local backend API, which operate on different origins. To navigate the browser’s security constraints, the system uses a viable, two-part solution that operates within the browser’s security model. First, a tunneling service creates a stable, public HTTPS address for the local API [100, 101, 102]. Second, and most importantly, this public URL is explicitly declared in the extension’s “manifest.json” file under the “host_permissions” key. This declaration, a necessary security step given that Manifest V3 does not permit direct permissions for localhost [103], grants the extension a specific, trusted channel to communicate with the pre-approved backend server. In short, the system does not bypass browser security; rather, it uses the intended “host_permissions” mechanism to enable secure data transmission, with Cross-Origin Resource Sharing (CORS) rules being the framework that enforces these permissions [27, 30].

Another protective layer is the Content Security Policy (CSP) [29]. Websites can use CSP to define trusted sources from which resources like scripts, images, and other media can be loaded. Usually, this helps prevent malicious code injection. For the developed extension, CSP on visited websites could potentially restrict the loading of external resources or the execution of certain scripts if they are not aligned with the page’s policy. By offloading the machine

learning logic to a backend API, the system largely avoids this issue. The API, running on a separate server, is not obligated to the CSP of the webpage the user is visiting. The extension's only requirement is to make a single, permitted network request to the API's URL, a much more reliable interaction than running complex code within a restrictive environment.

In summary, the combination of general web security policies like the Same-Origin Policy, CORS, and CSP along with the several limitations imposed by the Manifest V3 standard on client-side computational capabilities, dictated the necessity of the client-server architecture for the developed visual content filtering system.

3.1.1.2 Platform Focus Rationale

The Manifest V3 limitations are most pronounced in the Chromium ecosystem. It is worth noting that Mozilla Firefox currently maintains support for the older Manifest V2 standard [24]. As discussed previously, Manifest V2 offered flexibility, particularly regarding background processing, which could potentially allow different architectures closer to client-side ML analysis.

Despite the theoretical alternative presented by Firefox/Manifest V2, this research concentrated primarily on the Chromium/Manifest V3 environment. Several key factors drove this decision. Firstly, the dominant global market share of Chromium-based browsers makes this platform the most critical and relevant target for a broadly applicable safety tool [22, 62]. Secondly, a specific research goal was to confront the challenges posed by the ongoing, industry-wide shift to Manifest V3 and develop a functional solution within those contemporary constraints [104]. Lastly, attempts within this project to establish an efficient development workflow for Firefox encountered several practical difficulties related to configuration and testing processes, proving unexpectedly complex and hindering progress (e.g., persistent issues with manifest file compatibility, which is the central configuration file that defines an extension's permissions and structure, and the debugging of temporary add-ons within the testing environment).

3.1.1.3 Infeasibility of Network-Level Visual Filtering

Before settling on the client-server architecture, alternative approaches were considered. Ideally, a filtering system would operate transparently at the network level, protecting the user without requiring any local browser extension or configuration. However, exploring alternative approaches revealed that filtering visual content directly at the network level poses substantial technical limitations. The universal use of HTTPS encrypts web traffic, including images and videos, between the browser and server. Analyzing visual content at the network layer would therefore require decrypting this traffic. Reliable HTTPS decryption generally necessitates installing a custom root SSL certificate (a special digital file that allows a third-party tool to position itself as a trusted authority for all of the user's web traffic) on the user's device [105]. Modern browsers are designed to distrust such certificates for security reasons and typically block connections or issue severe warnings [106]. Furthermore, network filtering tools generally cannot selectively decrypt only visual data (as the type of content within an encrypted packet is not visible until the entire packet is decrypted). Instead, they usually perform a "man-

in-the-middle” decryption of all traffic (an interception technique where the tool decrypts, inspects, and then re-encrypts all data passing between the user and the internet). This process creates various privacy risks and usability problems, such as exposing sensitive user data like passwords or financial information to the filtering tool and breaking website functionality [105]. Such a method presents an unacceptable security risk and installation barrier for a safety tool intended for a general audience. Given these fundamental security and practicality challenges, network-level visual analysis was deemed infeasible for this project, reinforcing the utility of a browser-extension-based approach. By operating within the browser environment, the extension can access and analyze visual content only after the browser has securely decrypted the HTTPS traffic, thereby avoiding problematic certificate manipulations and the associated security risks of external network-level filtering.

3.1.1.4 Infeasibility of a Cloud-Hosted API

A client-server architecture became necessary due to fundamental technical browser constraints (such as the Manifest V3, Same-Origin Policy, and others detailed in Section 3.1.1.1). The subsequent choice to implement that server locally rather than in the cloud was then made based on project-level goals of cost and user privacy. A fully cloud-hosted API was therefore considered but ultimately rejected for two primary reasons: unsustainable operational costs and significant privacy implications.

Firstly, regarding cost, the financial investment for cloud infrastructure can become a barrier for a non-commercial tool [32]. For instance, a service like “Google Cloud Vision API” prices its “Safe Search” detection on a per-image basis. While a small number of requests might fall within a free tier, the cost structure presents challenges on a large scale. A single user on a media-rich website could generate thousands of analysis requests daily. At a rate of approximately \$1.50 per 1,000 images, the cost for even one active user could become substantial over time, making a pay-per-use model financially impractical for a tool intended for wide, free distribution. Secondly, and more critically, a cloud architecture would require sending a user’s visual browsing data to a remote server, creating considerable privacy risks [107].

These risks are not merely theoretical. Recent academic analyses of commercial parental control apps have documented systemic privacy flaws, including excessive data collection and insecure data transmission, effectively turning some protection tools into surveillance software [8]. To be more specific, a cloud-based version of this system would expose a complete visual record of a user’s browsing activity to a third party, a tangible threat that runs counter to the project’s goal of creating a trustworthy safety tool.

Given that a core objective is to provide an accessible and private safety tool, both the potential cost and the documented privacy trade-offs of a cloud solution were deemed unacceptable. The decision to use a locally-hosted backend directly addresses these concerns, ensuring the system remains free for the end-user and that all sensitive visual data remains on the user’s machine.

3.1.1.5 Architectural Decision Summary and Performance Context

In conclusion, the locally-hosted client-server model was adopted as the most pragmatic and effective strategy to achieve this project’s comprehensive goals. The decision was dictated by a combination of prevailing technical constraints. Fundamental browser security policies [17, 29, 30, 59], and the prevailing Manifest V3 standard [17, 18, 19, 40] make a purely client-side approach architecturally non-viable. Concurrently, network-level filtering faces major obstacles due to HTTPS encryption [105, 108], while a fully cloud-hosted solution introduces unacceptable privacy risks and operational costs [32, 107]. The chosen design enables the core visual filtering functionality by using a local backend for ML processing. The primary performance consideration for this architecture involves communication overhead. While experiments detailed in Chapter 4 demonstrate that the “EraX-NSFW-V1.0” model itself processes images quickly [53], the overall system responsiveness is a function of the total communication overhead, which is impacted by factors such as network speed and the tunneling tools used (Cloudflare/Ngrok) [101, 102].

3.1.2 System Workflow

The visual content filtering process operates through a series of sequential steps, from the initial page scan to the final blurring of detected content. This client-server interaction creates an end-to-end process that can be broken down into the following key stages:

- When a page starts loading, the extension scans the page for visual elements (images, video frames, GIFs).
- It captures these visual elements and prepares them for analysis.
- The captured visual elements are sent via a secure Tunneling Service (such as Cloudflare Tunnel or Ngrok), which provides a public URL to the local detection backend.
- The backend operates on the host machine (“localhost: 5000”) as a local Python Flask API and receives the base64-encoded image data from the extension (a standard method for transmitting binary data as text [42]). It then uses the “EraX-NSFW-V1.0” model to analyze the media against NSFW classification confidence thresholds.
- The API returns a JSON response with the final prediction.
- The extension then acts on this prediction, either allowing the content to be displayed unmodified (“SFW”) or applying a visual blur to hide it (“NSFW”).

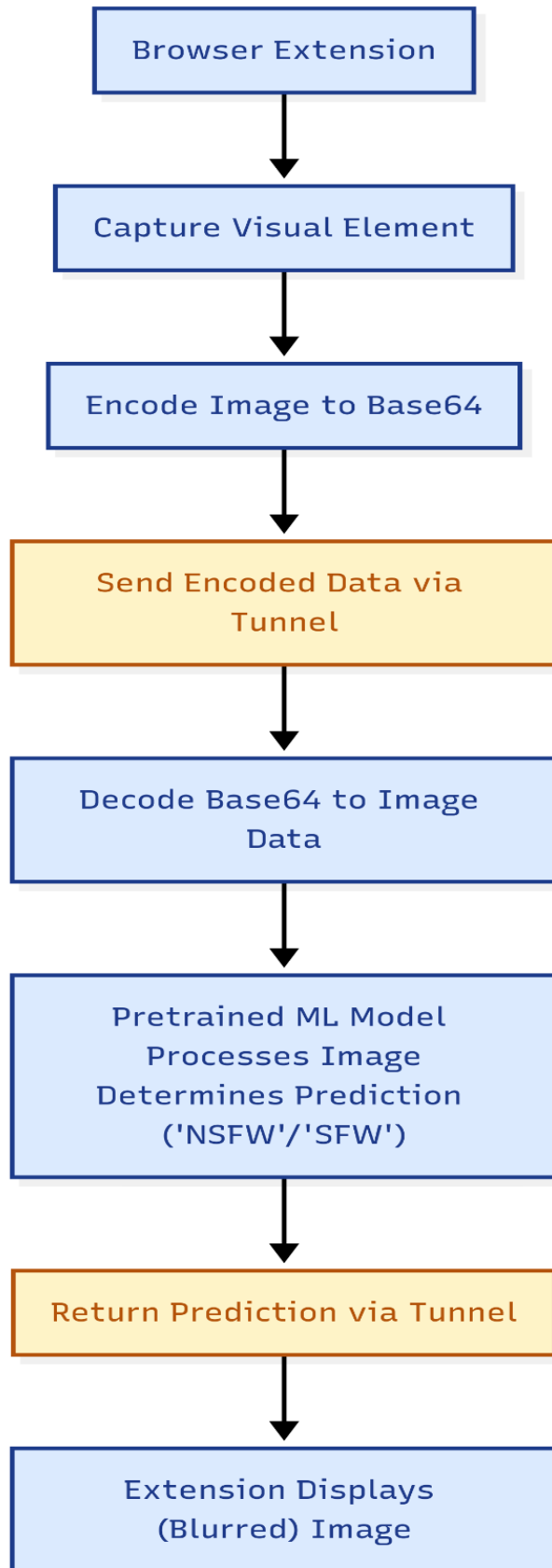


Figure 1: Workflow

3.2 Implementation Details and Tools Used

The project utilized specific hardware and software tools to facilitate the development, deployment, and testing of the system. The following subsections detail the specific components of this environment.

3.2.1 Hardware

All development and testing were conducted on the student’s personal computer, equipped with an AMD Ryzen 7 8840U processor with integrated Radeon 780M Graphics (running at 3.30 GHz) and 32GB of RAM. This setup provided the necessary computational resources for running the backend API and browser extension concurrently during experiments.

3.2.2 Software

The development environment primarily consisted of Visual Studio Code (VS-Code) running on Windows 11. The key software components and libraries are detailed below:

- **Backend API:** The server-side component was developed in Python, using a combination of key libraries. Flask (a lightweight web framework for creating APIs [25]) was used for the web framework, and PyTorch (a machine learning library for tensor operations [109]) was used for the core machine learning logic. The “EraX-NSFW-V1.0” pretrained model (built upon the YOLOv11 framework) was used for NSFW detection [20]. For image data processing, the system employed Pillow (a powerful image manipulation library [110]) and OpenCV (a library for real-time computer vision tasks [111]). The API was designed to receive image data (encoded in base64 format [42], which converts the image to a text string) via HTTP POST requests from the browser extension.
- **Browser Extension:** The client-side extension was built using standard web technologies: JavaScript for the core logic (intercepting images, communicating with the API), HTML for the structure, and CSS for styling. Configuration data was managed using JSON files.
- **API Tunneling:** To navigate potential CORS issues during local development, tunneling software was used to securely connect the browser extension to the Flask API. Both Ngrok and Cloudflare Tunnel were employed [101, 102]. These tools generate a temporary, public HTTPS URL that forwards requests to the local server (running on “localhost: 5000”), enabling secure real-time testing without exposing the development machine.
- **Testing Environment:** The primary browser used for testing the extension’s functionality and performance was Google Chrome. The custom-built website used for the comparative benchmark (as detailed in Section 3.3.3) was created using the Canva platform [112].

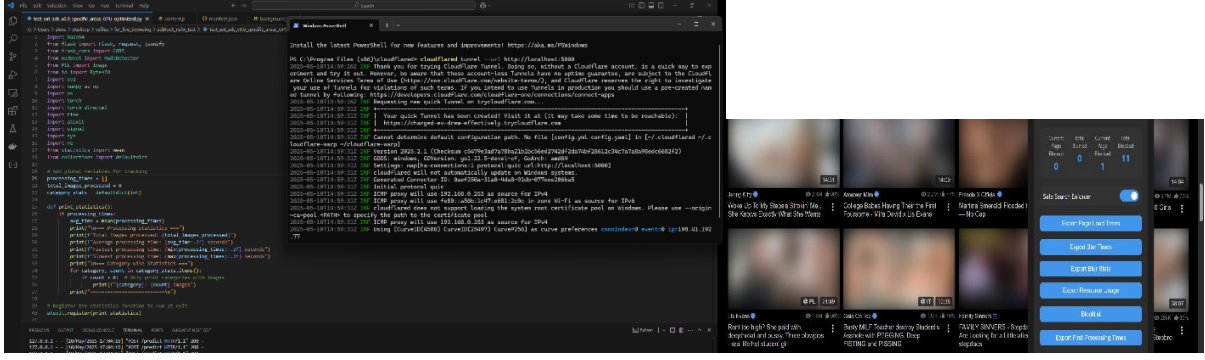


Figure 2: The experimental setup showing the key software components: the browser client, the tunneling service, and the local backend API

3.3 Experimental Setup and Procedures

This section details the comprehensive experimental methodology used to evaluate the system’s performance. It begins by establishing the operational definition of NSFW content (Section 3.3.1) and the specific filtering logic of the developed system (Section 3.3.2). It then describes the configuration for the comparative benchmark (Section 3.3.3) and justifies the deliberate exclusion of advertisements from the visual analysis (Section 3.3.4), before concluding with the distinct procedures used to measure accuracy (Section 3.3.5) and responsiveness (Section 3.3.6).

3.3.1 Operational Definition of NSFW Content

The term “Not Safe For Work” (NSFW) is not a formal technical standard but an informal internet slang term with inherent ambiguity. According to the online repository Urban Dictionary, it is used to describe online posts associated with nudity, profanity, and other content that could be linked with pornography [113]. Given this broad and subjective definition, this thesis must establish its own consistent operational criteria for what will be considered an accurate prediction during the evaluation of the results. Therefore, for the purposes of this research, NSFW content is defined as visual media that depicts genitals or implies direct sexual acts, such as pornographic scenes with at least partial nudity. This definition is also extended to include depictions of individuals in underwear to better capture suggestive or borderline content. Within this scope, a distinction is made between underwear and swimwear. Swimwear is generally associated with public, non-sexual activities (e.g., sports, beaches), whereas depictions of underwear are more frequently used in private or sexually provocative contexts. To ensure a consistent and cautious labeling process, the following rules were applied. First, images clearly featuring underwear were classified as NSFW, while images clearly featuring swimwear in appropriate public settings were classified as SFW. Crucially, to handle cases of visual ambiguity where a garment could not be definitively identified as swimwear (e.g., in a tightly cropped image where a bikini bottom is visually indistinguishable from briefs), the item was conservatively classified as NSFW. The

aforementioned operational rule ensures that any uncertainty was resolved in favor of the project’s primary child safety objective. Similarly, this research adopts a common industry distinction where depictions of female breasts are considered NSFW, while images of shirtless men are classified as SFW, a standard also reflected in Meta’s community policies [114]. Adopting such a context-dependent distinction serves as a conservative operational criterion that aligns with the project’s overall goal of prioritizing child safety by erring on the side of caution.

Establishing these operational criteria was a methodological necessity. The specific training data and classification standards for the pre-trained “EraX-NSFW-V1.0” model are not publicly documented, which is a common challenge for many high-performance, specialized models. Without a clear, internal standard defined within this thesis, a reproducible and scientific evaluation would be impossible, as the model’s performance would be measured against an unknown benchmark. This lack of transparency is a common challenge in the field, as other models that claim high accuracy are often evaluated on private, inaccessible datasets, making direct comparisons of their effectiveness unreliable [115]. Furthermore, the creation of public datasets for this specialized task is a well-documented challenge. The curation of such material involves substantial ethical and practical difficulties, which explains the general scarcity of large-scale, unified datasets for sensitive content detection [49]. This issue is compounded by findings that major public datasets have contained illegal and harmful content [15]. Therefore, defining the evaluation criteria internally was a foundational step for ensuring a precise and valid assessment.

3.3.2 Implemented Filtering Logic and Adaptive Mechanisms

To technically implement the operational criteria established in Section 3.3.1, the system leverages the “EraX-NSFW-V1.0” model, which is built upon the YOLOv11 object detection framework [20]. The model is trained to identify five explicit classes: “anus”, “make_love”, “nipple”, “penis”, and “vagina” [20]. For the purposes of this research, a positive detection of one or more of these classes is used as the technical proxy for identifying content that meets this thesis’s definition of NSFW. To account for suggestive or borderline content where there is no direct nudity (such as images of people in underwear), the system relies on contextual detection. An image featuring underwear may still be flagged as NSFW if the model also detects one of its five explicit classes, like ‘nipple’ or ‘anus’, in the same frame.

This operational methodology not only adheres to the definitional principles outlined above but also aligns closely with the specific technical standards of major online platforms. For instance, Meta’s “Adult Nudity and Sexual Activity” policy states its purpose is to restrict content to which users may be sensitive and defaults to removing sexual imagery to prevent the sharing of non-consensual or underage content [114]. Furthermore, the classes detected by the “EraX-NSFW-V1.0” model are a direct subset of content explicitly prohibited by Meta, which includes “visible genitalia”, “visible anuses”, “visible female nipples”, and “explicit sexual activity” [114]. This alignment with industry standards validates the criteria selected for this thesis’s filtering system.

The classification of a visual element as NSFW occurs when the “EraX-NSFW-V1.0” model detects one or more of these labels with a confidence score that exceeds a manually set threshold within the Python Flask API. To address the contextual ambiguity that the five classes alone cannot capture, the system incorporates a custom context-aware technique that employs a heuristic-based approach. This technique enhances the model’s effectiveness without retraining, by using three mechanisms.

- First, “Domain Analysis” uses regular expressions (a sequence of characters that specifies a match pattern in text [34]) to scan the visited website’s URL for high-risk keywords.
- Second, “Textual Context Analysis” checks an image’s “alt text” (alternative text: a brief textual description of an image [116]) and surrounding captions for unsafe keywords.

The development of the keyword list was a multi-stage process designed to maximize detection effectiveness. The initial lexicon was seeded with a list of common profane and pornographic terms. This base was then augmented by performing a term frequency analysis on the textual content of representative adult content websites, from which the most frequent terms were extracted and manually reviewed for relevance. This expanded list underwent a final iterative refinement phase during preliminary system testing to remove keywords that produced high false-positive rates on safe content.

- Lastly, the system incorporates a third, model-driven adaptive mechanism. If the “EraX-NSFW-V1.0” model detects a label of these classes (“anus”, “make_love”, “nipple”, “penis”, and “vagina”) with a sufficiently high confidence score, the system flags the entire website domain as “high-risk” for the remainder of the browsing session.

A key principle in the design of these three mechanisms was computational efficiency. This lightweight, heuristic-based design was a deliberate choice, as more complex alternatives like retraining the model or performing deep semantic analysis were deemed infeasible due to the prohibitive resource costs and the lack of public [32], ethically-sourced datasets [15], as established in Section 2.4. By relying on fast regular expression matching for text analysis and simple numerical comparisons for the model-driven feedback loop, the adaptive logic was engineered to be lightweight, ensuring it would add only minimal processing overhead to the core model inference.

These three mechanisms create an adaptive system designed to strategically balance two competing objectives: maintaining a non-disruptive user experience and providing robust child safety. The logic operates on a simple principle: the system uses a default, less-sensitive setting to avoid incorrectly blurring safe content during general browsing. When any of the three mechanisms detects a contextual indicator of risk, the system dynamically switches to a highly sensitive setting. This approach ensures that the system’s full protective capability is deployed only when it is most needed. The primary purpose of the baseline evaluation presented in Section 4.1 was to leverage empirical data to find the optimal numerical thresholds for these respective “default” and “high-sensitivity” operational states. The methodology in this chapter,

therefore, defines the system’s adaptive strategy, while Chapter 4 provides the data for its final configuration and validation.

3.3.3 Comparative Benchmark Methodology

To contextualize the performance of the developed system, a comparative benchmark was conducted against two distinct alternative filtering technologies. This section details the methodology for that controlled experiment, which is distinct from the live-browsing procedure used for the baseline and adaptive system evaluations (described in Section 3.3.5). A primary goal of this analysis was to directly evaluate the trade-offs between different architectural approaches (client-server vs. client-only). While client-only solutions face long-term viability challenges due to Manifest V3, including one in the benchmark (“NSFW Filter”), it is essential for a comprehensive architectural comparison. This allows for a direct contrast of the potential for higher visual accuracy afforded by a backend model against the qualitatively observed responsiveness of a client-only solution. The specific open-source tools for this benchmark were selected because, unlike many commercially available “black box” alternatives, they offer a degree of transparency even though their training datasets are private [35, 54, 117, 118]. Many service-based APIs (such as “Google Cloud Vision” and “Sightengine”) and other available browser extensions (such as “PuritySight”) do not publicly disclose the specific models or training data they use, making them unsuitable for a reproducible scientific comparison [117, 118]. Furthermore, the creation of new high-performance models from scratch is hampered by the general scarcity of large-scale, publicly available datasets for this sensitive task, a problem compounded by the ethical and practical difficulties of curating such material [15, 49]. Given these constraints, the “NSFW Filter” was chosen as the client-only representative because its underlying “NSFWJS” library is well-documented and claims a high accuracy of up to 93% [57]. Similarly, “NudeNet” was selected as the server-based alternative because it is a model with a known architecture (YOLOv8), defined classes, and published studies suggesting a comparable performance of up to 93% [35, 83, 119]. Selecting these two highly effective tools provides a robust benchmark to empirically assess whether the newer architecture of the proposed system offers a tangible improvement over these established solutions.

3.3.3.1 Benchmark Dataset and Environment

To ensure a fair and direct comparison, a controlled testing environment was established, which differs from the “live-browsing” methodology. For this purpose, a dataset was compiled to reflect a realistic browsing experience by sourcing images from diverse categories, despite its modest size. A key requirement for the benchmark was the curation of unambiguous content; therefore, the NSFW portion was composed of clear, explicit material like pornography. The SFW portion contained non-explicit images, such as pictures of inanimate objects, vehicles, or faces with no sexual context. A deliberate focus on unambiguous content was a key methodological choice. It allows for a direct evaluation of each tool’s core technical capability on clearly defined visual data, rather than its performance on subjective material where definitions of “NSFW” can vary and introduce confounding variables. It is critical to note that all images used in this benchmark were sourced from publicly available internet domains. A meticulous manual review was conducted to ensure the dataset did not contain illegal material, such as content depicting minors or non-consensual acts. However, it must be acknowledged that the curation of any explicit content carries inherent risks [15], and this dataset was compiled exclusively for the technical evaluation of the filtering systems.

To simulate real-world browsing conditions, the dataset was hosted on a publicly accessible website designed and published using the Canva platform [112]. It provides a template-based service that was chosen for its simplicity in creating a controlled environment without a complex web development framework. The test site was therefore hosted on Canva’s public infrastructure. Although effective, this setup introduced practical limitations on the total sample size. Several factors constrained the number of images. A primary constraint was the need to conclude experiments involving the “NSFW Filter” extension before its supporting Manifest V2 framework is fully deprecated by Chromium browsers in July 2025 [23]. Further limitations included the 5GB cloud storage cap of the Canva platform’s free tier, and the notable time required for manual image curation. Despite these constraints, using a live website provides a more realistic benchmark than analyzing a static folder of images. This approach allows for testing the extension’s full end-to-end workflow, including its ability to interact with the page’s Document Object Model (DOM, a representation of the page’s structure that allows an extension to access and capture visual elements) and process media as the browser renders it.

3.3.3.2 Tools Configuration

The following section details the specific configurations used for each of the three tools in the comparative benchmark (Section 4.3). The developed system, in this specific benchmark, was evaluated in two distinct configurations to clearly demonstrate the impact of its adaptive logic. It was tested using a static confidence threshold of 0.2 (meaning the model must be at least 20% confident in its detection to classify an image as NSFW) to provide a baseline. Subsequently, the system was tested with its context-aware adaptive mechanism (the set of heuristic-based rules detailed in Section 3.3.2) enabled, representing the complete proposed system.

To ensure a fair and consistent evaluation during the comparative benchmark (Section 4.3), the operational definition of NSFW content established for this thesis was applied to all alternative tools. A specific configuration was therefore required for the “NudeNet” model, as it utilizes a broad detection taxonomy of eighteen classes for which the training criteria are not publicly documented. Since there is no specific documentation to address this ambiguity, a deliberate methodological choice was made to select specific classes for the analysis.

The full list of classes for “NudeNet” includes:

All Classes	
Classes included	Classes disregarded
FEMALE_GENITALIA_COVERED	FACE_FEMALE
ANUS_EXPOSED	FEET_EXPOSED
BUTTOCKS_EXPOSED	BELLY_COVERED
FEMALE_BREAST_EXPOSED	FEET_COVERED
FEMALE_GENITALIA_EXPOSED	ARMPITS_EXPOSED
ANUS_COVERED	ARMPITS_COVERED
FEMALE_BREAST_COVERED	MALE_BREAST_EXPOSED
BUTTOCKS_COVERED	FACE_MALE
MALE_GENITALIA_EXPOSED	BELLY_EXPOSED

Table 2: All classes of “NudeNet” [35]

To align the benchmark with the thesis’s focus, the selected subset for a positive NSFW detection included classes depicting both explicit nudity (e.g., “FEMALE_BREAST_EXPOSED”, “MALE_GENITALIA_EXPOSED”) and relevant suggestive “covered” classes (e.g., “BUTTOCKS_COVERED”) that correspond to this project’s criteria of treating depictions of underwear as NSFW. Conversely, more general anatomical classes like “FACE_FEMALE”, “BELLY_EXPOSED”, and “ARMPITS_EXPOSED” were intentionally disregarded. This was a necessary step to ensure all tools were being tested on the same, well-defined task. Including these out-of-scope classes would mean evaluating “NudeNet” on a different and much broader task than the developed system, making a direct comparison scientifically invalid.

A corresponding step was taken to carefully curate the benchmark dataset to match this specific focus. Using SFW images that could correctly trigger one of NudeNet’s disregarded classes would have created a definitional mismatch and produced meaningless results. For example, a SFW picture of a person in a bikini at the beach would correctly trigger “NudeNet’s” “BELLY_EXPOSED” class (counting as a True Positive for “NudeNet’s” own broad goal). However, against this thesis’s specific ground truth, that same detection would be a False Positive. To prevent such a skewed outcome, the benchmark dataset was curated to contain only unambiguous examples relevant to the single, shared task being evaluated (e.g., the SFW portion contained pictures of inanimate objects or vehicles, while the NSFW portion contained clear, explicit material).

The two-part approach of selecting specific detection classes and curating the test dataset accordingly was an intentional methodological choice. It was essential for ensuring a valid and fair “apples-to-apples” comparison by preventing the results from being skewed by definitional ambiguity (in other words, ensuring both systems agreed on what constituted an “NSFW” image for the purpose of the test).

The final tool compared in this benchmark was the “NSFW Filter”, which operates entirely on the client side and was evaluated across a range of its user-configurable sensitivity settings [36]. While its reliance on the deprecated Manifest V2 framework makes it an unsustainable long-term solution, its inclusion is essential as an architectural counterpoint to evaluate the trade-off between the inherent low latency of client-side processing and the higher accuracy potential of a server-based model. The extension is based on the “NSFWJS” library, which uses either the MobileNetV2 or the InceptionV3 model. For this benchmark, the lightweight MobileNetV2 model was selected because while the library’s documentation claims the same 93% accuracy for both MobileNetV2 and the InceptionV3 model, the former’s smaller size is better suited for a client-side tool to avoid negatively impacting browser responsiveness and speed [54]. As the tool’s documentation does not offer a precise technical definition for this setting, the evaluation was conducted by systematically testing its performance at various levels [36].

The two-part process of aligning the detection criteria and curating the dataset was therefore a crucial methodological control, designed to ensure the benchmark provided a valid and fair comparison of each tool’s core detection capability on a single, well-defined task.

3.3.4 Exclusion of Advertisements from Analysis

A complete methodological picture requires defining what is outside the operational scope of the visual analysis. A notable exclusion is the system’s inability to analyze advertisements. Research shows advertisements are a substantial source of inappropriate material for minors [120], and modern digital platforms increasingly use server-side techniques to embed ads directly into content streams [121, 122]. A technical challenge arises because the extension works by reading the structure of the main webpage. Advertisements are often delivered in isolated containers (separate, self-contained components within the webpage), and for security reasons, the browser prevents the extension from reaching inside these containers to analyze their visual content [123]. While a comprehensive safety tool would need to address such a limitation, potentially through the use of filter lists (pre-defined lists of rules that block requests to known ad-serving domains [124]), the visual analysis component of this research focused exclusively on visual media (images and video-gif frames) embedded directly within the main page’s DOM (a representation of the page’s structure that allows the extension to access and capture image elements [39]), which the extension can technically access. Such a focus was a deliberate methodological choice, intended to isolate the evaluation of the core visual analysis pipeline from the distinct challenge of traditional ad-blocking. The potential integration of filter lists is discussed further as future work in Section 6.2.2.

3.3.5 Accuracy Evaluation Procedure

To evaluate the baseline and adaptive performance of the developed system (results presented in Sections 4.1 and 4.2), a different methodology from the controlled benchmark was required. While scripted testing (an approach that follows a predefined set of test cases and procedures) is excellent for providing repeatable, verifiable results [125, 126], its significant time investment, inflexibility, and high maintenance overhead would be prohibitive for evaluating the system’s robustness in the unpredictable environments of diverse, live websites [127]. In contrast, exploratory testing (live-browsing), where a tester explores an application without a predefined plan, is better suited for discovering unknown issues and assessing usability in real-world scenarios [128]. Therefore, the exploratory approach was chosen as the more appropriate methodology for this task.

Many of these sites utilize features like “infinite scroll” or asynchronous loading, which constantly alter the page’s Document Object Model (DOM) [117]. To handle these changes, the extension employs a “MutationObserver” [117, 129], a web API that allows the system to react to modifications in the page’s structure, such as new images appearing as a user scrolls. Testing this reactive logic with unpredictable manual browsing provides a more rigorous evaluation, as automated scripts typically follow a predictable path and are less suited for assessing the system’s robustness in unscripted scenarios. For the baseline and adaptive system evaluations (Sections 4.1 and 4.2), the live-browsing approach was therefore used over a static, pre-labeled dataset as it allows for a more holistic evaluation of the complete end-to-end system pipeline. The approach provides a pragmatic assessment of the system’s real-world

performance against the varied formats and layouts encountered during a typical browsing session, rather than just testing the model in isolation.

A key methodological decision was to focus on primary visual content by intentionally excluding small frames (visual elements with a width or height smaller than 128 pixels) from the analysis. This decision was made to avoid the risk of including a high volume of UI elements, such as icons (for instance, with some UI elements on social media platforms being as small as 40x40 pixels) and logos, which could artificially inflate accuracy metrics. Including thousands of such easily-classified SFW elements would skew the results by inflating the True Negative count and obscure the model’s performance on the more challenging, substantive visual content that is the focus of this work. Such a choice represents a methodological trade-off. While the approach ensures a more accurate assessment of performance on primary content, it also means the system’s effectiveness on small-format media, like thumbnails or certain pop-up advertisements, remains an unevaluated area.

These browsing sessions were conducted across diverse website categories, such as social media platforms, sports websites, and adult content platforms, as well as numerous other sites encountered during general browsing to ensure a varied dataset. The process for the final dataset for evaluation contained a mix of explicitly pornographic content, borderline or suggestive material often found on social media, and a majority of clearly safe content from general-interest websites. During these sessions, all visual elements captured by the browser extension were saved locally for subsequent analysis. To establish the “ground truth” for this evaluation, each saved image was manually reviewed by the author, a process which involved applying the specific classification criteria outlined in Section 3.3.1, such as the presence of visual media depicting genitals or implying direct sexual acts.

This meticulous process resulted in a curated dataset. For the classification of each visual element, there are two key pieces of information: the ground truth classification (as determined by the manual review) and the prediction generated by the “EraX-NSFW-V1.0” model (if it exceeds the confidence threshold, it is predicted as NSFW). The aforementioned procedure provides the foundation for the quantitative performance analysis presented in the following sections.

3.3.6 Latency Measurement Procedure

To quantify the system’s responsiveness from a user’s perspective, a metric called “Time to First Intervention” was measured for the first visual element processed on a page. The data collection for this metric was handled programmatically by the browser extension. The process begins within the “content.js” script (the extension’s primary script for interacting with web page content), which records an initial timestamp using “performance.now()” (a high-resolution timer for web browsers) when the page has finished loading. After the first visual element is captured, sent to the backend API, and a final classification is received, the total elapsed time since the page load is calculated (effectively measuring the full round-trip from browser to local server and back). This final value is then sent to the “background.js” script (the extension’s central script for managing background tasks) via the “logFirstProcessingTime()” function and stored locally along with the specific website domain. It is important to clarify the precise trigger for this measurement. The extension begins its initial analysis scan only after the “window.onload” event fires. This event signals that the browser has finished loading all of the page’s initial static resources, including the DOM structure and all originally referenced images and stylesheets. This specific starting point is a key methodological decision as it ensures the test accurately measures the processing time of the extension itself, rather than including the website’s initial loading performance.

Of particular importance is the fact that to ensure the validity of these latency measurements, a final procedural step was taken to account for the “cold start” delay inherent in machine learning systems [130]. Usually, the very first request to the backend server after it is launched will be significantly slower due to the one-time cost of loading the AI model into memory [130]. To isolate this outlier and ensure the collected data represents the system’s typical, steady-state performance (in other words, the speed of the system after the initial, one-time cost of loading the model into memory), a warm-up procedure was conducted before each data collection session. To mitigate the impact of this initial “cold start” delay on the results, a practical warm-up procedure was implemented before each data collection session. While a full profiling study to determine the precise warm-up point was not conducted, a practical and conservative heuristic was adopted. This involved browsing non-test websites until the backend model had processed over 100 images (a threshold chosen to be significantly larger than what would be required for the one-time costs of model loading and process initialization), thereby ensuring the system was operating in a steady state before formal latency measurements began. The purpose of this procedure was to minimize the potential skewing effect of the initial “cold start” delay on the overall statistical distribution of the latency measurements. Formal data collection on the target websites started only after the warm-up phase was completed, providing a more reliable and representative measurement of the system’s typical, steady-state performance.

The focus on a steady-state “Time to First Intervention” was a deliberate methodological choice directly informed by the project’s primary safety objective. Research on the psychological impact of online content confirms that unwanted exposure to pornography is a prevalent issue for adolescents that causes negative emotional reactions such as shock, disgust, and confusion [37]. The system’s most critical function is therefore to prevent an initial harmful exposure. The speed of the first intervention consequently represents the most relevant metric for evaluating system success.

Other performance aspects, while important, were determined to be unsuitable for this specific benchmark. A measurement of the initial “cold start” latency, while valuable for understanding a user’s first experience, was methodologically excluded. The first inference request for a model can be significantly slower due to one-time initialization costs, creating a high-variance outlier (e.g., a single “cold start” could take 1 second or 10 seconds, making a statistical average unreliable) that would skew benchmark results intended to measure typical performance [130]. Similarly, a measurement of system throughput, an important metric for user experience on media-rich pages, was also considered. However, it was deliberately scoped out due to a key technical limitation. Subsequent analysis requests from a single page become queued because essential tasks like capturing image data and applying the final blur require manipulating the page’s Document Object Model (DOM). Such DOM operations must be handled sequentially on the browser’s single main thread [38]. While Web Workers provide a standard method for parallel processing, they were not a viable solution for this problem, as the Manifest V3 security model prohibits them from the direct DOM access these tasks require [40, 58].

This necessary sequential processing makes it impossible to isolate a model’s true processing time for a subsequent image from the unpredictable time spent waiting in a queue (e.g., the tenth image on a page must wait for the first nine to be fully processed and their results returned). Therefore, to maintain benchmark validity and align the measurement with the primary safety objective of the thesis, the analysis concentrated on the steady-state performance of the first intervention. A detailed analysis of both the “cold start’s” effect on user experience and the system’s overall throughput remains a valuable area for future research.

\

3.4 Performance Metrics

To evaluate the effectiveness and performance impact of the system, the following metrics were employed. These metrics cover both the accuracy and speed of the NSFW visual content filtering, allowing for a comprehensive assessment aligned with the project objectives.

3.4.1 NSFW Filtering Evaluation

The evaluation of the visual filtering component focused on the “EraX-NSFW-V1.0” model’s ability to correctly identify and hide inappropriate content while minimizing disruption to the user’s browsing experience [20].

To this end, performance was assessed using the standard confusion matrix elements, which provide a foundational understanding of the model’s classification behavior [131, 132]:

- True Positives (TP): NSFW content correctly identified as NSFW.
- True Negatives (TN): Safe For Work (SFW) content correctly identified as SFW.
- False Positives (FP): SFW content incorrectly identified as NSFW.
- False Negatives (FN): NSFW content incorrectly identified as SFW.

These are essential elements for understanding the types of errors the “EraX-NSFW-V1.0” model makes within the system.

From these core components, the following standard metrics were calculated to quantify overall performance: [131, 132]

- Accuracy: Overall proportion of correct classifications $(TP+TN) / \text{Total}$.
- Precision: Proportion of items flagged as NSFW that were actually NSFW $(TP / (TP + FP))$. It is important for measuring how much safe content is incorrectly blocked.
- Recall (True Positive Rate, Sensitivity): Proportion of actual NSFW items that were correctly identified $(TP / (TP + FN))$. Important for measuring the effectiveness of the extension in capturing harmful content.
- F1-Score: The harmonic mean of Precision and Recall $(2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}))$. Provides a single metric balancing the trade-off between missing NSFW content and incorrectly blocking safe content.
- False Positive Rate (FPR): Proportion of SFW items incorrectly identified as NSFW $(FP / (FP + TN))$. Directly measures the rate of incorrectly blocking safe content, impacting user experience.
- False Negative Rate (FNR): Proportion of actual NSFW items incorrectly classified as SFW $(FN / (FN + TP))$. This indicates the rate at which harmful content is missed by the system, impacting safety.

For a child safety tool, these metrics exist in a specific context. Specifically, a high Recall is prioritized, as failing to detect an NSFW image (a False Negative) is a more severe error than accidentally blocking a safe one (a False Positive). On the other hand, a high False Positive

Rate (FPR) must also be avoided, as it can degrade the user experience and erode the user's trust in the tool. Therefore, the F1-Score is a notably valuable metric as it measures the balance of this tradeoff.

3.4.2 Responsiveness Metric

In addition to classification accuracy, the system's practical effectiveness was also evaluated based on its responsiveness. As research confirms the unexpected appearance of NSFW content can be a disruptive and potentially harmful event, the speed at which the system can intervene is a critical measure of its practical effectiveness [2, 133].

Based on the project's primary safety objective, the key responsiveness metric used in this research is the "Time to First Intervention". This metric is formally defined as the total duration, in milliseconds, from when the page finishes loading (the "window.onload" event) until the classification response for the first visual element is received. The detailed methodological rationale for focusing on this specific metric is provided in the procedure description in Section 3.3.6. To make this goal measurable, the primary performance objective is to intervene under the average human reaction time of approximately 250 ms [31].

For context, a response under 100 ms is recognized as the ideal for a perceived instantaneous user experience [31]. However, achieving this ideal presents a significant architectural challenge for the client-server paradigm mandated by Manifest V3. The necessary steps for compliant data transmission introduce an inherent architectural overhead. The analysis in Section 4.4, therefore, deconstructs the total latency to quantify the time consumed by each of its primary components (such as client-side data preparation, network communication, and backend processing). Overcoming this inherent overhead to create a more seamless experience remains a key area for future improvement. Therefore, the critical objective for this research was to validate that the system could operate effectively within the human reaction time threshold, allowing it to intervene before a user can fully process the harmful visual content.

3.5 Chapter Summary

This chapter detailed the comprehensive methodology used to design, implement, and evaluate the visual content filtering system. It established the rationale for a client-server architecture, a pragmatic solution engineered to navigate the technical constraints of modern browsers, particularly Manifest V3. The specific hardware and software tools were outlined, followed by a detailed description of the experimental procedures. This included establishing a precise operational definition for NSFW content, which was implemented using the five classes of the "EraX-NSFW-V1.0" model as a technical proxy and enhanced by a custom context-aware logic, and explaining the procedures for accuracy and latency evaluation. Finally, the chapter formally defined the key performance metrics used to evaluate filtering effectiveness and system responsiveness, providing the methodological foundation for the results and analysis presented in the following chapter.

4 Results and Analysis

This chapter presents the comprehensive empirical evaluation of the developed visual content filtering system. The analysis begins by establishing the system’s baseline performance (Section 4.1), then demonstrates the improvements gained from the context-aware adaptive logic (Section 4.2). Subsequently, the system is benchmarked against alternative filtering solutions in a controlled environment (Section 4.3), followed by an in-depth analysis of its real-world responsiveness and latency (Section 4.4). The chapter concludes with a summary of the key findings from the evaluation (Section 4.5).

4.1 Baseline Performance of the “EraX-NSFW-V1.0”: A Universal Threshold Approach

This section contains the initial requirements detailing the baseline performance for the visual filtering effectiveness of the “EraX-NSFW-V1.0” model. The goal is to examine the system’s foundational capability to protect children by accurately distinguishing between “Safe For Work” (SFW) and “Not Safe For Work” (NSFW) visual media. It is important to note that the scope of this evaluation is focused on detecting explicit nudity and sexually suggestive content, as defined in Section 3.3.1. Other forms of potentially harmful material (like depictions of violence) were not part of the experiments. Furthermore, as established in Section 3.3.4, the visual analysis does not apply to most advertisements, as they are often delivered in isolated containers which are technically inaccessible to the extension.

To generate this dataset, an extensive data collection procedure was conducted across tens of distinct websites spanning a representative range of categories, including international news outlets (e.g., bbc.com), social media platforms (e.g., Reddit, X, Instagram), major sports sites (e.g., nba.com), and dedicated adult content platforms (e.g., pornhub.com, youjizz.com). As the user scrolls, the browser extension attempts to capture every visual element encountered (except visual elements with a width or height smaller than 128 pixels), and the backend API saves a copy of the original and classifies images in two separate folders: the predicted SFW frames and the NSFW frames as classified by the overall system.

To establish the ground truth for this experiment, the researcher subsequently performed a manual review of all images in both folders to verify the system’s classifications and identify any errors (to be more specific, missed SFW content in the NSFW folder or missed NSFW content in the SFW folder). The dataset’s SFW content was collected from general interest websites like news and sports platforms, where explicit material (while not absent) is not the

primary type of content. The NSFW content was sourced from a combination of dedicated adult websites, specific NSFW subreddits (Reddit’s communities), and explicit profiles on X. By using this approach, the dataset included content from social media feeds or general interest websites, which can contain borderline material (such as suggestive user-posted photos or revealing outfits in fashion articles) mixed with standard SFW images.

For each experiment, the data collection aimed to create a balanced dataset of over 300 SFW and 300 NSFW images. The slight variance in the total files processed per session, as reflected in Table 3, is a result of the practical constraints inherent in this live-browsing methodology, such as the data-transfer limits of the testing tools and the extensive manual effort required for ground-truth verification.

The evaluation for the baseline test employed a single, universal confidence threshold applied across all five models’ detection classes. The system’s sensitivity was therefore controlled by one single value applied uniformly across all five of the model’s detection classes. For example, a universal threshold of 0.2 requires the model to be at least 20% confident before flagging a detection of any class: “anus”, “make_love”, “nipple”, “penis”, or “vagina”. The universal threshold was established as the baseline approach after preliminary tests confirmed that a more granular method, such as setting different confidence thresholds for each explicit class (e.g., applying a lower threshold for a specific anatomical class like “penis” versus a higher one for a more suggestive class like “make_love”), did not yield a significant performance improvement for this task. The specific thresholds from 0.1 to 0.5 were selected for detailed analysis because preliminary testing indicated that thresholds beyond this range were not practical. Specifically, thresholds below 0.1 began to produce a notable increase in the rate of False Positives on safe content, indicating that any marginal gain in detection would come at a significant cost to the user experience. Conversely, thresholds above 0.5 proved impractical, as they produced an unacceptably high rate of False Negatives, failing to detect a significant amount of NSFW material. The system’s effectiveness at each threshold is measured using the standard classification metrics defined in Section 3.4.1 (e.g., Accuracy, Precision, and Recall), with the detailed results presented in the following table.

Mixed-Content Threshold	0.1	0.2	0.3	0.4	0.5
Total Files Processed:	684	707	681	804	691
Actual NSFW Files (TP+FN):	384	398	364	494	391
Actual SFW Files (TN+FP):	300	309	317	310	300
True Positives (TP):	343	307	264	246	52
False Positives (FP):	9	1	2	0	0
True Negatives (TN):	291	308	315	310	300
False Negatives (FN):	41	91	100	248	339
Metrics:					
Accuracy	0.927	0.870	0.850	0.692	0.509
Precision:	0.974	0.997	0.992	1.000	1.000
Recall:	0.893	0.771	0.725	0.498	0.133
F1-Score:	0.932	0.870	0.838	0.665	0.235
FPR	0.030	0.003	0.006	0.000	0.000
FNR	0.107	0.229	0.275	0.502	0.867

Table 3: Baseline performance of the universal threshold approach on “Mixed-Content” data

From a child safety perspective, the analysis must prioritize the metrics that quantify the risk of exposure to NSFW content, such as the False Negative Rate (FNR). This metric represents the proportion of actual NSFW images that the system fails to detect, making it the most critical indicator of the system’s ability to protect against such content. According to Table 3, the lowest FNR is (0.107) achieved at the lowest tested (0.1) threshold. This corresponds to a Recall value of 0.893, which is the highest detection coverage achieved. While this setting offers the best protection among the tested configurations, the remaining FNR of 0.107 indicates that there is still a substantial risk of exposure to harmful content.

While the primary goal is to create a protective tool, a smooth and non-disruptive browsing experience is also highly desirable. This aspect of the system is quantified by the False Positive Rate (FPR), which measures the proportion of SFW images that are incorrectly flagged as NSFW. The data in Table 3 shows that higher thresholds are well suited for this task, specifically the 0.4 and 0.5 settings with a perfect FPR of 0.0. This value translates to a perfect Precision of 1.000, indicating that when the system predicts an image as NSFW, that prediction is always correct, ensuring no SFW content is mislabeled. This perfect score, however, should be interpreted with caution, as it is likely a consequence of the specific size and composition of the SFW dataset collected for this baseline test (meaning the perfect score reflects the

performance on this specific experiment, and does not imply a universal zero error rate across all possible SFW images). Therefore, from a pure user experience perspective, these configurations offer a smooth performance without disrupting the user by incorrectly classifying safe content. However, given that the FNR at the 0.5 threshold was already at 0.867, testing even higher (and thus less sensitive) thresholds was deemed to have no practical utility for this project.

An in-depth analysis of the system’s performance also includes the holistic metrics of Accuracy and F1-Score, detailed Table 3. While the overall Accuracy appears strong, ranging from a peak of 0.927 at the 0.1 threshold down to 0.509 at the 0.5 threshold, this metric must be interpreted with caution. Since Accuracy measures the total proportion of correct classifications, it can be misleading on occasions where one class (such as SFW content) is more frequent, potentially masking a failure to detect the minority class of NSFW content [134]. The F1-Score also follows a downward trend, ranging from 0.932 to 0.235, and appears more reliable as it balances the trade-offs between Precision and Recall. The fact that both metrics peak at the 0.1 threshold suggests that the most sensitive configuration tested achieves the best statistical equilibrium in this experiment. To understand the context of balancing child safety with user experience, it is necessary to deconstruct these summary scores into their component metrics.

The performance indicated by the F1-Score is a result of the fundamental trade-off between Precision and Recall, as detailed in Table 3. For a child safety tool, Recall is the most critical of these two metrics, as it directly measures the system’s success in correctly identifying NSFW content. The data shows that Recall peaks (0.893) at the lowest tested threshold of 0.1 and declines as the threshold increases, falling to 0.133 at the 0.5 threshold. Inversely, Precision measures the reliability of the positive classifications made by the model, which improves as the threshold increases. Specifically, it increases from 0.974 at the 0.1 setting to a perfect 1.000 at thresholds of 0.4 and above. Such a conflict between detection coverage (Recall) and detection reliability (Precision) is central to the system’s design.

Ultimately, the analysis of moderate value thresholds confirms the limitations of a static approach. The 0.2 threshold, for instance, offers a very low FPR of just 0.003, which is highly desirable for user experience. However, the same setting achieves an FNR of 0.229, indicating that a notable portion of NSFW content remains undetected. This emphasizes a notable challenge of a static threshold approach, where even a well-balanced threshold struggles to provide a high level of safety and a seamless user experience simultaneously. The analysis of static thresholds makes it clear that an effective system requires a dynamic strategy, one that can leverage the benefits of a low threshold in risky contexts while maintaining the high-precision benefits of a higher threshold.

4.2 Enhancing Detection with Context-Aware Adaptive Filtering

Although a common tactic for improving performance is to fine-tune the AI model itself, that approach was deemed infeasible for this research. The significant financial cost and computational resources required, such as the multi-GPU server used for the original “EraX-NSFW-V1.0” model’s training, were unavailable [20]. Furthermore, large and precisely labeled public datasets necessary for such a task are scarce for individual researchers, especially given the nature of the task [49, 82]. Therefore, this thesis focuses on a more efficient, system-level approach to enhance performance by leveraging the existing pre-trained model more intelligently.

A standard methodology for this classification problem would be to plot a Receiver Operating Characteristic (ROC) curve because it would visualize the model’s performance by plotting the Recall against the FPR at various thresholds to help identify an optimal balance [135]. Similarly, a Precision-Recall (PR) curve could also be used, as it plots Precision against Recall for different thresholds. This method is often more informative for evaluating performance on imbalanced datasets, as it focuses on the trade-off in detecting the positive classes [135]. However, while these methods are effective for finding the best single static threshold, the analysis in the previous section demonstrates that any such “optimal” point necessitates a compromise between safety and user experience.

Therefore, this thesis explores an alternative approach: a dynamic technique designed to leverage the distinct advantages of both high and low thresholds. The goal is to achieve high detection of NSFW elements with a low threshold (high Recall and low FNR) when risk is present, while maintaining the non-disruptive user experience of a high threshold (high Precision and low FPR) during general browsing. To achieve this, the system operates from a default threshold of 0.2 along with three different mechanisms. The default value was chosen from the analysis in Table 3 as an effective threshold because it provides a very low False Positive Rate of 0.003 and a high Precision of 0.997, ensuring a stable and non-disruptive experience.

The first mechanism of the adaptive strategy is “Domain Analysis”, a content-based filter implemented within the backend Python script. This technique uses regular expressions (a sequence of characters that specifies a search pattern in text [34]) to scan the page’s URL, which is sent from the browser extension with each request. The regex patterns are designed to identify known risks, such as domains containing keywords explicitly associated with adult content (e.g., “porn”, “xxx”) or patterns that could potentially match specific NSFW subsections on general-interest platforms like Reddit and X.com. If the URL matches one of these predefined patterns, the system’s logic overrides the 0.2 default threshold and applies a much lower threshold of 0.005 to all images processed from that domain.

Similarly, the second mechanism is a “Textual Context Analysis” that occurs at the backend and analyzes the alt text (a brief textual description of an image [116]) that is sent along with each image from the extension. Using the same set of regular expression patterns, this mechanism scans the alt text for a comprehensive list of unsafe keywords, including common profanity and terms related to pornography. The keyword list was methodically compiled from empirical observations of recurring terminology gathered during the experimental browsing phase on adult content websites. If an image’s alt text contains a high-risk keyword, the threshold for all subsequent images on that page is lowered to 0.005.

The third dynamic mechanism directly leverages the reliability findings from the baseline experiments to create an adaptive feedback loop. The loop is triggered after the backend analyzes an image and produces a detection with a confidence score greater than 0.5 for the classes that are detected by the “EraX-NSFW-V1.0” model. Although the 0.4 threshold also demonstrated perfect Precision in the baseline tests, the higher 0.5 value was selected as the trigger to provide maximum assurance (in other words, to be as certain as possible before taking the significant action of escalating an entire domain) that a detection is a True Positive before escalating the entire domain to a high-risk status. This choice is justified by the results in Table 3, where a perfect Precision of 1.000 at this level makes such a detection a highly reliable indicator of genuinely harmful content. Upon such a detection, the backend signals the extension to flag the entire website domain as “high-risk”. For all subsequent images processed as long as the user remains on that same website, the system then applies the same low threshold (0.005) as in the aforementioned mechanisms. This mechanism is applied temporarily for the browsing session and resets when the user navigates to a different website, ensuring the system returns to its default 0.2 threshold for new domains.

This dynamic approach forms an overall robust defense. All the mechanisms are implemented to cover each other’s potential misses. For example, the content-based rules provide immediate protection against pages with obviously dangerous textual patterns, such as URLs or alt-text containing profanity or words like “porn”. Simultaneously, the model-driven feedback loop acts as a critical safety net for when there are no textual clues. In essence, the system operates with a default confidence threshold of 0.2 and dynamically lowers it to a high-sensitivity setting of 0.005 whenever a risk indicator is met. This near-zero value was chosen to maximize the model’s sensitivity (Recall) in contexts predetermined to be high-risk. This approach was validated during preliminary tests, and preliminary tests confirmed that any value in this near-zero range (values under 0.01, such as 0.005, 0.001) produced similarly high-recall results. It should be noted that such a safety-first approach means the system is designed to flag visual anatomy regardless of context, making the occasional incorrect blurring of clinical or educational content a necessary consequence of its core protective function.

The overall effectiveness of this completed dynamic adaptive system was evaluated using a dataset gathered through a structured browsing methodology. The data collection involved intentionally navigating a diverse range of websites, including those known to trigger each of the three adaptive mechanisms. This process was designed to systematically stress-test the adaptive logic by ensuring it was exposed to high-risk domains, text containing unsafe keywords, and content likely to produce a high-confidence model detection. The resulting

sample of 1560 processed files, while not exhaustive, represents a dataset specifically compiled to evaluate the system’s effectiveness across all of its operational states. This collection method yielded a realistic mix of content, including explicit material, clearly safe media, and, importantly, the borderline or suggestive material commonly found on social media platforms. The inclusion of such borderline and suggestive material represents a key methodological distinction from the comparative benchmark (detailed in Section 4.3). The purpose of the dataset in this section was to evaluate the adaptive system against the nuanced and unpredictable content encountered during live browsing, in contrast to the benchmark’s use of only unambiguous content for a controlled comparison of the tools’ core capabilities. The sample size was primarily constrained by the practical factors of data transfer limits and the significant time required for manual review [136, 137]. The results of this evaluation are presented in the following table.

Total Files Processed:	1560
Actual NSFW Files (TP + FN):	1009
Actual SFW Files (TN + FP)	551
True Positives (TP):	1005
False Positives (FP):	6
True Negatives (TN):	545
False Negatives (FN):	4
Metrics:	
Accuracy	0.994
Precision:	0.994
Recall:	0.996
F1-Score:	0.995
FPR	0.011
FNR	0.004

Table 4: Performance of the Context-Aware adaptive filtering system

The results presented in Table 4 reveal a notable improvement in the baseline performance. The notable improvement in baseline performance is immediately evident when comparing the F1-score of the adaptive system (0.995) with the best-performing static threshold from the baseline (0.932 at the 0.1 setting). This score indicates the system achieved a near-perfect balance between effectively blurring NSFW content and preserving a smooth user experience. Deconstructing this, the Recall value (0.996) surpasses the baseline’s highest Recall (0.893), indicating the adaptive system is significantly more effective at identifying the NSFW content it encounters. Simultaneously, its Precision (0.994) remains exceptionally high, rivaling the baseline’s most precise (but least effective) configurations without making the same drastic trade-off in detection capability. The ability to achieve these two high scores concurrently confirms that the adaptive strategy tackles the fundamental trade-off to a significant extent. In

other words, it adapts to the content encountered during browsing and exploits the advantages of a low threshold and a high threshold.

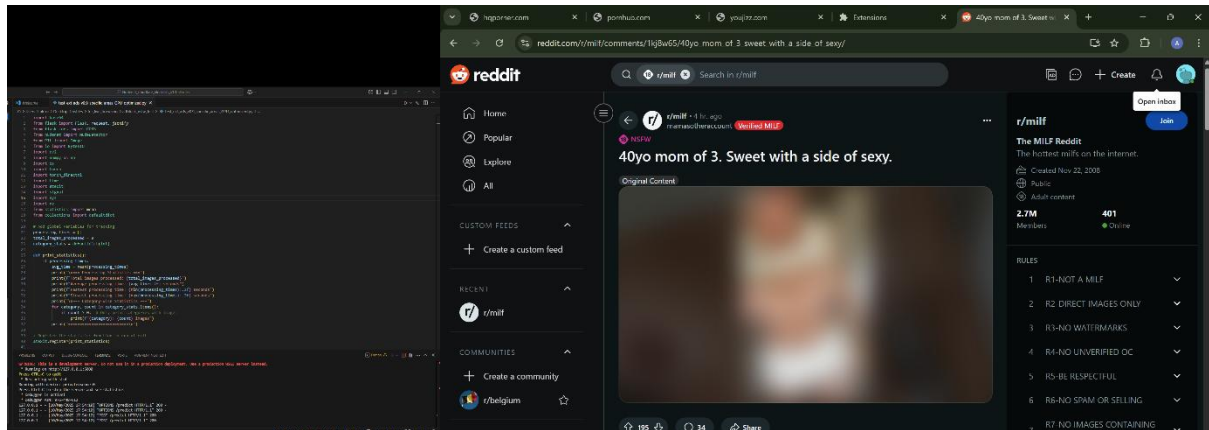


Figure 3: An example of the system's high recall, correctly identifying and blurring explicit content

A more in-depth analysis of the error rates provides a clearer picture of the system's enhanced protective capability. The most critical metric for a child safety tool, as also mentioned earlier, is the FNR, which represents the proportion of NSFW content the system misses. The adaptive technique achieved an FNR of just 0.004, a substantial reduction from the 0.107 FNR that was observed in the lowest threshold (0.1). Importantly, this gain in safety was achieved without sacrificing user experience, as the system's FPR of 0.011 also marked an improvement over the 0.030 baseline setting. This demonstrates the adaptive logic's ability to be both stricter in detecting harmful material and more accurate in allowing safe content to pass, successfully optimizing for both safety and usability.

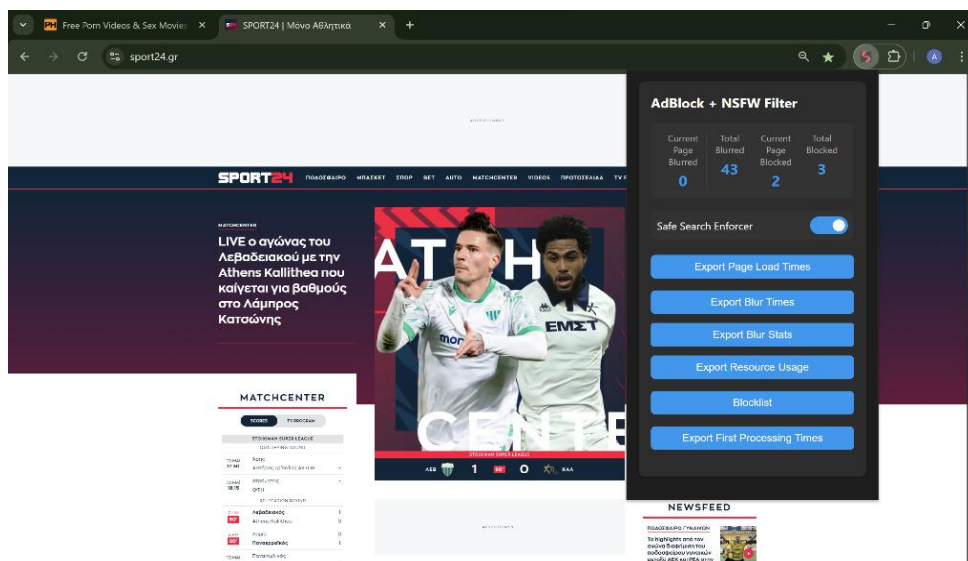


Figure 4: An example of the system's high precision, correctly allowing safe content to be displayed

However, it should be acknowledged that even with the adaptive logic, the system is not perfect. The evaluation identified 4 False Negatives and 6 False Positives, and while small, these errors highlight the inherent performance boundaries of any AI-based filtering tool. The FNs are crucial. While a specific root-cause analysis for each failure was outside the scope of this project, it is plausible that these errors are instances of the exact challenges for object detectors discussed in the literature review: the model’s known difficulty in detecting subjects that are small, partially occluded, or in a visually complex scene [66]. The purpose of using a realistic “in-the-wild” data collection method, rather than a perfectly curated dataset, was to expose the system to the variability and unpredictability of live web content. The emergence of these few errors is therefore not just a limitation but a valuable finding. It empirically demonstrates the practical performance ceiling of the underlying object detection technology when faced with state-of-the-art challenges. The result provides a clear, evidence-based direction for future work focused on improving model robustness against the specific, known failure modes of occlusion and small object detection [66].

It is also critical to acknowledge that the high performance of the adaptive system is contingent on the presence of these contextual triggers. In a “heuristically sterile” environment (such as a webpage deliberately designed by a malicious actor to contain harmful visual media without any corresponding textual cues or high-confidence initial images), the system’s performance would revert to its non-adaptive baseline, as detailed in Section 4.1. Therefore, the adaptive logic should be understood as a significant and practical enhancement for the typical browsing scenarios evaluated in this research. The challenge of defending against sophisticated, targeted evasion, which is discussed in more detail in Section 5.2.8, remains an important and distinct area for future work.

Concluding, this section demonstrated that the implemented technique represents a substantial improvement over a static threshold approach. By dynamically adjusting the sensitivity of the model based on the context, the system can provide robust protection against NSFW content while maintaining a pleasant user experience even when tested in a varied and uncontrolled browsing experience. It is notable that such a significant gain in detection effectiveness was achieved through a computationally lightweight adaptive logic. However, it is also important to acknowledge that these results were achieved on an “uncontrolled” real-world dataset. While this method was chosen to test the system’s robustness, the specific composition of the encountered content was not pre-determined, which is an inherent limitation of this evaluation. Having examined the effectiveness of this adaptive logic, the subsequent section will conduct a direct comparative benchmark of the developed system against alternative filtering tools under identical and controlled conditions to provide a comprehensive evaluation of its performance.

4.3 Comparative Benchmark Against Alternative Filtering Solutions

This section presents a direct comparative benchmark (using 180 SFW images and 180 NSFW images) of the developed system against two distinct alternative filtering technologies. To create a comprehensive evaluation, the system was tested against both a different backend model operating on the same client-server architecture (“NudeNet” based on YOLOv8n [35]) and a client-only solution (“NSFW Filter” based on MobilenetV2 [36]). The goal of this analysis is to contextualize the performance of the developed system by directly comparing the effectiveness of these different filtering tools. The scope of this benchmark is to evaluate these tools, simulating how a user would experience them, rather than merely comparing the performance of the underlying models in isolation. The general methodology for this benchmark, including the creation of the controlled dataset and the foundational setup for each tool, is described in Section 3.3.3.

The developed system, referred to as “EraX-NSFW-V1.0” in the benchmark [20], was evaluated in two distinct configurations to clearly demonstrate the impact of its adaptive logic. First, it was tested using a static confidence threshold of 0.2. This specific threshold was chosen because the baseline analysis in Section 4.1 identified it as the threshold that offered the most favorable balance between a low FPR (0.003) and a moderate FNR (0.229), making it the most logical non-adaptive configuration for comparison. It also provides a baseline to measure the system’s performance before the context-aware rules are applied. Subsequently, the system was tested with its full context-aware adaptive rules enabled, a configuration that represents the complete proposed system. Comparing these two configurations allows for a direct and quantitative measurement of the performance improvement provided by the custom-developed adaptive technique.

While the documentation of the “NSFW Filter” does not provide a precise technical definition for this setting, it functionally appears to control the model’s sensitivity via a percentage slider. A higher sensitivity percentage seemingly makes the filter stricter and more likely to classify an image as NSFW. A sensitivity of 87% was selected as the primary benchmark configuration, as this value is featured in the tool’s official documentation and represents a high-sensitivity setting [36]. To create a more comprehensive analysis of its performance across its full operational range, it was also tested at several other distinct levels, specifically 5%, 25%, 50%, and 60%.

To ensure a fair architectural comparison, “NudeNet” was tested using the same client-server implementation as the developed system (which uses “EraX-NSFW-V1.0”), running on the same backend API and communicating with the same browser extension. Furthermore, to align the comparison with the thesis’s goals of filtering explicit nudity, the experiment was configured to only consider the most explicit labels of the “NudeNet” model (e.g., “FEMALE_GENITALIA_EXPOSED”, “ANUS_EXPOSED”) while intentionally disregarding its more general or non-explicit classes (e.g., “FACE_FEMALE”,

“BELLY_COVERED”) that would lead to a high number of FPs [35]. Finally, the specific confidence thresholds of 0.2, 0.3, 0.4, and 0.5 were selected for testing because preliminary experiments (small-scale tests across multiple threshold values) determined these values offered the most representative and competitive performance for this specific task. Specifically, lower thresholds produced an unacceptably high number of False Positives on safe content, while higher thresholds proved ineffective at detection, producing a high number of False Negatives.

The decision not to apply the custom context-aware adaptive logic to the “NudeNet” model was a deliberate methodological choice, guided by the initial hypothesis established in Section 2.4.2. Based on public benchmarks and a preliminary analysis, which included initial, small-scale evaluations of both “NudeNet” and “EraX-NSFW-V1.0”, it was hypothesized that the “EraX-NSFW-V1.0” model would offer a more effective foundational performance. The results of the controlled benchmark presented in this section (Table 5) while based on a modest dataset, are consistent with this initial hypothesis. Specifically, the “EraX-NSFW-V1.0” model demonstrated a more reliable detection capability, achieving a lower FNR (the percentage of harmful images the system fails to detect) within this controlled experimental setting. Given that the formal benchmark data aligned with the initial hypothesis, it was determined that the most scientifically valuable path was to apply and validate the adaptive enhancements on the empirically stronger foundational candidate. This approach allowed for a focused investigation into the maximum potential of the adaptive system when built upon the more reliable model.

The following table presents the comprehensive results of this comparative benchmark. For the “Developed System”, performance is shown first for the “Baseline - 0.2 Threshold” (as demonstrated in Section 4.1) and then with the “Adaptive Rules Enabled” (the adaptive mechanism as demonstrated in Section 4.2) to demonstrate the impact of the custom logic. The performance of the client-only “NSFW Filter” is detailed across a range of its sensitivity settings. Finally, the “NudeNet” model is evaluated at its most competitive thresholds (0.2, 0.3, 0.4, 0.5), running on the same backend architecture as the developed system to ensure a fair comparison.

Configuration	Developed System (Baseline - 0.2 Threshold)	Developed System (Adaptive Rules Enabled)	NudeNet (0.2 Threshold)	NudeNet (0.3 Threshold)	NudeNet (0.4 Threshold)	NudeNet (0.5 threshold)
Total Files Processed:	360	360	360	360	360	360
Actual NSFW Files (TP + FN):	180	180	180	180	180	180
Actual SFW Files (TN + FP)	180	180	180	180	180	180
True Positives (TP):	170	179	158	147	156	86
False Positives (FP):	0	0	2	0	0	0
True Negatives (TN):	180	180	178	180	180	180
False Negatives (FN):	10	1	22	33	44	94
Metrics:						
Accuracy	0.972	0.997	0.933	0.908	0.884	0.739
Precision:	1.000	1.000	0.988	1.000	1.000	1.000
Recall:	0.944	0.994	0.878	0.817	0.780	0.478
F1-Score:	0.971	0.997	0.929	0.899	0.876	0.647
FPR	0.000	0.000	0.011	0.000	0.000	0.000
FNR	0.056	0.006	0.122	0.183	0.220	0.522
Configuration	NSFW Filter (5% Sensitivity)	NSFW Filter (25 % Sensitivity)	NSFW Filter (50% Sensitivity)	NSFW Filter (60% Sensitivity)	NSFW Filter (87% Sensitivity)	
Total Files Processed:	360	360	360	360	360	
Actual NSFW Files (TP + FN):	180	180	180	180	180	
Actual SFW Files (TN + FP)	180	180	180	180	180	
True Positives (TP):	129	163	172	175	177	
False Positives (FP):	19	35	46	50	58	
True Negatives (TN):	161	145	134	130	122	
False Negatives (FN):	51	17	8	5	3	
Metrics:						
Accuracy	0.806	0.856	0.850	0.847	0.831	
Precision:	0.872	0.823	0.789	0.778	0.753	
Recall:	0.717	0.906	0.956	0.972	0.983	
F1-Score:	0.787	0.862	0.864	0.864	0.853	
FPR	0.106	0.194	0.256	0.278	0.322	
FNR	0.283	0.094	0.044	0.028	0.017	

Table 5: Comparative performance benchmark of filtering solutions

The results from the controlled benchmark detailed in Table 5 indicate that the Developed System (“Adaptive Rules Enabled”) achieved the most effective and balanced performance. This is quantitatively supported by its F1-score of 0.997, which suggests a strong equilibrium between precision and recall, and it outperforms both “NudeNet” (0.929 at a 0.2 threshold) and the “NSFW Filter” (0.864 at 50% and 60% sensitivity). A deeper analysis of the specific metrics can help deconstruct the strengths and potential trade-offs of each filtering solution. To understand these results in detail, the performance of each system can be understood by examining two key aspects of performance. The first is each system’s ability to handle SFW content, necessary for a non-disruptive user experience. The second is its effectiveness in detecting NSFW content, which determines its safety value. As the following analysis demonstrates, the different architectural approaches of the tested tools force them into a direct conflict between these two critical goals.

In evaluating the first key aspect of performance, the handling of SFW content, a system’s ability to avoid FPs is important for maintaining a non-disruptive user experience. The Developed System with “Adaptive Rules Enabled” demonstrated flawless performance in this regard. As shown in Table 5, it achieved a perfect Precision of 1.000 and a corresponding FPR of 0.000, indicating it made no errors in classifying safe images during the controlled test. “NudeNet” also proved highly effective, achieving excellent results. It maintained perfect Precision (1.000) at the higher thresholds (0.3, 0.4, and 0.5), and even at its most sensitive tested configuration (0.2 threshold), it maintained a high Precision of 0.988 with a minimal FPR of just 0.011, making only two misclassifications on safe content. In contrast, the client-only “NSFW Filter” (at 87% sensitivity) struggled, registering a high FPR of 0.322. This result means that the filter incorrectly blocked nearly a third of all SFW images, which might lead to a frustrating user experience. This quantitative finding of a high rate of FPs appears consistent with some qualitative user feedback for the tool’s underlying library (“NSFWJS”), where community-submitted issues have reported False Positives on non-explicit subjects such as artwork or animals [138, 139, 140]. Such a tendency for FPs may be due to their underlying architecture. As seen in Section 2.2.1, “NSFWJS” performs image classification, categorizing the entire image as “Sexy” or “Porn” based on overall content [57], rather than object detection, which precisely identifies specific anatomical parts. This can potentially make it more susceptible to misinterpreting context in suggestive but non-explicit images, clearly prioritizing safety at the expense of user experience.

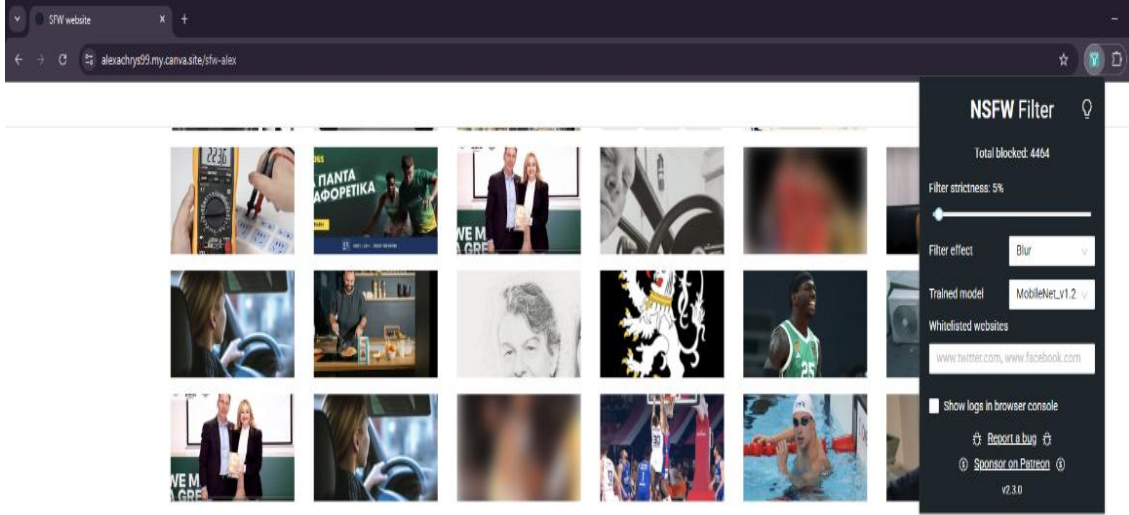


Figure 5: The “NSFW Filter” at low sensitivity (5%) on a safe website, resulting in several False Positives

While the user experience is important, the most critical aspect of the system’s performance is the effectiveness in detecting NSFW content. It is measured by the Recall, which represents the percentage of harmful items successfully identified, and by the FNR, which is the percentage missed. The Developed System (“Adaptive Rules Enabled”) proved to be the most effective, achieving an exceptional Recall of 0.994 and the lowest FNR of just 0.006. The “NSFW Filter” (at 87% sensitivity) also demonstrated strong detection capability, with a high Recall of 0.983 and a low FNR of 0.017. In contrast, “NudeNet” (even at its best-performing 0.2 threshold) was significantly less reliable as a safety tool in this experiment. Its Recall was much lower at 0.878, resulting in an FNR of 0.122, which indicates it failed to detect over 12% of the harmful images in the test, a performance profile that appears to prioritize a non-disruptive user experience at the cost of its safety function. Based on these results, while the “NSFW Filter” is also effective at detection, the Developed System is the most dependable for this primary safety task, and “NudeNet” is the least reliable.

Despite the developed system’s (using “EraX-NSFW-V1.0”) superior performance with “Adaptive Rules Enabled”, it is important to acknowledge the limitations of the developed system and the context of this experiment. Even the best tool was not perfect, as evidenced by the False Negative recorded during the benchmark test in Table 5. This reflects a recognized challenge for object detectors and, more specifically, for the YOLO-based frameworks (a limitation that also applies to “NudeNet”, which is based on YOLOv8), a point discussed in Section 5.2.4. Such models can struggle to detect small, overlapping, or occluded objects within complex environments [66]. It is plausible that this specific failure occurred because the target anatomical region was so small or visually ambiguous that the model failed to generate a meaningful confidence score, leading to a False Negative even with the system’s high sensitivity.

However, the performance gap observed between the “EraX-NSFW-V1.0” and “NudeNet” models could be attributed to architectural nuances within the YOLO family. As seen in Section 2.4.2, the YOLOv11n framework is a more recent and efficient design than YOLOv8n, demonstrating a modest (~6%) improvement in accuracy on public benchmarks while using fewer parameters. For a specialized task like NSFW detection, which often depends on identifying small or fine-grained details, such a “modest” improvement on a general benchmark can translate into a decisive performance advantage [95, 96]. The higher performance of the developed system in this benchmark is consistent with the initial hypothesis established in Section 2.4.2. The finding suggests that the architectural improvements in the underlying YOLOv11n framework, such as its greater efficiency in distinguishing complex features, are a plausible explanation for the significantly lower FNR observed in the Developed System.

Furthermore, the scope of this comparative benchmark must be considered. The evaluation was conducted on a manually curated dataset of 360 images. Although modest in scale, this approach was necessary to establish a controlled and perfectly balanced testing environment, ensuring a fair comparison between the capabilities of different tools. It is also important to note that there is inherent subjectivity in the term “NSFW”, as each model is trained on different private datasets and may use different criteria to define what constitutes unsafe content, a factor that likely contributes to the different error profiles seen across the tools.

The preceding analysis demonstrates a clear performance trade-off among the tools, establishing that only the Developed System with the configuration of its adaptive technique (“Adaptive Rules Enabled”) effectively balances user experience with robust safety. It achieved this by adjusting its sensitivity using its three adaptive techniques as mentioned in 4.2. It is critical to acknowledge, however, that such a superior performance is contingent on the presence of these contextual triggers, as in a heuristically-sterile environment designed to evade them, the system’s performance would revert to its strong, but less effective, baseline. The “NSFW Filter” (at 87% sensitivity), while offering high detection rates, may disrupt user experience due to its high rate of FPs on safe content. It is important to acknowledge the architectural trade-off regarding responsiveness. The client-only design of the “NSFW Filter” gives it an inherent latency advantage. By operating entirely within the browser, it avoids the significant data serialization and network communication overhead that was measured at approximately 200 ms for the client-server architecture (as detailed in Section 4.4). While a direct, programmatic latency comparison was not feasible due to the technical impossibility of instrumenting a third-party extension, this architectural superiority in speed was qualitatively evident during testing, with interventions from the “NSFW Filter” appearing more immediate. Conversely, “NudeNet” (at its 0.2 threshold) provides a much better user experience but sacrifices its core safety function by failing to detect a notable portion of harmful content. Beyond the performance advantage, the Developed System holds a critical architectural advantage. It was designed to be fully compliant with Google’s Manifest V3 [17], the current standard for Chromium browser extensions. This makes it a more sustainable solution than the “NSFW Filter”, which relies on the Manifest V2 framework that faces deprecation and reduced functionality in the modern browser ecosystem [23].

In summary, the data from the controlled benchmark of this section indicates that within the context of this experiment, the developed adaptive system with its adaptive mechanisms offered a more accurate and reliable solution than the tested alternatives, while also being built on an architecturally sustainable framework. Notably, the adaptive mechanisms (as described in Section 4.2) provided a clear improvement over the system’s baseline configuration, which on its own already demonstrated competitive performance against the other tools. Given these findings on detection accuracy, the analysis can now shift to evaluate the system’s real-world performance and responsiveness. Therefore, the following section will examine the system’s end-to-end processing latency to quantify its impact on the user’s browsing experience.

4.4 Analysis of System Responsiveness and Latency

Beyond detection accuracy, the overall responsiveness of the developed system plays a critical role in the user’s experience. The unexpected appearance of NSFW content can be a disruptive and potentially harmful factor during the browsing experience [2]. Assessing these timing aspects is crucial, as they directly indicate if the system is suitable for real-time NSFW protection. Therefore, this section moves beyond accuracy metrics to quantitatively estimate the system’s speed under realistic conditions. The responsiveness metric for this analysis is the “Time to First Intervention”. As established in Section 3.4.2, the analysis focuses on this initial measurement to avoid confounding variables like queuing delays and to provide the cleanest measure of the system’s true reaction speed.

To specifically simulate the system’s protective function in a high-risk environment, the latency data presented in the following figure was collected exclusively while browsing adult content platforms where immediate intervention is critical.

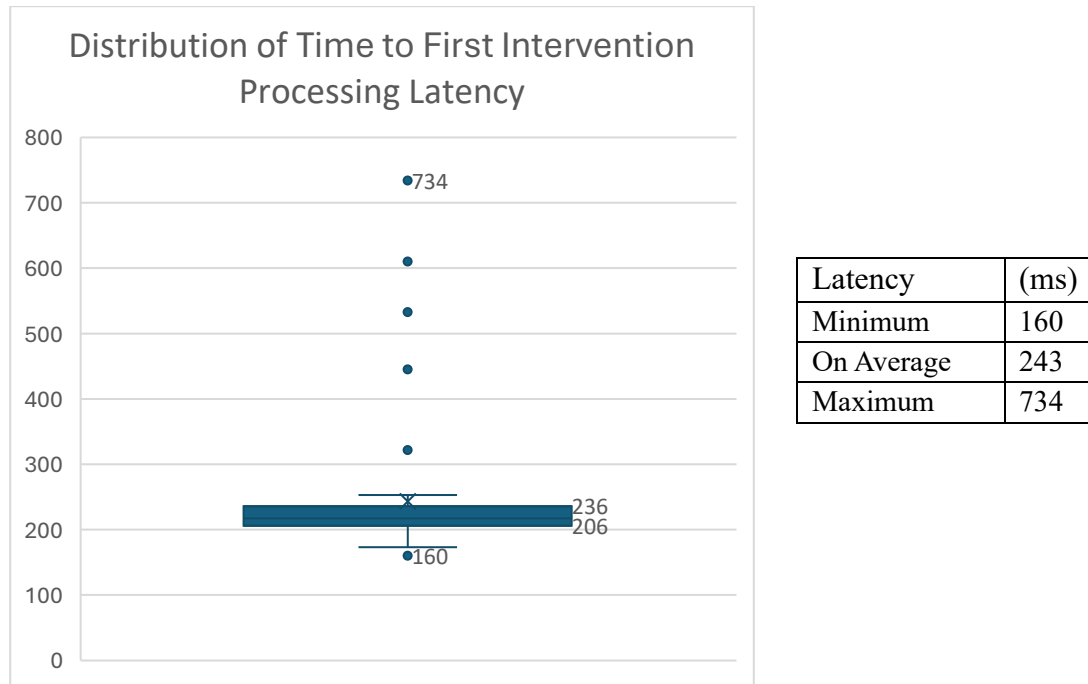


Figure 6: Distribution of “Time to First Intervention” on explicit content domains

To obtain a comprehensive understanding of the system’s performance, latency was measured at three key points. First, the “Time to First Intervention” was measured programmatically. The process begins within the “content.js” script (the extension’s primary script for interacting with web page content), which records an initial timestamp using “performance.now()” (a high-resolution timer for web browsers) at the moment the page finishes loading. The final elapsed time is calculated after the first visual element is fully processed. Second, to isolate the server’s contribution to the delay, the backend’s processing time was measured directly within the Python script of the Flask API. The duration of each request to the “/predict” endpoint was recorded using Python’s time module, averaging approximately 100 ms per frame during testing (this measurement reflects the backend processing time only and does not include network latency from the tunneling service). For a direct architectural comparison, the same measurement was performed for the “NudeNet” model running on the identical backend, which yielded a slower average processing time of approximately 160 ms. This finding provides further support for the initial selection of the “EraX-NSFW-V1.0” model, which proved to be not only more accurate in the benchmark but also more computationally efficient on the backend. Finally, a separate experiment was conducted to isolate the model’s (“EraX-NSFW-V1.0”) speed. In this experiment, Python’s “time.perf_counter()” (a high-precision timer in Python) in a loop with a warm-up phase (a common practice to ensure the system is in a steady state before measurement [130]), the measured average inference time for the “EraX-NSFW-V1.0” model using PyTorch on the test CPU was 39.54 ms.

The results presented in Figure 6 provide a detailed statistical picture beyond a single average. While the mathematical average “Time to First Intervention” was 243 ms, the data shows that 50% of all response times fell within a more representative interquartile range of 206 ms to 236 ms. The average is skewed higher by several outliers, which ranged up to 734 ms and are likely caused by external factors like network congestion or client-side CPU spikes. As detailed in Section 3.3.6, a system-level warm-up was conducted before data collection (the overall system was operating and processing over 100 images before recording began), a procedure distinct from the loop-based warm-up used to isolate the model’s inference time. Such an approach was followed to account for the initial “cold start” delay of the overall system (backend and extension) [130]. A comprehensive understanding of this latency is achieved by comparing it to the backend-specific measurements to isolate the various sources of delay. With an average backend processing time measured at approximately 100 ms, the remaining ~143 ms of overhead can be attributed to the client-side processing and network communication required by the system’s architecture.

The ~143 ms of client-side and network overhead is composed of several distinct processes that occur before and during transmission. First, a notable component of this client-side overhead is the DOM (a representation of the page’s structure that allows the extension to access and capture image elements [39]) traversal required by the extension. In order to detect images to analyze and subsequently blur them if they are classified as NSFW, the “content.js” script must query the page’s DOM, specifically to handle dynamically loaded content (in other words, blur images when the user scrolls down a website). This process of searching for and accessing page elements introduces latency. This is a known performance consideration, as developer guidelines from browser vendors warn that a high number of complex DOM

manipulations by content scripts can negatively impact page load times and responsiveness [123]. As mentioned in Section 2.2.1, while offloading tasks to a Web Worker is a common optimization strategy, it was not viable for this core task, as Manifest V3’s security model prevents Web Workers from having the direct DOM access required to capture image data from the page [40, 58]. After an element is found, the script must capture its image data. This involves rendering the visual media onto an in-memory HTML canvas (a browser element used for drawing graphics) using the “drawImage()” function (a method for drawing an image onto a canvas element). The canvas operation is a necessary intermediate step because the Base64 string can only be generated from the pixel data on the canvas and not directly from the original “” or “<video>” element. This rendering process itself consumes processing time and contributes to latency, a known performance consideration detailed in browser developer documentation [41].

The script then performs the Base64 encoding, which converts the image into a text format (and subsequently analyzed by the “EraX-NSFW-V1.0” model) [41]. This step is an important part of the architectural design, as it converts the image into a text format [141]. This method was necessary to ensure compatibility with modern browser security policies (e.g., CORS, CSP) and the constraints of Manifest V3 (which can block the direct transmission of binary image data) [17, 29, 30]. Another important factor is the overhead from the network’s round-trip time, as the data must be transmitted from the browser to the backend API via a tunneling service. This rerouting of traffic through the tunneling service introduces latency in two ways, first, it forces the data to travel a longer physical distance to the service’s public servers and back (a phenomenon known as the “trombone effect” where network traffic must travel to a remote point and back, much like the slide of a trombone) [142, 143], a round trip whose duration is directly dependent on the user’s own internet connection speed (both upload and download), and it adds computational overhead from encrypting and decrypting the secure connection.

While a precise breakdown of each component’s contribution was not measured, it is reasonable to infer their relative impact. The network round-trip and the Base64 encoding process (which is computationally intensive and increases data payload size by approximately 33% [141]) likely represent the most substantial portions of this delay, whereas DOM traversal and canvas rendering for a single image are likely minor in comparison [41, 123].

The 100 ms backend processing time, which occurs after the data is received, is also a combination of components. The model’s inference, which was measured at 39.54 ms, and the remaining ~60 ms of backend overhead, consists of other server-side tasks essential for the system’s operation and evaluation. This includes the time spent on Base64 decoding (converts the text data received from the browser back into an image for the model to analyze) and the I/O (Input/Output) delay from saving image files to the local machine (a requirement of the research methodology for the manual verification of the model’s effectiveness). It is important to note here that the custom adaptive logic itself contributes negligibly to the aforementioned backend overhead, as its simple conditional checks and regular expression matching are computationally inexpensive and represent a tiny fraction of the total processing time.

The 39.54 ms measurement for the nano variant is a key finding. This direct, real-world measurement was performed using the PyTorch framework on the project’s specific hardware and differs from the standardized ONNX CPU benchmarks used in Section 2.4.2 for the initial framework comparison. This distinction is important. While standardized benchmarks are essential for an objective comparison between different frameworks in a controlled setting, this direct, project-specific measurement is a more practical indicator of the actual performance a user would experience with this thesis’s specific hardware and software configuration. The faster time measured here is expected, as performance is highly dependent on the specific machine (AMD Ryzen 7 8840U). These results also finalize the justification for using the nano variant. Initial tests on the small variant yielded a much slower inference time of over 100 ms and caused system instability (temporary freeze) on the test hardware, which relies on integrated graphics. Considering the fixed overhead of approximately 200 ms from network and data preparation tasks, the nano variant’s ~40 ms performance keeps the total latency under the 250 ms human perception threshold. The small variant’s ~100 ms performance, however, would have pushed the total latency well beyond this critical limit, confirming that the nano variant was the only practical and effective choice for this project.

This detailed breakdown reveals a critical insight: the core model inference (~40 ms) represents only a fraction of the total user-perceived latency. For context, research on human-computer interaction indicates that a response time of 100 ms is perceived as instantaneous [31]. While the system’s typical response time of 206-236 ms is fast enough to operate within a typical human reaction time of 250 ms, it is clear that there is a significant area for improvement to make the user experience feel truly seamless. A more rigorous analysis concludes that the main performance bottleneck is the ~200 ms of architectural overhead, not the AI model itself. This distinction is paramount, and as will be discussed in Chapter 6, future work should address this by exploring architectural enhancements and client-side optimizations to reduce the latency inherent in the data preparation and transmission pipeline.

4.5 Chapter Summary and Key Findings

This chapter detailed the empirical evaluation of the developed visual content filtering system. The initial baseline tests revealed the inherent challenge of using a single, static confidence threshold (Section 4.1). These tests showed that even the best protective setting (at a 0.1 threshold) still resulted in a 0.107 FNR, while settings that eliminated False Positives came at the cost of missing over 50% of NSFW content. To address this trade-off, a context-aware adaptive system was implemented (Section 4.2). The results demonstrated that this adaptive approach was highly effective, reducing the crucial FNR to just 0.004 on a large, uncontrolled dataset.

This was further supported in a controlled comparative benchmark (Section 4.3). In this benchmark, while small, the developed system with its adaptive mechanisms offered a superior balance of metrics, achieving an F1-score of 0.997 and an FNR of 0.006. Within the context of this specific experiment, it was the only system to maintain both a high rate of NSFW detection and a near-zero rate of errors on safe content.

Beyond pure detection accuracy, the chapter analyzed the system’s practical and architectural aspects (Section 4.4). The latency analysis revealed a typical “Time to First Intervention” between 206 and 236 ms, a speed sufficient to intervene within a typical human reaction time. A crucial finding emerged from this analysis. The primary performance bottleneck is not the AI model’s inference time of approximately 40 ms. Instead, the bottleneck is the roughly 200 ms of architectural overhead required by the client-side data pipeline. That pipeline includes tasks like DOM traversal (searching the webpage for images), canvas rendering, and Base64 encoding. The overhead of ~200 ms represents the inherent architectural cost of operating within the strict security constraints of Manifest V3. Identifying and quantifying this cost is a significant finding of this research, as it precisely defines the primary bottleneck and establishes a clear target for future optimization efforts.

5 System Analysis: Constraints, Limitations, and Ethical Considerations

Building upon the methodology and results, this chapter presents a critical analysis of the developed filtering system. It first outlines the technical and experimental constraints that dictated the system’s architecture (Section 5.1). Subsequently, it examines the inherent limitations of the final filtering approach (Section 5.2) and concludes with a discussion of the important privacy and ethical considerations involved in this work (Section 5.3).

5.1 Foundational Constraints of the Research

The design and execution of this research were shaped by a series of key constraints. The following subsections detail these limitations, outlining the architectural challenges that dictated the system’s design, the practical resource and experimental limitations that defined the project’s scope, and the specific methodological limitations of the data collection procedures.

5.1.1 Architectural Constraints

The client-server architecture of the developed system was a practical and necessary solution, shaped directly by the technical constraints of the modern browser ecosystem. The primary limitation arises from Google’s Manifest V3 framework (the restrictive Google framework for browser extensions, as detailed in Section 3.1.1), which prevents the direct client-side execution of complex machine learning models [17]. This constraint also rendered common optimization techniques, like Web Workers, non-viable for the project’s goals, as the Manifest V3 security model prevents them from having the direct DOM access required to capture image data from a webpage [40, 58]. Although browsers like Mozilla Firefox offer a more flexible environment by maintaining support for the older Manifest V2 standard [24], this research intentionally focused on the more restrictive Chromium platform due to its wide use [22, 62]. Furthermore, while advanced Multimodal Large Language Models (MLLMs) offer powerful content analysis capabilities (e.g., by combining audio and visual elements), their substantial computational requirements make them impractical for a lightweight, real-time browser application, further reinforcing the necessity of a backend processing approach and lightweight mechanisms (like the adaptive mechanisms used in Section 4.2) [44, 50].

Exploring alternative approaches like filtering visual content directly at the network level also proved infeasible. The universal use of HTTPS encrypts most web traffic [105]. Bypassing this encryption requires methods like “man-in-the-middle” decryption, which introduces significant security and privacy risks for the user [106]. This strategy was therefore rejected as an insecure and impractical solution for this project [108]. Furthermore, while some commercial VPN services offer content filtering, these tools typically operate by blocking entire domains or URLs [144]. They do not provide the granular, real-time analysis of visual content within an allowed, encrypted connection, which is the specific challenge this thesis addresses.

5.1.2 Resource and Experimental Scope Constraints

The decision to use a pre-trained model was also a consequence of practical constraints. Training a new neural network from scratch was deemed infeasible, primarily due to the immense challenge of acquiring a suitable dataset [49]. Ethically-sourced, large-scale, and precisely labeled datasets for NSFW object detection are not readily public, a problem highlighted by findings of illegal material in major public datasets like LAION [15]. Furthermore, this project faced significant resource limitations. The prohibitive financial cost of using cloud services for model training was beyond the scope of this academic research [32], and the local test machine lacked a dedicated NVIDIA GPU, making it impossible to leverage CUDA for high-speed processing [145]. This constraint not only guided the selection of a pre-trained model but also necessitated the choice of its lightest variant (the YOLOv11n, on which the “EraX-NSFW-V1.0” model is based). As the analysis in Section 4.4 demonstrates, small-scale tests with larger model variants caused system instability on the available hardware, confirming that only the nano variant could achieve the processing speeds necessary to operate under the 250 ms human perception threshold defined for this project’s objectives (Section 2.4) [31].

Finally, the scale of the experiments was constrained by factors related to the software. The dataset created for the comparative benchmark was limited by the 5GB cloud storage on the Canva platform’s free tier [146], which was used to host the test environment. The datasets for all experiments were also limited by the data transfer caps of the free-tier tunneling services and the significant time required for the manual image review process [136, 137]. To mitigate the inherent subjectivity in this manual review and ensure the results were consistent, the evaluation strictly adhered to the specific classification criteria that were formally defined in Section 3.3.1.

These constraints necessitated the use of a modest dataset of 360 curated images for the controlled comparative benchmark. While this scale was a deliberate methodological choice to create a balanced environment for comparing the tools, the results should be interpreted as a valuable, indicative snapshot of relative performance, rather than a definitive statement on performance across all possible scenarios.

5.1.3 Methodological Limitations for Data Collection

The use of a manual “live-browsing” methodology to evaluate the baseline (Section 4.1) and adaptive (Section 4.2) systems was a deliberate choice to test the system’s robustness in a dynamic, unpredictable web environment, where static datasets cannot fully simulate. However, such an approach has inherent methodological considerations.

To gather a varied and realistic dataset, browsing sessions were conducted across a diverse range of websites. The resulting data, while rich, should be noted that it represents a snapshot of these real-world scenarios rather than a statistically random sample of the entire internet [147]. Furthermore, in any study where the researcher is also the participant, there is a potential for observer effects. An awareness of the system’s hypotheses could have led to an unintentional focus on certain types of content [148]. Recognizing these factors was integral to the research design. The results from the live-browsing tests are therefore presented as strong pragmatic indicators of the system’s real-world potential. The controlled benchmark in Section 4.3 was designed specifically to complement this approach by providing a statistically rigorous evaluation under reproducible conditions.

5.2 Operational and Security Limitations of the Implemented System

The developed system, while effective, operates within a set of specific boundaries and trade-offs. The following subsections provide a comprehensive analysis of these limitations, examining the system’s operational scope, its architectural performance trade-offs, its inability to analyze certain types of content, the inherent vulnerabilities of the AI model, the security considerations of the client-server architecture, the contextual challenges of the adaptive logic, and the defined platform scope.

5.2.1 Specialization and Scope of Content Detection

The developed system is a specialized tool with a focused operational scope. The visual analysis is designed exclusively to detect nudity and sexually suggestive content and is not trained to identify other forms of harmful material, such as depictions of violence or gore. This specialization means that while the system is tailored for its primary purpose, it does not represent a comprehensive solution for all types of inappropriate online content.

5.2.2 Architectural Performance and Latency Trade-Offs

The system's performance is defined by its architectural trade-offs, particularly regarding processing latency. While the system's typical "Time to First Intervention" was measured between 206–236 ms, a speed sufficient to intervene within a typical human reaction time [31], this inherent delay results in a brief but perceptible "flash" of unfiltered content before the blur is applied. This issue, sometimes referred to as a "Flash of Original Content", is a known challenge in reactive content filtering where DOM elements render before an extension can programmatically intervene [149]. This represents a critical performance limitation of the system's reactive, post-rendering analysis approach and is a primary target for future work.

This initial intervention speed, however, becomes a significant bottleneck on pages with multiple images. This issue stems from JavaScript's single-threaded execution model [38]. The model forces the extension to process each visual element sequentially, one after another. This sequential process is resource-intensive. For every image, the extension must perform client-side tasks like DOM traversal, canvas rendering, and Base64 encoding before making a full round-trip API call to the backend [123]. The delay from this entire process, when repeated for numerous images and compounded by network latency, can result in a noticeable lag on media-rich pages [143]. This sequential processing therefore creates a clear scalability bottleneck (a point where the system's performance degrades under increasing load), particularly impacting the user experience on the very media-rich environments, like social media feeds, it is designed to protect.

Finally, the measured steady-state latency must be distinguished from the system's performance upon its initial launch. A user in a real-world scenario would not perform a formal warm-up (a typical user would expect immediate functionality upon launch) and would therefore encounter a significant, one-time "cold start" delay [130]. This longer response time on the first analysis request is due to the overhead of loading the AI model into memory for the first time. The measured steady-state latency is therefore framed not as an implementation flaw, but as the fundamental architectural cost required to achieve both Manifest V3 compliance and a privacy-preserving, local-first design. This local-first design, while central to the project's privacy goals, introduces a significant usability trade-off. The requirement for a user to install and maintain a local Python server creates a prohibitive technical barrier for the intended non-technical audience (e.g., parents), positioning the prototype as a proof-of-concept for validating the filtering logic, rather than a deployable, user-friendly tool.

5.2.3 Architectural Inability to Analyze Ad Content

The system's inability to analyze advertisements is a notable limitation. This represents a critical gap in protection, as advertisements are a substantial source of inappropriate material for minors [120]. Modern digital platforms increasingly use server-side techniques to embed advertising directly into content streams to bypass ad blockers [121, 122, 150]. This creates a technical challenge, as the extension works by reading the structure of the main webpage (DOM). Modern advertisements are often delivered in isolated containers, and for security reasons, the browser prevents the extension from reaching inside these ad containers to analyze their visual content [123].

5.2.4 Inherent Limitations and Vulnerabilities of the AI Model

The filtering approach is also subject to the inherent limitations of the AI model. Object detectors (like the YOLO pretrained model used in this thesis) can struggle to identify sensitive regions that are small, partially occluded, or in complex visual environments, which can lead to False Negatives [66].

In addition to inherent perceptual limitations, a complete analysis must consider the system's resilience against deliberate evasion. On a more sophisticated level, the AI model itself is subject to a type of threat known as adversarial attacks [151]. These attacks involve making small, often human-imperceptible modifications to an image's pixel data to cause a misclassification. For example, an attacker could design a subtle visual pattern that, when overlaid on a pornographic image, might be ignored by a human but would cause the YOLO-based model to classify the image as SFW, effectively bypassing the filter [151]. Acknowledging this adversarial threat is important because academic research has shown that these attacks are a substantial, practical concern for machine learning security [151]. Defending against such targeted attacks is a critical and active area of research that was beyond the scope of this project. The system's limitations in this area highlight an important direction for future improvement.

5.2.5 Security Vulnerabilities of the Client-Server Architecture

The adoption of a client-server architecture, while a necessary solution to address the constraints of Manifest V3, introduces a different set of security considerations. The focus of risk shifts from client-side browser constraints to the security of the local network and server components. A comprehensive analysis must consider these potential vulnerabilities, using well-established security guidelines as a reference. A key resource for this is the Open Worldwide Application Security Project (OWASP), a non-profit foundation dedicated to improving software security [152]. Specifically, their Browser Extension Vulnerabilities Cheat Sheet provides a valuable framework for identifying common risks [26].

Endpoint Discoverability and Access: A primary security consideration is the nature of the public endpoint created by the tunneling service. While the URL is active and publicly accessible on the internet, the system's primary defense against discovery is that the address is ephemeral: it is both temporary for a single session and randomized, preventing persistent external access [101, 102, 153]. However, any public endpoint represents a potential attack surface. For instance, an automated scanning tool (software designed to systematically probe systems for security weaknesses [154]) could discover the temporary URL and use the brief window of opportunity to launch a denial-of-service attack (an attack that attempts to make a service unavailable by overwhelming it with a flood of traffic [155]) or probe the API for common web application vulnerabilities (such as Injection flaws or Broken Access Control, as defined by established security standards, such as the OWASP Top 10 [156]). While the ephemeral nature of the URL acts as a significant mitigating control against casual discovery, it does not eliminate the risk entirely. An attacker with sufficient resources could still actively scan for and discover the temporary endpoint during a user's session. Therefore, a complete threat model must consider not only vulnerabilities on an already-compromised host machine, but also those that could be exploited by a remote attacker who successfully identifies the active endpoint.

Insecure Communication Channel: The system relies on a tunneling service to create a public HTTPS endpoint, which secures the communication channel from the browser to the edge of the tunnel (the tunneling service's public servers) [157]. However, a potential vulnerability exists in the next step of this chain: the connection from the tunnel client software to the local Flask API (e.g., on "localhost:5000") may be an unencrypted HTTP connection [158]. This specific vulnerability becomes more significant in a scenario where the host machine is already compromised. In such a case, a user faces severe primary threats that far exceed the scope of this tool, such as the encryption of personal files by ransomware (malware that blocks access to a victim's data until a ransom is paid [159]) or the theft of sensitive data through keystroke loggers (malware that secretly records every key pressed, allowing an attacker to steal passwords, credit card numbers, and other private information [160]). Within this high-threat context, the unencrypted local channel creates an opportunity for malicious software to perform a secondary, but still important, Man-in-the-Middle (MitM) attack (an attack where an attacker secretly intercepts and potentially alters communications between two parties) [161]. Such an attack could allow for the interception or manipulation of the image data and classification results being sent between the two local components (e.g., allowing a malicious process on the same machine to intercept an "NSFW" classification from the server and replace it with an "SFW" response before it reaches the extension, thereby disabling the blur).

Potential for Data Exfiltration: In the same high-threat context of a compromised host discussed in the previous section, a malicious browser extension creates another significant risk. While a user would already be facing the severe primary threats mentioned above (e.g., ransomware and keystroke loggers), a compromised browser extension, for instance, through a supply-chain attack (an attack that targets a trusted third-party vendor or software to gain access to a target's systems [162]), could theoretically be modified to exfiltrate the Base64-encoded image data to a malicious third-party server (e.g., a compromised dependency could allow an attacker to modify the extension to secretly transmit every analyzed image to an external server, creating

a severe privacy breach) [163]. Furthermore, for this research, the local server saved images to disk for evaluation. A production-ready version of this system would need to process all data strictly in-memory. Adopting an in-memory approach would prevent leaving a persistent trail of sensitive browsing history on the user's hard drive (a trail that could be discovered later by other malware or through forensic analysis) [164].

Local Server Vulnerabilities The Python Flask API, though running locally, constitutes a web server and is therefore an attack surface. The server must be configured for production, for example, with debugging modes turned off, to prevent information leakage, as detailed in Flask's security documentation [165]. A malicious webpage could also attempt to send crafted requests directly to the "localhost:5000" endpoint. Such requests could probe for common web application vulnerabilities, potentially causing the server to crash or behave in unexpected ways (e.g., a malicious script could send a specially crafted, broken request to the local server, designed to confuse and crash the Python program, which would turn off the filter until the user restarts it).

The vulnerabilities detailed above present significant barriers to creating a production-ready tool. Therefore, a dedicated effort to mitigate these security risks would be a critical prerequisite for any real-world deployment. A robust security posture would necessitate several critical hardening steps. To counter the "Insecure Communication Channel" risk, implementing local TLS (Transport Layer Security) is the direct solution [166]. Securing the connection between the tunneling client and the local Flask API with encryption prevents any local Man-in-the-Middle attack from intercepting a user's visual browsing data. To mitigate the "Potential for Data Exfiltration", the backend must be refactored to enforce in-memory processing. Handling all visual data in active memory (RAM) without writing to the hard drive prevents a persistent trail of sensitive browsing history from being left on the user's disk [167]. Finally, addressing "Local Server Vulnerabilities" would require formal penetration testing. Such a process, where security professionals perform a simulated cyberattack, is necessary to proactively find and fix exploits before a malicious actor can [167].

5.2.6 Contextual Challenges of the Adaptive Logic

Furthermore, the system's adaptive filtering logic (the mechanism from Section 4.2 that escalates a domain to "high-risk") introduces a unique contextual challenge. A legitimate website, such as a medical or educational platform, may contain an anatomically explicit but non-pornographic image or alt text (containing a word for genitals) for informational purposes. The system's model will likely return a high-confidence detection for such an image. This, in turn, triggers the adaptive mechanism to flag the entire domain as "high-risk" for the remainder of the browsing session, leading to overly aggressive filtering and the unnecessary blurring of safe content on that site.

5.2.7 Platform and Environment Scope

Finally, the current implementation has a defined platform scope. The system was designed and validated for desktop browsers only and was not tested or developed for mobile environments. It is worth noting that development and testing are focused primarily on Chromium-based browsers (like Google Chrome, Microsoft Edge) due to the project’s specific goal of addressing the constraints of the Manifest V3 environment [17]. Making the extension fully compatible with other browsers, such as Mozilla Firefox, would likely require code modifications to account for differences in their respective extension APIs [168].

5.2.8 Vulnerability to Heuristic Evasion

A key limitation of the adaptive system is its vulnerability to deliberate heuristic evasion. As the evaluation in Section 4.2 demonstrated, the system’s high performance is dependent on the presence of contextual triggers. This creates a predictable failure mode: a malicious actor could construct a “heuristically sterile” webpage. For example, a site could host explicit images with neutral file names and no surrounding text, while being served from a domain with no risky keywords (e.g., a free image-hosting service).

In this scenario, the adaptive logic would not be activated. The system’s performance would consequently revert to its much weaker, non-adaptive baseline, where the False Negative Rate is significantly higher, as shown by the performance gap between the baseline and adaptive tests (Section 4.1 vs. 4.2). This dependency on simple, static rules is a well-documented challenge; such systems can be effective but often struggle to keep pace with dynamic and deliberately evasive online threats [169]. This represents a fundamental challenge and is a critical area for future work on more robust, semantic context analysis.

5.2.9 Synthesizing the Trade-Offs: The Inherent Tension Between Compliance, Security, and Performance

The preceding sections detailed distinct limitations in system performance and security. A deeper analysis reveals a direct link between these issues, originating from the compromises required for Manifest V3 compliance. The tunneling service is the clearest example of this tension. The latency analysis in Section 4.4 quantified that the service contributes substantially to the ~200 ms of performance overhead. At the same time, the security analysis in Section 5.2.5 identified the public endpoint from the same service as the system’s most significant external attack surface. Therefore, the component that enables a private and compliant solution is also the source of its main performance bottleneck and primary security vulnerability.

5.3 Privacy and Ethical Considerations

The system’s client-server architecture, in its current implementation, is designed with a strong emphasis on user privacy. All visual data is processed locally on the user’s computer. Specifically, the browser extension communicates exclusively with the backend server running on the same computer, ensuring that sensitive browsing data does not leave the user’s immediate control. The communication between the two components (extension-backend) is enabled by a secure tunneling service [101, 102, 153]. Although this creates a publicly accessible URL for the local server, this address is ephemeral and changes each time the service is initiated, preventing persistent external access. It is important to distinguish this privacy-centric design from the data handling procedures required by the research methodology for this project. Saving images to disk was not a feature of the proposed tool, but a necessary procedural step undertaken for the explicit purpose of evaluation and validation. A released version of the system should not perform this function and should process all visual content entirely in-memory to maintain user trust and data security.

Beyond the system’s architecture, relying on third-party, pre-trained AI models introduces further ethical complexities regarding transparency and bias. Models like “EraX-NSFW-V1.0” are often considered “black boxes” as their internal operations and the full extent of their training data are not always transparent. This lack of transparency creates a substantial risk of inherent bias. Large-scale datasets used to train such models can contain problematic or even illegal content, which can embed harmful biases into the model, leading to unfair or inaccurate filtering across different contexts [15]. A key ethical question is how the model would perform on non-Western content. Since the training data for specialized models is often not public, the model may be biased towards a Western-centric definition of nudity. An error could lead to incorrectly flagging non-explicit images of traditional tribal attire as NSFW, a significant risk for a tool intended for global use. Proof of this is that academic studies have documented that content moderation models can exhibit demographic biases, sometimes performing less accurately on images of people with darker skin tones, which can lead to either lower protection for certain demographic groups or the over-censorship of specific contexts [170]. This creates the tangible and severe risk that the tool could provide a disparate and lower standard of safety for children from certain ethnic backgrounds. For example, a model that performs less accurately on images of people with darker skin tones could fail to detect harmful content as effectively as it does for those with lighter skin tones. Such an outcome represents a critical ethical failure.

Researchers face a methodological dilemma, as general-purpose public datasets often lack the specific examples needed for high-performance NSFW detection, necessitating the use of specialized models trained on private data [49, 53]. These specialized models, however, risk inheriting unknown biases, a documented problem in content moderation systems [170]. Proactively measuring bias is exceptionally difficult because public benchmark datasets curated specifically for NSFW fairness analysis are not currently available [49, 171, 172]. The absence of such benchmarks creates a fundamental trade-off in the practice of applied AI ethics

between the risks of creation (curating explicit data) and the risks of adoption (using an opaque, third-party model). The creation of robust fairness benchmarks is therefore a crucial area for future research.

The model’s “black box” nature also creates a challenge for user trust. An incorrect classification offers no explanation to the user, a frustrating experience that can erode confidence in the tool. A significant field of research known as Explainable AI (XAI) aims to address such a problem by making AI decisions understandable to humans [173]. While implementing a full XAI system was beyond the scope of this project, acknowledging its relevance is important. Future systems could incorporate XAI techniques to improve transparency and provide users with insight into why specific content was flagged (for instance, the system could show the specific bounding box the model identified, highlighting the region that led to the NSFW classification).

The ethical challenges extend to the very creation of the datasets used for training. As noted, public datasets have been found to contain illegal material [15], but the process of creating a private dataset also involves significant ethical considerations. Using web-scraped images often occurs without the explicit consent of the individuals depicted, raising substantial privacy concerns. Furthermore, there is a potential for significant psychological harm to the human researchers and annotators who must manually review and label thousands of explicit and potentially disturbing images to build an effective training set [174].

Moreover, the definition of “NSFW” content is not a universal standard but is highly subjective and dependent on individual, cultural, and regional norms. The definition of what constitutes offensive or inappropriate content is highly subjective and varies greatly based on individual perception, age, and cultural context [43, 117]. A filter that is appropriate in one context may be overly restrictive or dangerously permissive in another. Consequently, due to this ambiguity and the inherent limitations of any AI model, a responsible and ethical implementation must empower the end-user. Features like adjustable sensitivity settings (e.g., allowing the user to adjust the NSFW detection threshold in the backend API) are not merely for convenience but are a necessity to allow users to adjust the tool’s behavior according to their values and needs.

6 Conclusion and Future Work

This concluding chapter synthesizes the research presented in this thesis. It begins by summarizing the key findings and contributions of the work (Section 6.1). Subsequently, it transitions to explore several promising avenues for future work and potential system improvements (Section 6.2). The thesis concludes with a final summary of the project’s primary contributions and their broader implications for online child safety (Section 6.3).

6.1 Summary of Findings and Contributions

This thesis details the design and validation of a functional system for real-time visual filtering. The work presents a client-server paradigm as a necessary solution to the technical constraints of Manifest V3 and justifies a local-server implementation as a pragmatic choice for prioritizing user privacy and avoiding prohibitive operational costs.

The empirical evaluation in Chapter 4 established that a static detection threshold forces a fundamental trade-off between user safety and user experience (Section 4.1). The lightweight, context-aware adaptive technique was implemented to address this trade-off, and it demonstrated a significantly improved detection rate on a real-world dataset (Section 4.2). To further contextualize these results, a controlled comparative benchmark was conducted. Within the specific context of this modest, 360-image experiment, the developed adaptive system demonstrated a more effective balance of performance metrics than the tested alternatives (Section 4.3).

Finally, the latency analysis quantified the system’s responsiveness, confirming it can intervene within a typical human reaction time (Section 4.4). A crucial finding from this analysis was that the primary performance bottleneck is not the AI model itself, but the significant architectural overhead required by the client-side data pipeline. The research also provided a critical analysis of the architectural trade-offs, concluding that the local-server model represents a key compromise, prioritizing the privacy of on-device processing while introducing a set of network security risks that must be carefully managed (as detailed in Section 5.2.5).

Architectural Choice/Strategy	Primary Benefit	Inherent Cost / Risk	Mitigating Strategy Explored in Thesis
Client-server architecture (vs. client-only)	Manifest V3 compliance, potential for higher Accuracy	Latency overhead, increased complexity	Validated as a necessary architectural cost.
Local server (vs. cloud-hosted)	User privacy, no operational cost	Significant usability barrier, local security risks	Identified as a key privacy-vs-usability compromise.
Context-aware adaptive logic (vs. static threshold)	Achieves high Recall and high Precision simultaneously	Vulnerability to deliberate heuristic evasion	Empirically validated as a highly effective enhancement.
Use of pre-trained model (vs. custom-trained)	Avoids prohibitive cost and ethical burden of curation	Inherits “black box” nature and potential for bias	Identified as a core ethical trade-off, future work proposed.

Table 6: Summary of core architectural and performance trade-offs

6.2 A Strategic Roadmap for Future Research and Development

Building on the findings of this research, several promising avenues exist for future development. The following subsections detail a roadmap for advancing this work, moving from immediate system enhancements to broader, long-term research challenges.

6.2.1 Enhancing Performance and User Experience

A more practical future enhancement to address the “flash” of unfiltered content would be to implement immediate client-side placeholders [149]. This would involve modifying the “content.js” script to take immediate action the moment the MutationObserver detects a new image (e.g., the extension could instantly apply a generic blur to any new image and only remove it once the backend confirms the image is SFW, ensuring no unfiltered content is ever visible) before sending the data to the backend for analysis. This would achieve the goal of preventing any glimpse of potentially harmful content in a much more user-friendly way. While this was not the focus of the current work, which prioritized validating the detection system’s accuracy, this client-side optimization represents just one of several ways performance could be enhanced.

While exploring ways to guarantee that no unfiltered content is visible to the user, an alternative configuration was considered involving a “pre-scan” that blocks the webpage from rendering until all images have been analyzed. However, this approach was deemed infeasible for two primary reasons. First, a browser extension does not have the technical capability to fundamentally halt the browser’s native page rendering process, as it can only react to content as it appears on the page. Second, forcing a user to wait for a full-page scan on a media-heavy website would result in a poor user experience and contradicts the MutationObserver-based design [117, 129], which is built to handle dynamically loading pages.

A notable area for improving the system’s usability would be the implementation of a user-controlled sensitivity slider. The analysis in Chapter 4 demonstrated a clear trade-off between higher safety at lower thresholds and a more seamless user experience at higher thresholds. A user-adjustable setting would empower individuals to customize the filtering strictness to their personal preference. The aforementioned feature was not implemented in the current research because the primary goal was to first establish and validate the core effectiveness of the filtering system itself. Implementing and testing a user interface for real-time communication with the backend was considered a separate engineering challenge that was outside the primary research scope of this thesis.

Further performance optimizations could be achieved on the client side to address the latency overhead from processing multiple images on a single page. A powerful strategy would be to implement client-side batch processing, an approach that is feasible even within the significant constraints of Manifest V3. Client-side batch processing is possible by separating the tasks: the “content.js” script (which has DOM access) would capture the image data, and then a Web Worker would perform the computationally intensive Base64 (the process of converting an image to a text string for transmission) encoding on a background thread [42]. Because the worker itself does not access the DOM, this method complies with Manifest V3’s security model while preventing the user interface from freezing [123]. However, the decision was made not to implement this advanced technique for two key reasons. First, the primary research objective was to validate the core client-server architecture and adaptive logic. Second, the robust implementation of a multi-threaded processing pipeline is a substantial software engineering task. It would require building and debugging a complex system to ensure stability and prevent errors, which was beyond the defined scope of this thesis that was focused on validating the core client-server architecture and adaptive logic.

6.2.2 Expanding Protection Scope

During preliminary, small-scale testing, a more comprehensive, multi-layered defense system proved highly effective at enhancing user safety during the initial development and informal testing stages. This system integrated several additional protective mechanisms that were ultimately excluded from the final, formal evaluation to isolate and assess the performance of the visual analysis component. One such layer involved integrating ad-blocking filter lists, such as the widely-used EasyList [124], to proactively block known ad-serving domains and other harmful websites, directly addressing the limitation that the visual filter cannot analyze ad content (for example, preventing access to an adult content website at the network level before any of its visual media can be rendered) [175]. Another effective mechanism was an automated “Safe Search” enforcement feature, which guarantees that major platforms like Google and YouTube default to their strictest content filtering modes (for instance, automatically modifying the URL to force Google to filter out explicit search results) [176]. Finally, a user-configurable blocklist was implemented, which allows users to manually block any specific domains that bypass the automated systems (allowing a guardian to add example-harmful-site.com to the list, immediately preventing future access) [121, 122].

While these features were confirmed to be effective and would be essential components of a production-ready tool, they were not included in the benchmarked system to maintain a clear focus on the primary research goal: evaluating the performance of the core visual analysis engine and its adaptive logic.

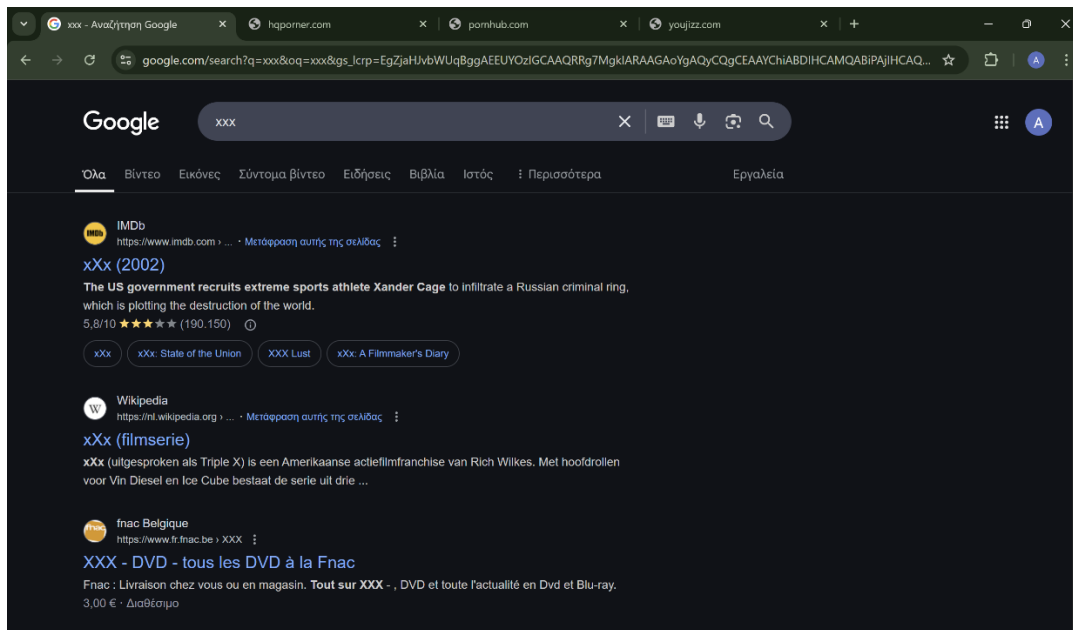


Figure 7: The implemented “Safe Search” enforcement feature

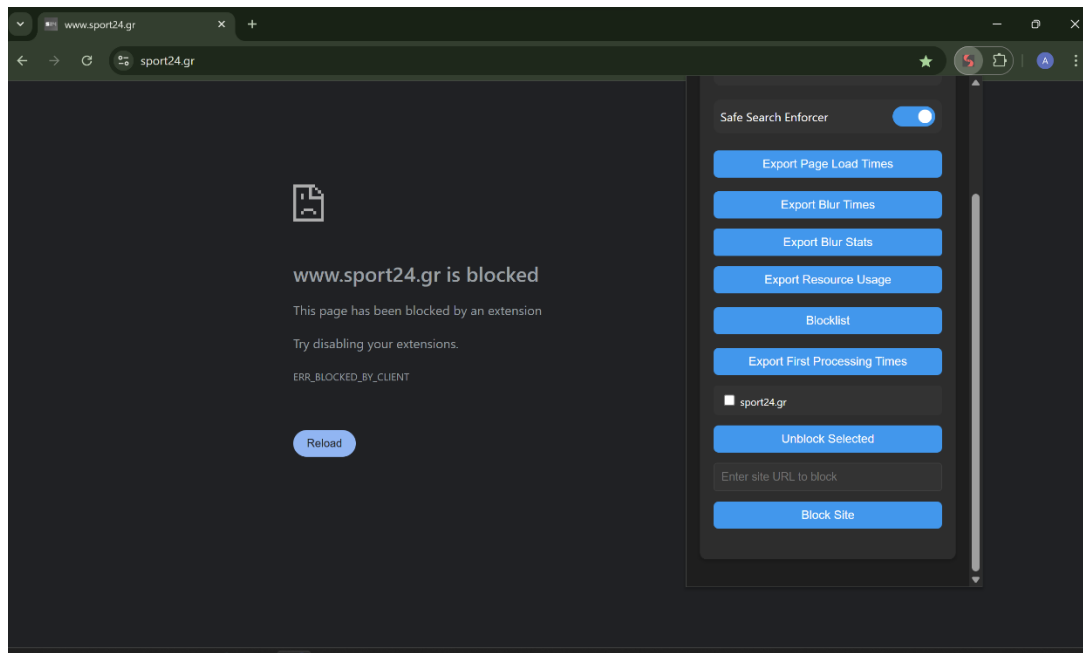


Figure 8: The user-configurable domain blocklist for manual control

6.2.3 Addressing Security and Ethical Challenges

The critical analysis in Chapter 5 highlights several avenues for future research, beginning with the challenge of scientific reproducibility. Future evaluations would ideally be grounded in large-scale, ethically sourced benchmark datasets. Creating such a resource for sensitive NSFW content, however, remains a massive task. Navigating the complex issues of data sourcing, user consent, and the psychological well-being of human annotators would demand a multi-disciplinary effort [174]. A successful benchmark would represent a significant contribution, enabling more rigorous and standardized comparisons of filtering technologies.

A critical next step for creating a production-ready tool involves a comprehensive security audit and hardening of the client-server architecture. Such an effort would build directly on the vulnerabilities identified in Section 5.2.5 by implementing several specific measures to fortify the system’s defenses. First, enforcing local TLS (Transport Layer Security, a cryptographic protocol designed to provide communications security over a computer network) would be essential to secure the communication channel between the tunnel client and the Flask server on the user’s machine [166]. Another vital task is to conduct formal penetration testing (a simulated cyberattack performed by ethical security professionals to find vulnerabilities) on the API endpoints. Regular penetration testing is a standard practice for identifying and patching potential exploits in web applications and APIs before they can be leveraged by attackers [167]. Finally, future work should develop a more robust permissioning model for the extension to ensure it adheres strictly to the Principle of Least Privilege (the security concept of giving a program only the absolute minimum permissions it needs to function), a foundational best practice recommended by security authorities like OWASP specifically for browser extensions [26, 177].

Another crucial technical direction involves investigating the system’s robustness against adversarial attacks (small, often human-imperceptible modifications to an image’s pixel data designed to cause a misclassification) identified in Section 5.2. Future research should explore the implementation and evaluation of specific defense mechanisms, such as adversarial training (a technique that improves a model’s resilience by including adversarially modified images in its training data) or defensive distillation. This is essential for developing the next generation of robust safety tools [151].

Further research must also address the profound ethical dimensions of using pre-trained models. A formal bias audit constitutes an essential endeavor to quantify model performance across different demographic groups and address inherited biases (e.g., by testing the model’s performance on benchmark datasets specifically designed to measure fairness across different skin tones and cultural contexts) [170]. To tackle the “black box” problem, future versions could integrate techniques from Explainable AI (XAI) (e.g., by implementing a formal XAI technique such as Saliency Mapping, which generates a visual heatmap over a classified image to highlight the specific pixels or regions that most strongly influenced the model’s decision). A comprehensive XAI implementation constitutes a significant research project in its own right. Providing users with insight into classification decisions, however, would greatly enhance trust. This level of transparency is a core principle of responsible AI development [178].

6.2.4 Long-Term Technological and Architectural Evolution

Additional improvements could target the core detection engine itself. As the field of computer vision evolves rapidly, future work could involve incorporating newer pre-trained models, such as the latest iteration of the YOLO framework, YOLOv12, which has better accuracy and efficiency on public benchmarks compared to older architectures [179]. While this could potentially increase detection performance, retraining or fine-tuning a model specifically for this task remains a substantial challenge. As analyzed in Chapter 2, ethically-sourced, large-scale, and precisely labeled datasets for NSFW object detection are not publicly available, a problem exacerbated by findings of illegal material in major public datasets [15]. Additionally, exploring future hardware acceleration options, such as improved DirectML support (a Microsoft API for accelerating machine learning on a wide range of hardware, which was supported by the test machine) in newer YOLO versions (as this was not available for YOLOv11), could improve backend performance on non-NVIDIA hardware [180].

Finally, a major architectural evolution to address the scalability and the initial image blur latency issues would be the migration of the local backend API to a centrally managed cloud service. The primary benefit of this approach would be to offload all computational demands from the user’s computer. This would not only guarantee a fast and consistent response time irrespective of the user’s hardware capabilities, but it would also reduce the processing latency component by leveraging powerful cloud servers [181]. Such cloud infrastructure could also feasibly support the expansion of protection to mobile environments, possibly through the development of a dedicated safe browser that securely communicates with the cloud API for its visual filtering capabilities [181]. A cloud migration, however, could only become a viable

and ethical path forward if the significant challenges that informed the original architectural design were overcome. To clarify that rationale, the initial decision to use a client-server model was a technical necessity driven by browser constraints like Manifest V3. In contrast, the subsequent decision to host that server locally was a deliberate, project-level choice made to ensure user privacy (by preventing a visual record of a user’s browsing history from being sent to a third party) and to avoid the prohibitive operational costs of a cloud-based service (such as the pay-per-use API fees detailed in Section 3.1.1.4, which are financially impractical for a non-commercial tool). Therefore, a future shift to the cloud is not a technical fix, but an evolution that depends on solving these core economic and ethical challenges [32, 107].

6.3 Concluding Remarks

In conclusion, this thesis presented and evaluated a novel and practical hybrid system for real-time visual content filtering. The primary contributions of this work address the identified research gaps. First, it demonstrates an architecturally sustainable client-server paradigm as a necessary and viable solution to the powerful constraints of Manifest V3. Second, it provides an empirical validation of a resource-efficient, context-aware adaptive logic as a system-level method for significantly improving a pre-trained model’s accuracy without the need for costly retraining. Finally, the strategic analysis of architectural trade-offs, delivered through a comprehensive benchmark, contextualizes the system’s effectiveness. This research validates a pragmatic and sustainable approach to online child safety, paving the way for more robust and intelligent protective solutions.

APPENDICES

A The First Appendix: AI and Machine Learning Concepts

COCO [97]: A large-scale dataset designed for object detection, segmentation, and captioning, containing over 300,000 labeled images with more than 80 object categories. It introduced stricter evaluation standards compared to previous datasets.

Confusion Matrix and Classification Metrics [131]: A table summarizing the performance of a classification model by comparing predicted labels to true labels, detailing True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Derived metrics like Accuracy, Precision, Recall, F1-Score, FPR, and FNR quantify different aspects of classification effectiveness.

mAP (mean Average Precision) [97]: A metric used in object detection to evaluate how well models identify and localize objects. It averages the Precision scores across multiple Intersection over Union (IoU) thresholds (from 0.5 to 0.95), providing a balanced measure of detection accuracy.

ONNX (Open Neural Network Exchange) [92]: An open-source format developed to represent machine learning models, enabling interoperability between different frameworks and tools. It allows models trained in one environment to be deployed for inference in another (e.g., running a model on a CPU via an ONNX runtime).

OpenCV [110]: An open-source library for real-time computer vision applications. The backend API utilizes OpenCV for essential image processing tasks, such as handling the image data received from the browser extension before it is passed to the AI model for analysis.

Pillow [111]: A powerful image manipulation library for Python, forked from the original Python Imaging Library (PIL). In this system, Pillow is used on the backend to open, process, and manage the image data sent from the browser extension.

PyTorch [109]: An open-source machine learning library widely used for applications such as computer vision. In this thesis, it was used in the backend API, in conjunction with ONNX Runtime, to load and run the “EraX-NSFW-V1.0” (YOLOv11n based) object detection model for NSFW classification.

YOLOv8 [90]: A fast and accurate single-stage object detection model from the “You Only Look Once” family of architectures. It serves as the foundational architecture for the “NudeNet” model, one of the alternative filtering technologies used in the comparative benchmark.

YOLOv11 [89]: A recent iteration of the YOLO framework, noted for its improvements in efficiency, accuracy, and the ability to interpret complex scenes on public benchmarks. It is the foundational architecture for “EraX-NSFW-V1.0”, the primary model selected for the developed system in this thesis.

The Second Appendix : System, Web, and Browser Technologies

Base64 [42]: A method for encoding binary data, such as images and audio files, into an ASCII text format using a set of 64 standard characters. Its primary function is to enable the storage and transfer of media across systems designed to handle only text. A key trade-off of this process is that it substantially increases the original file size.

Content Security Policy (CSP) [29]: A computer security standard that helps prevent cross-site scripting (XSS) and other code injection attacks by allowing web administrators to control the resources a browser can load for a given page. It adds a layer of security that can affect how browser extensions interact with web content and external services.

CORS [30]: Cross-Origin Resource Sharing (CORS) is a security mechanism that uses HTTP headers to let a server specify which origins (domains, schemes, or ports), other than its own, are permitted to access its resources. By default, browsers block cross-origin requests for security reasons under the “same-origin policy”. CORS provides a controlled method for safely bypassing this restriction, sometimes using a “preflight” request (a preliminary check that uses the HTTP “OPTIONS” method) to ask for server permission before the actual request is sent.

DOM (Document Object Model) [39]: A programming interface standard for web documents, representing the page as a logical tree of objects (nodes). It allows programs and scripts (like those in a browser extension) to dynamically access and manipulate a document’s content, structure, and style. This interaction is crucial for the extension to identify, analyze, and modify visual media elements on web pages.

Filter lists / Blocklists [182]: Sets of rules or criteria used to filter out unwanted content like ads, trackers, malicious websites, or spam emails. They are commonly used in ad-blocking software, email filters, and security systems to protect users and improve their online experience.

Flask [25]: A lightweight Python web framework used to build web applications and APIs. For this project, Flask was employed to create the local backend API that hosts the “EraX-NSFW-V1.0” model, receives image data from the browser extension, and returns NSFW detection results.

HTML Canvas API (toDataURL()) [183]: The HTML <canvas> element, scripted typically with JavaScript, is used for drawing graphics on a web page. The “toDataURL()” method of the canvas returns a data URI containing a Base64 encoded representation of the image drawn on the canvas, a technique used in this project for capturing and encoding visual media.

JavaScript (JS) and its Execution Model [184]: A high-level programming language central to web development and browser extension logic. Its predominantly single-threaded nature for synchronous DOM operations can influence the handling of computationally intensive client-side tasks, such as the sequential processing of multiple images for Base64 encoding in this project.

Manifest V3 [17]: The latest specification for developing Chrome browser extensions, emphasizing enhanced security, privacy, and performance. Its restrictions heavily influenced the client-server architecture of the developed system, making direct client-side ML model execution infeasible.

MutationObserver [117, 129]: A Web API built into modern browsers that allows a script to watch for changes made to the Document Object Model (DOM). In this project, the extension uses a MutationObserver to detect and process images that are loaded dynamically after the initial page load, such as with infinite scroll.

Regular Expressions [34]: A sequence of characters that specifies a search pattern in text. The system's adaptive logic employs regular expressions in its "Domain Analysis" and "Textual Context Analysis" mechanisms to efficiently scan URLs and alt-text for predefined high-risk keywords.

Tunneling services [101, 102, 153]: They create secure connections from a public URL to a locally hosted server, essential in this project for enabling browser extension communication with the local Flask API during development and testing, mimicking a deployed environment

Web Worker [185]: A browser feature that allows a script to run in a background thread, separate from the main execution thread of a webpage. While useful for heavy computations, Manifest V3's security model prevents Web Workers from directly accessing the DOM, making them unsuitable for the core image-capturing task of this project.

Bibliography

- [1] M. Zeligs, “Internet for Kids: Weighing the Benefits and Risks”, December 2024. [Online].
- [2] C. C. f. England, “‘A lot of it is actually just abuse’ - Young people and pornography”, 2023.
- [3] M. Altin, D. De Leo, N. Tribbia, L. Ronconi and S. Cipolletta, “Problematic Pornography Use, Mental Health, and Suicidality among Young Adults”, September 2024.
- [4] L. C. J. B. Andamon, I. H. O. Bacote, K. G. Estampa and G. Genelza, “Regain Consciousness: The Impact of Internet Pornography on Children And Adolescents – A Review”, *International Journal of Human Research and Social Science Studies*, pp. 160-177, April 2025.
- [5] F. Eltaher, R. K. Gajula, L. Miralles-Pechuán, P. Crotty, J. Martínez-Otero, C. Thorpe and S. McKeever, “Protecting Young Users on Social Media: Evaluating the Effectiveness of Content Moderation and Legal Safeguards on Video Sharing Platforms”.
- [6] “Google Family Link Review 2025: It’s Free, but Is It Good?”, safetydetectives, 2022. [Online]. Available: <https://www.safetydetectives.com/best-parental-control/google-family-link/>.
- [7] B. A., “Quick and Easy Ways to Bypass Google Family Link Block”, [Online]. Available: <https://famisafe.wondershare.com/family-locator/bypass-family-link-to-download.html>.
- [8] E.-M. Maier, L. M. Tanczer and L. D. Klausner, “Surveillance Disguised as Protection: A Comparative Analysis of Sideloaded and In-Store Parental Control Apps”, 2025.
- [9] Z. S., “How to Put Parental Controls on Google in 2025 | Only What Works!”, [Online]. Available: <https://www.wizcase.com/blog/put-parental-controls-on-google/>.
- [10] “Content Filtering: A Digital Shield for Safe Browsing”, [Online]. Available: <https://cyberhygiene.community/blogdetails/content-filtering>.
- [11] “Fact Sheet: Child & Youth Problematic Online Pornography”, 2025.

- [12] “How Do Kids Bypass Google Family Link?”, April 2024. [Online]. Available: <https://www.mobicip.com/blog/how-do-kids-bypass-google-family-link>.
- [13] “The Evolving Landscape of Media Regulation: Trends and Challenges”, The Legallo Editorial, 2025. [Online]. Available: <https://thelegallo.com/evolving-landscape-of-media-regulation/>.
- [14] “Regulation of Online Content: Navigating Legal Landscapes”, edicts.blog, December 2024. [Online]. Available: <https://edicts.blog/regulation-of-online-content/>.
- [15] “AI images of child sexual abuse getting ‘significantly more realistic’, says watchdog”, www.theguardian.com, April 2025. [Online]. Available: <https://www.theguardian.com/technology/2025/apr/23/ai-images-of-child-sexual-abuse-getting-significantly-more-realistic-says-watchdog>.
- [16] “Meta Transparency Report”, Meta, 2025. [Online]. Available: <https://seosandwitch.com/ai-content-moderation-stats/>.
- [17] Chrome, “Manifest V3”, [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>.
- [18] “Remove remotely hosted code”, developer.chrome.com, [Online]. Available: https://developer.chrome.com/docs/extensions/develop/migrate/improve-security?utm_source=chatgpt.com.
- [19] “TF.JS uses eval(), effectively making it impossible to deploy it in Chrome extensions”, 2023. [Online]. Available: <https://discuss.ai.google.dev/t/tf-js-uses-eval-effectively-making-it-impossible-to-deploy-it-in-chrome-extensions/28833>.
- [20] P. Đ. Thục, Đ. T. Khang, M. T. H. Khương, M. T. C. Đức, M. P. N. T. Kha and P. H. Nhật, “EraX-NSFW-V1.0: A Highly Efficient Model for NSFW Detection”, EraX JS Company, 2024. [Online]. Available: <https://huggingface.co/erax-ai/EraX-NSFW-V1.0?not-for-all-audiences=true>.
- [21] “AdGuard vs uBlock Origin: An In-depth Comparison by AboutCookies”, AboutCookies, April 2025. [Online]. Available: <https://www.aboutcookies.org.uk/ad-blockers/adguard-vs-ublock-origin>.
- [22] B. Team, “Web Browser Market Share in 2025”, Backlinko, March 2025. [Online]. Available: <https://backlinko.com/browser-market-share>.
- [23] Chrome, “Manifest V2 support timeline (June 2025)”, [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/mv2-deprecation-timeline>.

- [24] M. o. MV2-3, “Manifest V3 & Manifest V2 (March 2024 update)”, [Online]. Available: <https://blog.mozilla.org/addons/2024/03/13/manifest-v3-manifest-v2-march-2024-update/>.
- [25] “Flask API”, [Online]. Available: <https://flask.palletsprojects.com/en/stable/>.
- [26] “Browser Extension Security Vulnerabilities Cheat Sheet”, [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Browser_Extension_Vulnerabilities_Cheat_Sheet.html.
- [27] “Declare permissions”, Chrome for developers, [Online]. Available: https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions?utm_source=chatgpt.com.
- [28] Chrome for developers, [Online]. Available: https://developer.chrome.com/docs/extensions/develop/migrate/blocking-web-requests?utm_source=chatgpt.com.
- [29] Mozilla, “Content Security Policy (CSP)”, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>.
- [30] “Cross-Origin Resource Sharing (CORS)”, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>.
- [31] PubNub, “How Fast is Real-Time? Human Perception and Technology”, medium.com, [Online]. Available: <https://medium.com/@PubNub/how-fast-is-real-time-human-perception-and-technology-fe7246381935>.
- [32] U. Yildirim, R. Dutta, B. Yildirim and A. Vaidya, “Experimentation in Content Moderation using RWKV”, September 2024.
- [33] “System Requirements of training YOLOv12”, [Online]. Available: <https://docs.ultralytics.com/models/yolo12/#key-improvements>.
- [34] H. Fakher, “Relevance of Regular Expressions in the Age of AI”, [Online]. Available: <https://medium.com/@thecuriouschronicles/relevance-of-regular-expressions-in-the-age-of-ai-dacde54c8242>.
- [35] NotAI-tech, “NudeNet's package release in Python”, [Online]. Available: <https://pypi.org/project/nudenet/>.
- [36] “NSFW filter Github”, [Online]. Available: <https://github.com/nsfw-filter/nsfw-filter/tree/v2.3.0>.

- [37] R. Ballester-Arnal, B. Gil-Julia, M. Elipe-Miravet, C. Giménez-García and M. D. Gil-Llario, “Experiences and Psychological Impact Derived from Unwanted Exposure to Online Pornography in Spanish Adolescents”, *Sexuality Research and Social Policy*, vol. 21, p. 1594–1606, 2023.
- [38] “Is JavaScript Single-Threaded? Understanding the Basics”, [Online]. Available: <https://startup-house.com/blog/is-javascript-single-threaded-understanding-the-basics>.
- [39] Mozilla, “Document Object Model DOM”, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.
- [40] “No Direct DOM Access: in Manifest V3”, medium.com, [Online]. Available: <https://rahulnegi20.medium.com/building-persistent-chrome-extension-using-manifest-v3-198000bfl6b6>.
- [41] “Optimizing canvas”, mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas.
- [42] “Base64”, [Online]. Available: <https://www.keycdn.com/support/image-base64-encoding#:~:text=Base64%20encoding%20is%20a%20way,a%20filename%20for%20various%20reasons>.
- [43] API4AI, “The Role of AI in Content Moderation: Detecting NSFW Content with Image Processing”, medium.com, 9 October 2024. [Online]. Available: <https://medium.com/@API4AI/the-role-of-ai-in-content-moderation-detecting-nsfw-content-with-image-processing-aaadd9cb867>.
- [44] N. AlDahoul, M. J. T. Tan, H. R. Kasireddy and Y. Zaki, “Advancing Content Moderation: Evaluating Large Language Models for Detecting Sensitive Content Across Text, Images, and Videos”, 27 November 2024.
- [45] V. C. Strasburger, A. B. Jordan and E. Donnerstein, “Health effects of media on children and adolescents”, *Pediatrics*, vol. 125, p. 756–767, 2010.
- [46] D. K. Citron, “Sexual privacy. Yale LJ”, *Yale Law Journal*, 2018.
- [47] “Understanding Underage Exposure to Porn and Its Effects”, [Online]. Available: <https://fightthenewdrug.org/understanding-underage-exposure-to-porn-and-its-effects/>.
- [48] D. Carnevali, L. Cavazzana, L. Grimoldi, P. Magnoni, N. Principi and A. Pellai, “The use of online pornography among minors: epidemiological analysis of the phenomenon”, February 2022.

- [49] R. Zhang, D. Zhu, C. Wu and J. Xu, “Sensitive Information Detection Based on Deep Learning Models”, *Applied Sciences*, vol. 14, no. 17, 2024.
- [50] M. Wu, Y. Zhao, J. Cao, M. Xu, J. Z. X. Wang and C. W. Fu, “ICM-Assistant: Instruction-tuning Multimodal Large Language Models for Rule-based Explainable Image Content Moderation”, in *AAAI Conference on Artificial Intelligence*, 2025.
- [51] S. L. Hor, H. A. Karim, M. H. L. Abdullah, N. AlDahoul, S. Mansor, M. F. A. Fauzi, J. See and A. S. B. Wazir, “An Evaluation of State-of-the-Art Object Detectors for Pornography Detection”, in *IEEE International Conference on Signal and Image Processing Applications (ICSIPA)*, 2021.
- [52] W.-J. Lee, “Exploring NSFWJS: Safe Browsing with”, medium.com, 5 November 2024. [Online]. Available: <https://medium.com/@weijenle/exploring-nsfwjs-safe-browsing-with-machine-learning-b3260cd76478>.
- [53] D.-D. Phan, T.-T. Nguyen, Q.-H. Nguyen, H.-L. Tran, K.-N.-K. Nguyen and D.-L. Vu, “A Novel Pornographic Visual Content Classifier based on Sensitive Object Detection”, *International Journal of Advanced Computer Science and Applications(IJACSA)*, vol. 12, no. 5, 2021.
- [54] “NSFW filter using NSFJS”, [Online]. Available: <https://nsfw-filter.com/>
<https://github.com/nsfw-filter/nsfw-filter>.
- [55] C. Szegedy, V. Vanhoucke, S. Ioffe and J. Shlens, “Rethinking the inception architecture for computer vision”, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- [56] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks”, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2019.
- [57] “NSFWJS library”, [Online]. Available: <https://github.com/infinitered/nsfwjs>.
- [58] “Javascript 101: Web workers”, medium.com, [Online]. Available: <https://medium.com/geekculture/javascript-101-web-workers-1c70bcb4ba90>.
- [59] “Same-origin policy”, Mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy.
- [60] “WebAssembly”, developer mozilla, [Online]. Available: <https://developer.mozilla.org/en-US/docs/WebAssembly>.
- [61] “What is a content delivery network (CDN)? | How do CDNs work?”, cloudflare, [Online]. Available: <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>.

- [62] statcounter, “Browser Market Share Worldwide”, [Online]. Available: <https://gs.statcounter.com/browser-market-share>.
- [63] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, October 2020.
- [64] M. Rastegari and S. Mehta, “MOBILEVIT: LIGHT-WEIGHT, GENERAL-PURPOSE, AND MOBILE-FRIENDLY VISION TRANSFORMER”, March 2022.
- [65] L. Papa, P. Russo, I. Amerini and L. Zhou, “A survey on efficient vision transformers: algorithms, techniques, and performance benchmarking”, March 2024.
- [66] M. L. Ali and Z. Zhang, “The YOLOFramework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection”, *Computers*, vol. 13, no. 12, December 2024.
- [67] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu and A. C. Berg, “SSD: Single Shot MultiBox Detector”, in *Computer Vision–ECCV 2016: 14th European Conference*, Amsterdam, 2016.
- [68] M. Tan, R. Pang and Q. V. Le, “EfficientDet: Scalable and Efficient Object Detection”, in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, 2020.
- [69] M. Carranza-García, J. Torres-Mateo, P. Lara-Benítez and J. García-Gutiérrez, “On the Performance of One-Stage and Two-Stage Object Detectors in Autonomous Vehicles Using Camera Data”, vol. 13, no. 1, 2020.
- [70] S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”, *Advances in neural information processing systems*, 2015.
- [71] A. Srivastava, A. Dalvi, C. Britto, H. Rai and K. Shelke, “Explicit Content Detection using Faster R-CNN and SSD MobileNet v2”, *International Research Journal of Engineering and Technology (IRJET)*, vol. 07, no. 03, p. March, 2020.
- [72] K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN”, in *IEEE International Conference on Computer Vision (ICCV)*, Venice, 2017.
- [73] Z. Cai and N. Vasconcelos, “Cascade R-CNN: Delving into High Quality Object Detection”, in *Proceedings of the IEEE conference on computer vision and pattern recognition.*, 2018.
- [74] “Support-vector networks”, *Mach Learn*, p. 273–297, 1995.

- [75] K. Song and Y.-S. Kim, “An Enhanced Multimodal Stacking Scheme for Online Pornographic Content Detection”, *Applied Sciences*, vol. 10, no. 8, April 2020.
- [76] I. Goodfellow, Y. Bengio and A. Courville, “Sequence Modeling: Recurrent and Recursive Nets”, in *Deep Learning. MIT Press.*, 2016.
- [77] “Bidirectional Recurrent Neural Network”, [geeksforgeeks.org](https://www.geeksforgeeks.org/bidirectional-recurrent-neural-network/), May 2025. [Online]. Available: <https://www.geeksforgeeks.org/bidirectional-recurrent-neural-network/>.
- [78] A. Barnwal, “Understanding Multimodal LLMs”, [medium.com](https://medium.com/%40avinashbarnwal123/understanding-multimodal-llms-be1c49a92617), November 2024. [Online]. Available: <https://medium.com/%40avinashbarnwal123/understanding-multimodal-llms-be1c49a92617>.
- [79] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention Is All You Need”.
- [80] J. Yuan, Y. Yu, G. Mittal, M. Hall, S. Sajeer and M. Chen, “Rethinking Multimodal Content Moderation from an Asymmetric Angle with Mixed-modality”, December 2023.
- [81] X. Zhang, J. Chong and K. Youcef-Toumi, “Strategy, How Does Perception Affect Safety: New Metrics and Strategy”, 12 December 2023.
- [82] “AI image generators trained on pictures of child sexual abuse, study finds”, 2024. [Online]. Available: <https://www.theguardian.com/technology/2023/dec/20/ai-image-generators-child-sexual-abuse>.
- [83] A. Dubettier, T. Gernot, E. Giguet and C. Rosenberger, “A Comparative Study of Tools for Explicit Content Detection in Images”, in *International Conference on Cyberworlds*, Sousse, Tunisia, 2023.
- [84] notAI-tech, “NudeNet: An ensemble of Neural Nets for Nudity Detection and Censoring”, [Online]. Available: <https://praneethbedapudi.medium.com/nudenet-an-ensemble-of-neural-nets-for-nudity-detection-and-censoring-c8fcef6cc92>.
- [85] P. Riccio, G. Curto, N. Oliver and T. Hofmann, “An Art-centric perspective on AI-based content moderation of nudity”., 2024.
- [86] M. S., T. R. and V. C., “A Review on YOLOv8 and Its Advancements”, in *Data Intelligence and Cognitive Informatics*, 2024, pp. 529-545.
- [87] R. Sapkotaa, Z. Meng, M. Churuvija, X. Du, Z. Mab and M. Karkee, “Comprehensive Performance Evaluation of YOLOv12, YOLO11, YOLOv10, YOLOv9 and YOLOv8 on Detecting and Counting Fruitlet in Complex Orchard Environments”, February 2025.

- [88] R. Khanam and M. Hussain, “Yolov11: An overview of the key architectural enhancements”, October 2024.
- [89] Ultralytics, “YOLOv11”, [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>.
- [90] “Ultralytics YOLOv8 documentation”, 2023. [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>.
- [91] “COCO dataset”, [Online]. Available: <https://cocodataset.org/#home>.
- [92] “ONNX website”, [Online]. Available: <https://onnx.ai/>.
- [93] “ONNX and Azure Machine Learning”, Microsoft, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-onnx?view=azureml-api-2>.
- [94] “ONNX vs PyTorch Speed: In-Depth Performance Comparison”, February 2024. [Online]. Available: <https://dev-kit.io/blog/machine-learning/onnx-vs-pytorch-speed-comparison>.
- [95] Ultralytics, “Performance table of YOLOv8 models”, [Online]. Available: <https://docs.ultralytics.com/models/yolov8/#performance-metrics>.
- [96] G. Jocher and J. Qiu, “Ultralytics YOLO11”, Ultralytics, 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/#supported-tasks-and-modes>.
- [97] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick and D. P., “Microsoft COCO: Common Objects in Context”, May 2014.
- [98] S. Han, H. Mao and W. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding”, February 2016.
- [99] “Additional Requirements for Manifest V3”, developer.chrome.com, [Online]. Available: <https://developer.chrome.com/docs/webstore/program-policies/mv3-requirements>.
- [100] “How to Expose Your Localhost Server to the Internet Using ngrok”., [Online]. Available: <https://medium.com/@ijayoa/how-to-expose-your-localhost-server-to-the-internet-using-ngrok-4e381080d8e9>.
- [101] “Cloudflare tunnel”, [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>.
- [102] “Ngrok website”, [Online]. Available: <https://ngrok.com/>.

- [103] “Improve extension security”, developer.chrome, [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/improve-security>.
- [104] N. Pantelaïos and A. Kapravelos, “Work-in-Progress: Manifest V3 Unveiled: Navigating the New Era of Browser Extensions”, April 2024.
- [105] C. Nachreiner, “Hiding undetected: Why security teams can no longer overlook HTTPS decryption”, [Online]. Available: <https://betanews.com/2023/09/07/why-security-teams-cant-overlook-https-decryption/>.
- [106] “Entrust distrust by major browsers”, developers.cloudflare.com, 2025. [Online]. Available: <https://developers.cloudflare.com/ssl/reference/migration-guides/entrust-distrust/>.
- [107] A. Luqman, R. Mahesh and A. Chattopadhyay, “Privacy and Security Implications of Cloud-Based AI Services : A Survey”, January 2024.
- [108] G. S. Poh, D. M. Divakaran, H. W. Lim, J. Ning and A. Desai, “A Survey of Privacy-Preserving Techniques for Encrypted Traffic Inspection over Network Middleboxes”, 2021.
- [109] “Pytorch”, [Online]. Available: <https://pytorch.org/>.
- [110] “Python Pillow”, [Online]. Available: <https://pypi.org/project/pillow/>.
- [111] “OpenCV”, [Online]. Available: <https://pypi.org/project/opencv-python/>.
- [112] “canva”, [Online]. Available: <https://www.canva.com/>.
- [113] “NSFW definition”, urbandictionary.com, [Online]. Available: <https://www.urbandictionary.com/define.php?term=NSFW>.
- [114] “Adult Nudity and Sexual Activity”, Meta, [Online]. Available: <https://transparency.meta.com/sv-se/policies/community-standards/adult-nudity-sexual-activity/>.
- [115] F. C. Akyon and A. Temizel, “State-of-the-Art in Nudity Classification: A Comparative Analysis”, 2023.
- [116] “Alt text”, harvard.edu, [Online]. Available: <https://accessibility.huit.harvard.edu/describe-content-images>.
- [117] M. Hilscher, H. Tjabben, H. Rätz, L. Besançon, A. Semmo, J. Döllner and M. Trapp, “Service-based Analysis and Abstraction for Content Moderation of Digital Images”, May 2021.

- [118] “PuritySight: Explicit, NSFW and porn content blocker”, [Online]. Available: <https://creati.ai/ai-tools/puritysight/> <https://stoptouchingyourself.click/> <https://chromewebstore.google.com/detail/puritysight-explicit-nsfw/phghbdcppdhcokoiijnjnomcpedjpmdd?hl=en>.
- [119] P. Bedapudi, “NudeNet: An ensemble of Neural Nets for Nudity Detection and Censoring”, medium, 2019. [Online].
- [120] J. Liu, R. Tandon, U. Durairaj, J. Guo, S. Zahabizadeh, S. Ilango, J. Tang, N. Gupta, Z. Zhou and J. Mirkovic, “Did your child get disturbed by an inappropriate advertisement on YouTube?”, November 2022.
- [121] A. Adegbola, “YouTube embeds ads into videos to beat ad blockers”, 14 June 2024. [Online]. Available: <https://searchengineland.com/youtube-embeds-ads-videos-ad-blockers-443187>.
- [122] “Making a Chrome Extension to Skip Video Ads on YouTube”, February 2024. [Online]. Available: <https://medium.com/@uriser/lets-demystify-chrome-extensions-my-first-extension-to-skip-video-ads-239dbf206942>.
- [123] Microsoft, “Minimize an extension's impact on page load time”, 19 09 2024. [Online]. Available: <https://learn.microsoft.com/en-us/microsoft-edge/extensions-chromium/developer-guide/minimize-page-load-time-impact>.
- [124] “EasyList”, [Online]. Available: <https://easylist.to/#easylist>.
- [125] “Difference between Scripted testing and Exploratory testing”, geeksforgeeks, July 2025. [Online]. Available: <https://www.geeksforgeeks.org/software-testing/difference-between-scripted-testing-and-exploratory-testing/>.
- [126] “What is Scripted Testing | How it Differs From Exploratory Testing?”, testsigma, April 2024. [Online]. Available: <https://testsigma.com/blog/scripted-testing/>.
- [127] “Exploratory vs Scripted Testing: Mastering the Art and Science of QA”, qabrain, July 2025. [Online]. Available: <https://qabrain.com/exploratory-vs-scripted-testing-mastering-the-art-and-science-of-qa>.
- [128] “EXPLORATORY TESTING: A Complete Guide to Exploratory Testing”, November 2024. [Online]. Available: <https://www.getxray.app/blog/complete-guide-to-exploratory-testing>.
- [129] “MutationObserver”, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/MutationObserver>.
- [130] “Model warmup”, medium.com, [Online]. Available: <https://medium.com/better-ml/model-warmup-8e9681ef4d41>.

- [131] M. Grandini, E. Bagli and G. Vi, “Metrics for multi-class classification: an overview”., 2020.
- [132] Wikipedia, “Confusion matrix”, [Online]. Available: https://en.wikipedia.org/wiki/Confusion_matrix.
- [133] “UI Response Times”, [Online]. Available: <https://slhenty.medium.com/ui-response-times-acec744f3157>.
- [134] M. Ghanem, A. K. Ghaith, V. G. El-Hajj, A. Bhandarkar, A. de Giorgio, A. Elmi-Terander and M. Bydon, “Limitations in Evaluating Machine Learning Models for Imbalanced Binary Outcome Classification in Spine Surgery: A Systematic Review”, vol. 13, no. 12, 2023.
- [135] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves”, 2006.
- [136] Cloudflare, “Pricing plans Cloudflare”, [Online]. Available: <https://www.cloudflare.com/plans/>.
- [137] Ngrok, “Pricing plans Ngrok”, [Online]. Available: <https://ngrok.com/pricing>.
- [138] “Github issues”, [Online]. Available: <https://github.com/infinitered/insfwjs/issues/893>.
- [139] “False Positive”, Github issues (animal), [Online]. Available: <https://github.com/infinitered/insfwjs/issues/824>.
- [140] “Github issues”, [Online]. Available: <https://github.com/infinitered/insfwjs/issues/768>.
- [141] “Base64 Encoded size increase”, MDN, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Glossary/Base64>.
- [142] “What Is a VPN Tunnel?”, paloaltonetworks, [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-a-vpn-tunnel#:~:text=VPN%20tunneling%20involves%20the%20creation,address%20and%20encrypts%20their%20data>.
- [143] “How VPNs affect Internet speed”, [Online]. Available: <https://www.cloudflare.com/learning/access-management/vpn-speed/>.
- [144] “The NordVPN browser extension: Features and benefits”, NordVPN, [Online]. Available: <https://nordvpn.com/blog/nordvpn-browser-extension/>.
- [145] “How To Get Started with CUDA”, [Online]. Available: <https://wandb.ai/mostafaibrahim17/ml-articles/reports/How-To-Get-Started-with-CUDA--Vmlldzo4MTU5MDg4>.

- [146] “Canva free tier”, [Online]. Available: <https://www.canva.com/docs/#:~:text=Cloud%20storage,storage%20for%20all%20your%20designs.>
- [147] “Sampling Bias: Types, Examples & How to Avoid It”, Simply Psychology, July 2023. [Online]. Available: <https://www.simplypsychology.org/sampling-bias-types-examples-how-to-avoid-it.html>.
- [148] “Observer Bias | Definition, Examples, Prevention”, scribbr, December 2021. [Online]. Available: <https://www.scribbr.com/research-bias/observer-bias/>.
- [149] “How to Avoid the Flicker Effect in AB Testing”, Kameleoon, [Online]. Available: <https://www.kameleoon.com/blog/ab-testing-flicker-effect>.
- [150] M. Gkolemi, P. Papadopoulos, E. Markatos and N. Kourtellis, “YouTubers Not MadeForKids: Detecting channels sharing inappropriate videos targeting children”, in *Proceedings of the 14th ACM Web Science Conference 2022*, 2022.
- [151] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay and D. Mukhopadhyay, “A survey on adversarial attacks and defences”, 2021.
- [152] “OWASP Foundation”, [Online]. Available: <https://owasp.org/>.
- [153] “Local Tunneling”, [Online]. Available: <https://beeceptor.com/docs/local-tunneling-by-exposing-service-port/>.
- [154] “Vulnerability Scanning Tools”, OWASP, [Online]. Available: https://owasp.org/www-community/Vulnerability_Scanning_Tools.
- [155] “What is a DDoS attack?”, Cloudflare, [Online]. Available: <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>.
- [156] “OWASP Top 10 - 2021”, OWASP, [Online]. Available: <https://owasp.org/Top10/>.
- [157] “Evolving to a SASE with Cloudflare”, Cloudflare, [Online]. Available: <https://developers.cloudflare.com/reference-architecture/static/cloudflare-evolving-to-a-sase-architecture.pdf>.
- [158] “Agent TLS Termination”, ngrok, [Online]. Available: <https://ngrok.com/docs/agent/agent-tls-termination/>.
- [159] “Ransomware”, CISA, [Online]. Available: <https://www.cisa.gov/stopransomware/ransomware-101#:~:text=Ransomware%20is%20an%20ever%2Devolving,ransom%20in%20exchange%20for%20decryption.>

- [160] “What is Keystroke Logging and Keyloggers?”, kaspersky, [Online]. Available: <https://www.kaspersky.com/resource-center/definitions/keylogger>.
- [161] Z. Cekerevac, P. Cekerevac, L. Prigoda and F. Al-Naima, “SECURITY RISKS FROM THE MODERN MAN-IN-THE-MIDDLE ATTACKS”, vol. 13, no. 1, pp. 34-51, January 2025.
- [162] “Cyberhaven Supply Chain Attack: Exploiting Browser Extensions”, March 2025. [Online]. Available: <https://www.darktrace.com/blog/cyberhaven-supply-chain-attack-exploiting-browser-extensions>.
- [163] S. Singh, G. Varshney, T. K. Singh and V. Mishra, “A Study on Malicious Browser Extensions in 2025”, March 2025.
- [164] A. Kleymenov and A. Thabet, “Mastering malware analysis (2nd ed.)”, *Packt Publishing*, 2022.
- [165] “Security Considerations — Flask Documentation”, Pallets, [Online]. Available: <https://flask.palletsprojects.com/en/stable/web-security/>.
- [166] R. Holz, J. Amann, O. Mehani, M. Wachs and M. A. Kaafar, “TLS in the wild: an Internet-wide analysis of TLS-based protocols for electronic communication”, in *Network and Distributed System Security Symposium (NDSS)*, 2016.
- [167] E. A. Altulaihan, A. Alismail and M. Frikha, “A Survey on Web Application Penetration Testing”, *Electronics*, vol. 12, no. 5, p. 1229, 2023.
- [168] “Chrome incompatibilities”, [Online]. Available: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Chrome_incompatibilities.
- [169] S. Dey and D. Chatterjee, “A Comprehensive Review of Website Content Filtering Algorithms: Techniques, Challenges, and Future Directions”, *The Bioscan*, vol. 20, no. Supplement 2, p. 424–429, May 2025.
- [170] E. Ferrara, “Fairness and Bias in Artificial Intelligence: A Brief Survey of Sources, Impacts, and Mitigation Strategies”, vol. 6, no. 1, 2024.
- [171] V. Narnaware, A. Vayani, R. Gupta, S. Swetha and M. Shah, “SB-Bench: Stereotype Bias Benchmark for Large Multimodal Models”, February 2025.
- [172] P. P. Gosavi, V. M. Kulkarni and A. F. Smeaton, “Capturing Bias Diversity in LLMs”, October 2024.
- [173] D. V. Carvalho, E. M. Pereira and J. S. Cardoso, “Machine learning interpretability: A survey on methods and metrics”, *Electronics*, vol. 8, no. 8, p. 832, June 2019.

- [174] “The Trauma Floor: The secret lives of Facebook moderators in America”, February 2019. [Online]. Available: <https://www.theverge.com/2019/2/25/18229714/cognizant-facebook-content-moderator-interviews-trauma-working-conditions-arizona>.
- [175] I. Flores, “YouTube Is Testing Server-Side Ad Injection to Kill Ad Blockers”, June 2024. [Online]. Available: <https://www.techtimes.com/articles/305742/20240617/youtube-testing-server-side-ad-injection-kill-blockers.htm>.
- [176] “How to: Enforcing Google SafeSearch, YouTube, and Bing”, OpenDNS, [Online]. Available: <https://support.opendns.com/hc/en-us/articles/227986807-How-to-Enforcing-Google-SafeSearch-YouTube-and-Bing>.
- [177] “What is the principle of least privilege?”, October 2024. [Online]. Available: <https://www.wiz.io/academy/principle-of-least-privilege-polp>.
- [178] S. Worth, B. Snaith, A. Das, G. Thuermer and E. Simperl, “AI Data Transparency: an Exploration Through the Lens of AI Incidents”, September 2024.
- [179] “YOLO12: Attention-Centric Object Detection”, [Online]. Available: <https://docs.ultralytics.com/models/yolo12/>.
- [180] Microsoft, “Microsoft's DirectML PyTorch”, [Online]. Available: <https://github.com/microsoft/DirectML>.
- [181] M. Stephen and D. Jerry, “Machine Learning Models for Forecasting and Minimizing Cloud Operational Costs”, p. 2025.
- [182] AdGuard, “Blocklists”, [Online]. Available: <https://adguard-dns.io/kb/private-dns/setting-up-filtering/blocklists/#:~:text=Blocklists%20are%20sets%20of%20rules,rules%20with%20a%20similar%20focus>.
- [183] Mozilla, “Canvas API”, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API.
- [184] “JavaScript: The Single-Threaded Marvel and the Myth of Multi-Threading”, [Online]. Available: https://medium.com/%40the_nick_morgan/javascript-the-single-threaded-marvel-and-the-myth-of-multi-threading-c8516366252a.
- [185] MDN, “Using Web Workers”, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers.